COMPUTER GRAPHICS • RASTERIZATION PIPELINE

图形学渲染管线
从一个顶点到一个像素

一本面向游戏客户端秋招的渲染管线深度学习书。从"CPU 发出一条 Draw Call 之后 GPU 里发生了什么"讲起，沿着数据流依次拆开坐标变换、投影与裁剪、光栅化与插值、纹理采样、深度测试、混合与抗锯齿，再站到硬件视角比较桌面 IMR 和移动端 TBR，最后用"阴影 + 光照"两遍渲染的完整案例把所有阶段串起来。书中每个关键结论都配有一个可编译运行的软件光栅化器（约 1000 行 C++）的实测数据和渲染图。

6 部分 · 14 章 · 速记附录

13 组实测实验 · 26 张渲染输出

26 张原理插图 + 22 张实测渲染图

3 个交互实验

MOBA 移动端 场景贯穿

CHAPTER 00

阅读指南：一张地图串起整条管线

先看清全貌：数据从哪里来、在哪里变形、在哪里变成像素、哪些阶段能写代码、哪些阶段只能配置。

一条主线：管线就是一条"把三维数据逐步离散化"的流水线

渲染管线的知识点看起来非常散：MVP 矩阵、齐次坐标、裁剪、光栅化、重心坐标、Mipmap、Early-Z、混合、MSAA、TBR……面试时也常被拆成一道道独立的题目来问。其实它们都在回答同一个问题：

> 🧭 **全书主线**
>
> **场景是连续的（三角形有无穷多个点），屏幕是离散的（只有 W×H 个像素）。管线要做的就是：把三维空间里的连续几何，经过一系列坐标变换压到二维屏幕上，再决定"每个像素被谁覆盖、长什么颜色、最终能不能留下来"。**
>
> 每一个阶段要么在*变换坐标*（几何阶段），要么在*采样离散化*（光栅化），要么在*计算颜色*（像素阶段），要么在*决定取舍*（测试与混合）。而几乎所有的经典问题——走样、Z-Fighting、透视扭曲、Overdraw——都是"离散化"或"有限精度"带来的副作用。

读的时候遇到一个新知识点，可以问自己两句：*它处在流水线的哪一站？它是为了解决哪一种"离散化/精度/性能"问题？* 能答上这两句，这个点就不再是孤立的一道题。

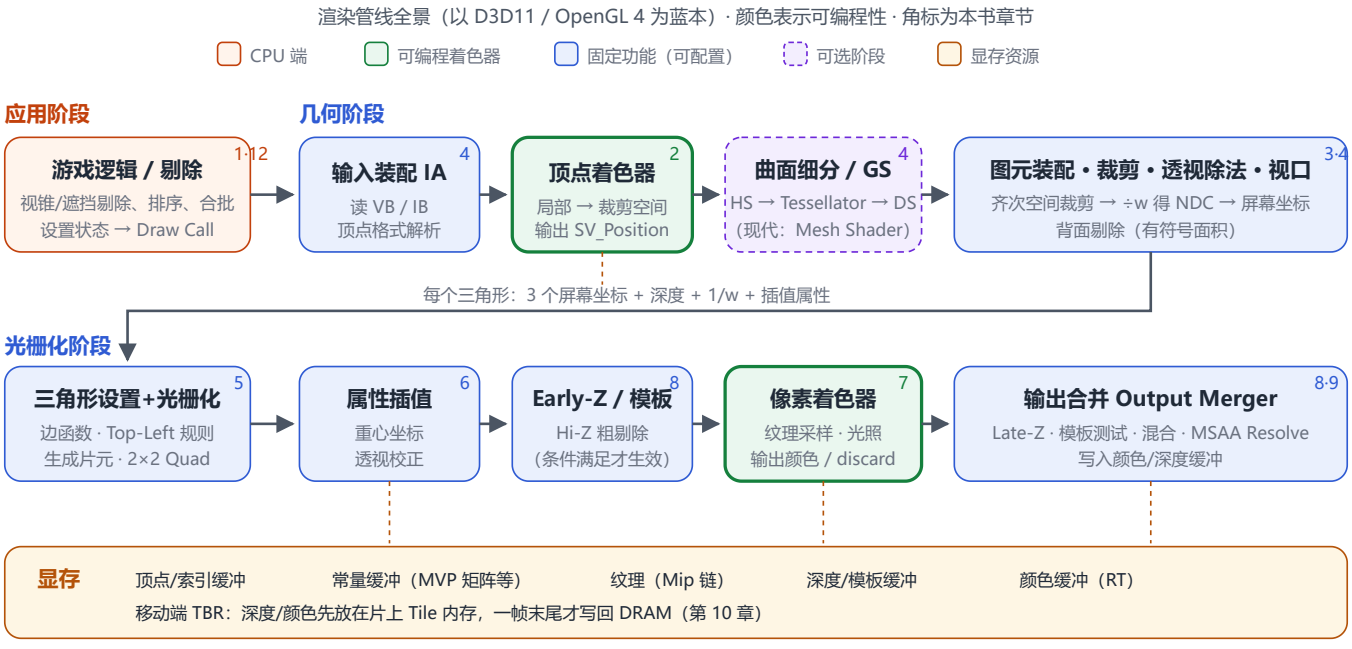


图 0-1　渲染管线全景。绿色两站是你写 Shader 的地方，蓝色站只能通过渲染状态配置。

全书结构

部分	章节	核心问题	配套实验
一 · 全景	1 一帧画面是怎么来的	CPU 和 GPU 怎么分工？Draw Call 为什么贵？GPU 为什么擅长"并行"？	—
二 · 几何阶段	2 坐标空间与变换	MVP 每一步在干什么？法线为什么要用逆转置？	E1
	3 投影与裁剪	透视矩阵怎么推？w 是什么？为什么在除以 w 之前裁剪？	E2
	4 顶点处理与图元装配	索引缓冲与顶点缓存、背面剔除、曲面细分与 Mesh Shader	E3
三 · 光栅化	5 光栅化	像素归谁？边函数、定点亚像素、Top-Left 规则	E4 · 交互
	6 属性插值	重心坐标、透视校正、2×2 Quad、ddx/ddy	E5 · E8 · 交互
四 · 像素阶段	7 片元着色与纹理	过滤、Mipmap、LOD 公式、各向异性、纹理缓存	E9
	8 深度与可见性	Early-Z 何时失效？Z-Prepass 值不值？Reversed-Z 为何更准？	E6 · E7 · 交互
	9 输出合并	模板、混合、半透明排序、预乘 Alpha、MSAA vs SSAA vs TAA	E10 · E11
五 · 架构与工程	10 GPU 硬件视角	IMR vs TBR/TBDR，带宽为什么是移动端第一敌人	E12
	11 综合案例：一帧的多个 Pass	Shadow Map 两遍渲染、深度偏移、前向/延迟/Forward+	E13
	12 性能分析与优化	瓶颈怎么定位？合批、Instancing、LOD、RenderDoc	—
六 · 面试	13 面试题库 + 附录速记卡	40 道高频题、易错说法、预留模拟面试题	—

标记约定

🔬 实测

来自配套程序 `demos/soft_raster.cpp` 的真实输出（Windows 11 + MSVC 14.44 /O2，单线程 CPU）。数字可复现，但它是*软件模拟*：用来证明"机制"，不代表某款 GPU 的绝对性能。

🎮 游戏实战

放到游戏客户端（尤其 MOBA 手游）里，这个知识点对应什么工程问题、引擎里怎么做。

⚠️ 面试易错

面试中常见的错误说法或追问陷阱，以及更准确的表述。

🧠 深入

推导或底层细节。第一遍可以跳过，准备二面/三面时再回来读。

配套程序怎么跑

软件光栅化器只依赖 C++ 标准库，一个文件实现了完整的"顶点着色 → 近平面裁剪 → 透视除法 → 定点光栅化（Top-Left）→ 透视校正插值 → Early/Late-Z → 像素着色 → 混合 → MSAA Resolve"，并在每个环节插了计数器：

编译与运行（在 demos/ 目录）

```
# MSVC（开发者命令行，或按 build_env.sh 设好 INCLUDE/LIB/PATH）
$ cl /nologo /std:c++latest /EHsc /O2 /utf-8 soft_raster.cpp
$ soft_raster.exe > run_output.txt        # 13 组实验，约 10 秒
$ python ppm2png.py                       # out/*.ppm → out/png/*.png（纯标准库）
```

书里出现的每一张渲染图都是这个程序输出的，构建电子书时以 base64 内嵌进 HTML，所以这本书是一个可以离线打开的单文件。

✅ 怎么用这本书准备面试

① 第一遍按顺序读，把图 0-1 能默画出来；② 每章末尾的"本章小结"背成口述版；③ 第 13 章的题目先自己答一遍再看参考答案；④ 被问到"你怎么证明"时，用书里的实测数字回答——**"我自己写过一个软光栅，测过 Early-Z 从前往后画能把 PS 执行次数从 599 万降到 22 万"**，比背定义有说服力得多。

PART ONE

第一部分　全景：一帧画面是怎么来的

在钻进每个阶段之前，先站远一点：CPU 和 GPU 如何分工、命令怎样从游戏逻辑一路走到显卡、GPU 为什么天生适合做这件事。后面所有的性能结论都建立在这一章的模型上。

CHAPTER 01

一帧画面是怎么来的

16.6 毫秒里，CPU 准备"画什么"，GPU 决定"每个像素长什么样"。

本章目标

- 说清一帧从游戏逻辑到屏幕显示的完整链路：游戏线程 → 渲染线程 → 驱动 → 命令缓冲 → GPU → 交换链
- 理解 Draw Call 的真实开销在哪里，以及合批 / Instancing / SRP Batcher 分别在省什么
- 理解 GPU 的 SIMT 执行模型：Warp、延迟隐藏、分支发散，以及它们对 Shader 写法的影响
- 建立"光栅化管线 = 以物体为序"的直觉，并能和光线追踪对比

1.1 两颗处理器的接力：CPU 和 GPU 怎样分工

假设你在玩一局 MOBA，屏幕上有地图、10 个英雄、小兵、技能特效和 UI。每一帧，引擎都要回答两个问题：

1. **画什么、按什么顺序画**——这是 CPU 的工作。游戏线程更新逻辑（移动、技能、动画），渲染线程据此做视锥剔除、按材质排序、合批，再把"设置状态 + 绘制"翻译成图形 API 调用。
2. **每个像素是什么颜色**——这是 GPU 的工作。它接收命令，把成千上万个三角形变换、光栅化成几百万个像素，并为每个像素运行着色器。

两者之间隔着一条**命令缓冲（Command Buffer）**：CPU 往里写命令，GPU 从里读命令，两边是异步的。CPU 调用 `Draw` 时 GPU 并不会马上开始画——命令只是被记录下来，稍后才被提交、执行。

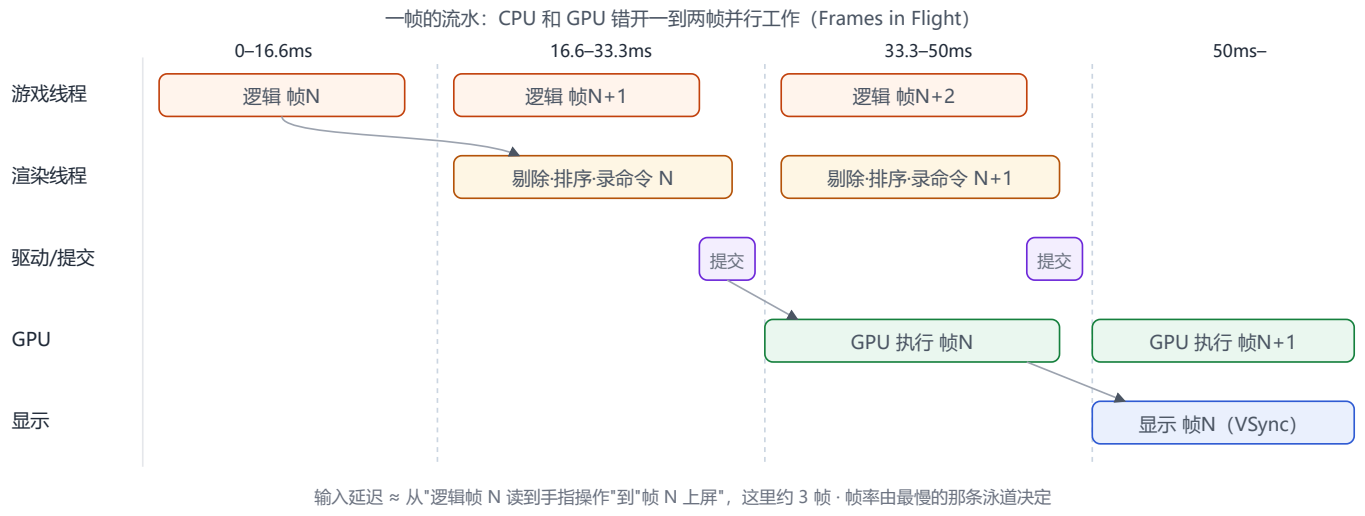


图 1-1　多线程渲染下的帧流水。CPU 领先 GPU 一到两帧是常态，这样两边都不用等对方。

这张图有两个直接的工程含义：

- **帧率取决于最慢的那条泳道。**游戏线程慢叫"逻辑瓶颈"，渲染线程/驱动慢叫"CPU 渲染瓶颈"（通常是 Draw Call 太多），GPU 慢叫"GPU 瓶颈"（顶点、填充率或带宽）。优化之前必须先知道是哪一条——第 12 章专门讲定位方法。
- **CPU 读 GPU 的结果代价极高。**如果 CPU 在帧 N+1 里同步读回帧 N 的渲染结果（比如 `ReadPixels`、遮挡查询立即取结果），就会让流水线"排空"：CPU 停下来等 GPU 画完，GPU 画完又等 CPU 发新命令。这就是为什么遮挡查询结果通常延迟一两帧使用。

1.2 Draw Call：一条命令为什么会贵

一次 Draw Call 就是 CPU 对 GPU 说"用当前的状态，画这些三角形"。真正贵的不是 `Draw` 这一下本身，而是它前面要准备的东西和驱动做的工作：

开销来源	具体是什么	典型优化
状态切换	换 Shader 程序、混合/深度状态、渲染目标；驱动可能要重新编译/链接管线状态	按材质/Shader 排序，减少 SetPass；DX12/Vulkan 预编译 PSO
资源绑定	绑定纹理、常量缓冲、顶点缓冲；上传每个物体的矩阵等常量	SRP Batcher（常量常驻 GPU，只更新变化的部分）、Bindless
驱动校验	旧 API（GL/DX11）每次调用都要检查状态合法性、跟踪资源依赖	换现代 API，命令缓冲多线程录制
提交与同步	命令缓冲刷新、用户态到内核态切换	减少 flush，批量提交

减少 Draw Call 的三种常见手段，省的东西各不相同：

静态合批

离线把共享材质的静态网格合并成一个大网格。省 Draw Call，代价是**内存变大**（每份实例的顶点都复制了一份）、无法单独剔除。

动态合批

每帧在 CPU 上把小网格的顶点变换到世界空间后拼起来。省 Draw Call，但**CPU 要做顶点变换**，只适合顶点很少的物体。

GPU Instancing

一次 Draw 画 N 份同一网格，每份的矩阵/颜色放在实例缓冲里，由顶点着色器用 `SV_InstanceID` 读取。适合草、树、小兵这种**同网格大量重复**的物体。

SRP Batcher

不减少 Draw Call 数量，而是让**每个 Draw Call 变便宜**：材质常量常驻显存，只在变化时更新，相同 Shader 变体的物体之间不再重复设置状态。

⚠️ 面试易错

"合批就是为了减少 Draw Call"只说对了一半。更准确的说法是：**合批是为了减少 CPU 端每次绘制的准备开销（主要是状态切换和常量上传）**。在 DX12/Vulkan/Metal 下单次 Draw 本身已经很便宜，真正的成本更多在 SetPass（换材质/Shader）。所以面试官常追问："SRP Batcher 不减少 Draw Call，为什么也能提速？"——答案就是它减少的是每次 Draw 之间的状态设置。

1.3 GPU 为什么快：成千上万个"慢而简单"的线程

CPU 为了让*一个*线程尽可能快，花大量晶体管做分支预测、乱序执行和大缓存；GPU 反其道而行：把晶体管堆成大量简单的计算单元，同时跑成千上万个线程，**用"线程多"来掩盖"内存慢"**。

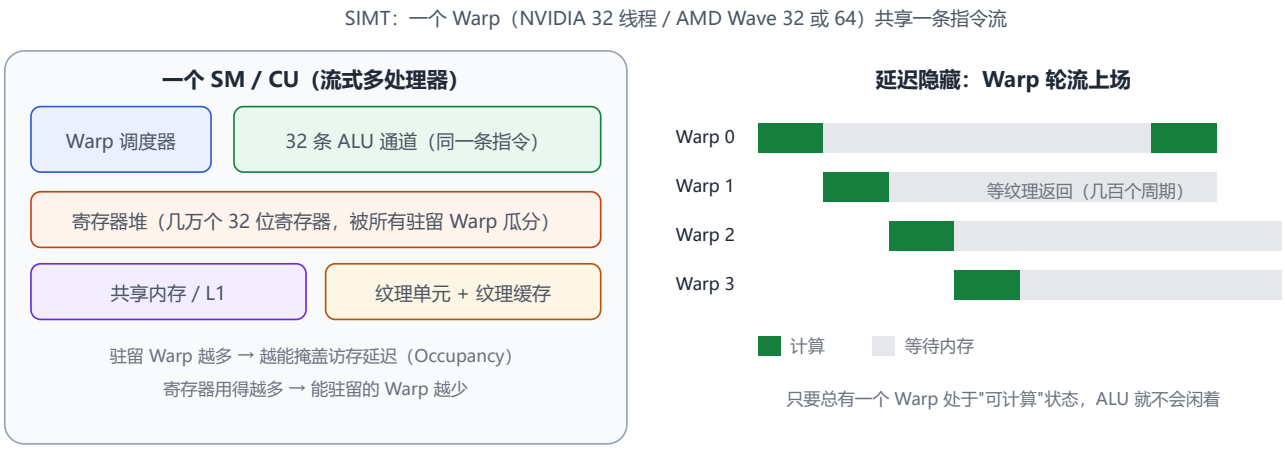


图 1-2　SIMT 执行与延迟隐藏。GPU 不靠缓存命中，而是靠"切换到别的 Warp"来掩盖访存延迟。

这个模型直接解释了 Shader 编写里的几条经验：

- **分支发散（Divergence）**：同一 Warp 的线程如果走了 if 的不同分支，硬件只能*两条分支都执行*，用掩码屏蔽不该写结果的线程。所以"按像素随机的分支"很贵，而"按材质/按 Draw Call 一致的分支"（uniform branch）几乎免费。
- **寄存器压力与 Occupancy**：一个复杂 Shader 用了很多寄存器，SM 上能同时驻留的 Warp 就变少，掩盖延迟的能力就下降。这是"Shader 变复杂，即使 ALU 不是瓶颈也变慢"的常见原因。
- **像素着色器按 2×2 Quad 打包**：一个 Warp 的 32 个线程其实是 8 个 2×2 像素块。第 6 章会看到，这是导数 `ddx/ddy` 能工作的前提，也是小三角形浪费的根源。

> 🧠 **深入：统一着色器架构**
>
> 早期 GPU（DX9 时代）有专门的"顶点单元"和"像素单元"，场景顶点多时像素单元闲着，反之亦然。从 DX10 / Xbox 360 开始，GPU 改为**统一着色器架构**：同一批计算单元按需运行顶点、像素、计算着色器。今天说"管线有 VS、PS 两个阶段"指的是*逻辑阶段*，物理上它们跑在同一批 SM 上。

1.4 可编程与固定功能：哪些能写代码，哪些只能配置

回到图 0-1，管线里的阶段分两类：

类型	阶段	你能控制什么
可编程	顶点着色器、（外壳/域/几何着色器）、像素着色器、计算着色器；新架构下的任务/网格着色器	任意代码：变换、光照、采样、discard、写深度
固定功能、可配置	输入装配、裁剪、透视除法、视口变换、背面剔除、光栅化、深度/模板测试、混合	只能设置状态：剔除模式、深度函数、混合因子、模板操作、多重采样数……

固定功能阶段之所以不开放编程，是因为它们是**极高频、极规则**的操作（每秒数十亿次的覆盖测试、深度比较），做成专用硬件比通用 ALU 快一到两个数量级、也更省电。这也解释了为什么"在 PS 里 discard"或"在 PS 里写深度"会破坏性能：你让一个本可以在固定硬件里提前完成的决定，被迫推迟到可编程阶段之后（第 8 章）。

1.5 两种画法：光栅化与光线追踪

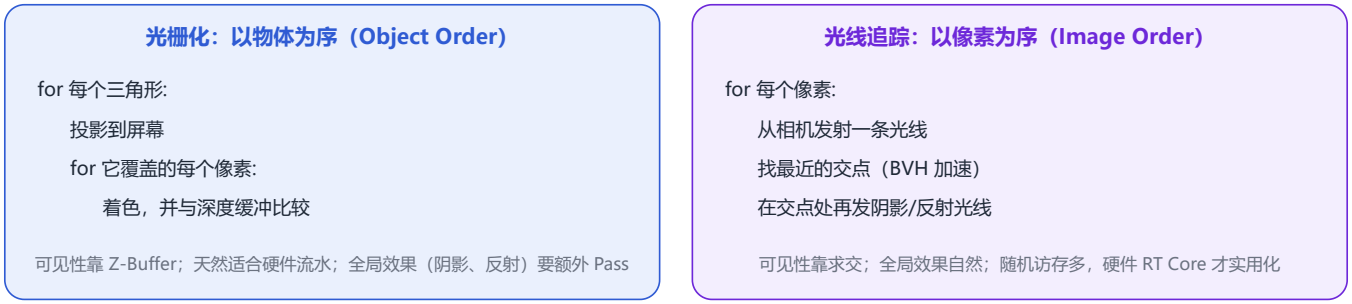


图 1-3　两种方法的循环顺序正好相反。今天的游戏几乎都是"光栅化为主 + 局部光追"的混合方案。

光栅化的核心优势是**数据访问高度规则**：一个三角形覆盖的像素在屏幕上是连续的一片，访问的纹理也在一小块区域里，缓存命中率极高；光线追踪的光线会四散到场景各处，访存随机。这正是光栅化能在手机上以 60fps 跑的原因，也是本书只讲光栅化管线的原因。

1.6 游戏实战：一局 MOBA 的一帧里有什么

以一款移动端 MOBA 为例，一帧通常由十几个 Pass 组成（不同项目差异很大，下面是一种典型结构）：

#	Pass	做什么	本书对应
1	Shadow Map	从主光源方向只写深度（英雄、小兵、建筑）	第 11 章
2	不透明物体（前向）	地形、建筑、角色；从前往后排序以利用 Early-Z	第 8 章
3	天空/背景	放在不透明之后画，大部分像素被深度测试拒掉	第 8 章
4	半透明 / 特效	技能特效、粒子；从后往前排序，大量使用加法混合	第 9 章
5	后处理	Bloom、调色、抗锯齿（FXAA 或 MSAA Resolve）	第 9 章
6	UI	血条、技能按钮、小地图；大量半透明、Overdraw 高	第 9、12 章

🎮 游戏实战：为什么 MOBA 手游这么在意 Overdraw 和带宽

MOBA 镜头是固定俯视角，屏幕中央经常同时出现十几个技能特效叠在一起，团战时半透明粒子的 Overdraw 能达到十几层；而手机 GPU 的带宽只有桌面显卡的几分之一、还要和 CPU 共享、发热就降频。所以手游渲染优化的第一优先级通常是**"少写像素、少读写显存"**，而不是"少算几条 ALU 指令"。第 10 章的 TBR 架构和第 12 章的优化清单都是从这个约束出发的。

本章小结

- CPU 决定**画什么**（剔除、排序、录制命令），GPU 决定**每个像素长什么样**；两者通过命令缓冲异步协作，通常错开 1～2 帧。
- 帧率由最慢的泳道决定；CPU 同步读 GPU 结果会让流水线排空。
- Draw Call 贵在**状态切换、资源绑定和驱动校验**；静态合批、动态合批、Instancing、SRP Batcher 省的东西各不相同。
- GPU 用 SIMT + 大量驻留 Warp 来掩盖访存延迟；分支发散和寄存器压力是 Shader 性能的两大隐形杀手。
- 固定功能阶段（光栅化、深度测试、混合）比可编程阶段快得多，破坏它们的"提前决定"会付出代价。

▶ **自测：GPU Instancing 和静态合批都能减少 Draw Call，什么情况下选哪个？**

▶ **自测：为什么说"一个 Shader 里写了 if 不一定慢"？**

PART TWO

第二部分　几何阶段：把三维世界压到屏幕上

几何阶段处理的是顶点：把模型从它自己的坐标系一路变换到屏幕上，中间要过投影、裁剪、透视除法几道关。这一部分的关键词是"空间"和"w"。

CHAPTER 02

坐标空间与变换

一个顶点要换五次"身份证"：局部、世界、观察、裁剪、屏幕。

本章目标

- 能画出 Model → View → Projection → 透视除法 → 视口变换 的完整链路，并说出每个空间的用途
- 理解齐次坐标为什么是 4 维、点和方向向量为什么一个 w=1 一个 w=0
- 能手推 LookAt 矩阵，能证明法线要用"模型矩阵的逆转置"
- 搞清行向量/列向量、行主序/列主序、左手/右手系这三对容易混淆的约定

2.1 五个空间，四次变换

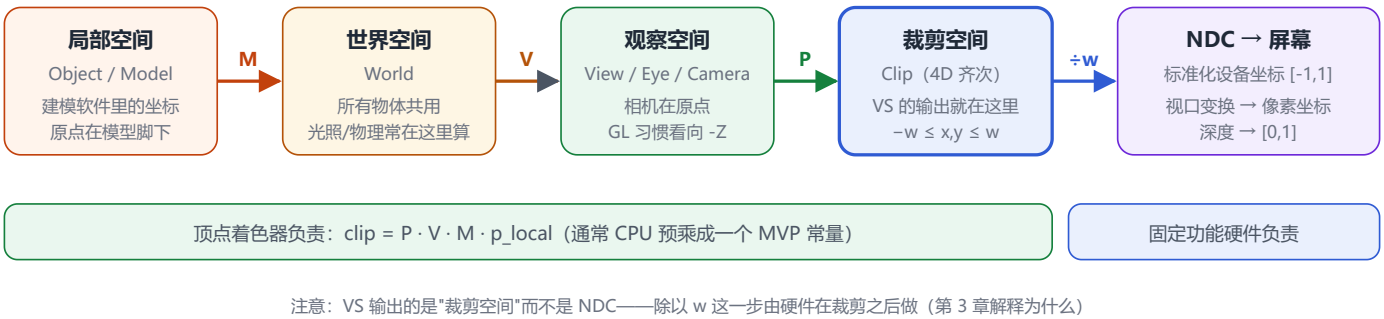


图 2-1 坐标空间链路。M、V、P 是三个 4×4 矩阵，÷w 和视口变换是固定功能。

为什么要分这么多空间，而不是一步到位？因为**每个空间都有它最方便做的事**：

- **局部空间**：美术建模、骨骼蒙皮都在这里定义，一份网格可以被摆放到场景里任意多次（实例化的前提）。
- **世界空间**：光源、碰撞、寻路都用它；光照计算常在这里做（也可以在观察空间做）。

- **观察空间**：相机固定在原点、看向固定方向，投影公式就变得很简单；深度就是 −z。
- **裁剪空间**：视锥体被变成一个"规整"的形状，判断"在不在视野里"只需要比较 `|x| ≤ w`。
- **屏幕空间**：光栅化在这里做，单位是像素。

2.2 齐次坐标：为什么用 4 个数表示 3D 点

3×3 矩阵能表示旋转和缩放，但**表示不了平移**——因为平移不是线性变换（它把原点移走了）。给坐标加一个分量 w，就能把平移写进矩阵乘法里：

$$T \cdot p = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{bmatrix}$$

这样做还有两个额外的好处：

点 vs 方向

点写成 `(x,y,z,1)`，方向（法线、光照方向、速度）写成 `(x,y,z,0)`。平移项乘以 w=0 后消失——**方向不应该被平移**，齐次坐标自动保证了这一点。

透视投影

透视的本质是"除以距离"，这不是线性运算。齐次坐标的约定是 `(x,y,z,w)` 表示 3D 点 `(x/w, y/w, z/w)`，于是投影矩阵只需要**把距离放进 w**，除法交给硬件。这是第 3 章的主角。

2.3 模型矩阵：先缩放，再旋转，最后平移

模型矩阵通常由 TRS 组合而成：`M = T · R · S`。在列向量约定下矩阵从右往左作用，所以顺序是**先缩放 → 再旋转 → 最后平移**。顺序不能换：如果先平移再旋转，物体会绕世界原点"公转"而不是绕自己的中心"自转"。

实测程序里 E1 用的模型矩阵是 `translate(0,0,-5) * rotY(30°)`，也就是"先绕自己的 Y 轴转 30°，再放到 z=-5 处"。

2.4 观察矩阵：把相机搬到原点

观察矩阵 V 是"相机在世界中的变换"的**逆**：相机往右移 1 米，等价于整个世界往左移 1 米。最常用的构造方式是 LookAt：给定相机位置 eye、目标点 at 和大致的上方向 up，先求出相机的三个正交基，再拼成旋转部分，平移部分是 −(基·eye)：

soft_raster.cpp · lookAt（右手系，相机看向 −Z）

```
static M4 lookAt(V3 eye, V3 at, V3 up) {
    V3 f = nrm(at - eye);          // 前方（forward）
    V3 s = nrm(cross(f, up));      // 右方（side）= f × up
    V3 u = cross(s, f);            // 真正的上方，保证三者正交
    M4 r = ident();
    r.m[0][0] = s.x;  r.m[0][1] = s.y;  r.m[0][2] = s.z;  r.m[0][3] = -dot(s, eye);
    r.m[1][0] = u.x;  r.m[1][1] = u.y;  r.m[1][2] = u.z;  r.m[1][3] = -dot(u, eye);
    r.m[2][0] = -f.x; r.m[2][1] = -f.y; r.m[2][2] = -f.z; r.m[2][3] = dot(f, eye);  // 前方映射
    return r;
}
```

为什么旋转部分可以直接"把基向量写成行"？因为相机的世界矩阵旋转部分是以 s、u、−f 为*列*的正交矩阵，正交矩阵的逆就是它的转置，转置后基向量就变成了*行*。这样省掉了一次通用矩阵求逆。

> ⚠️ **面试易错：up 和视线平行**
>
> 如果相机正好朝正上方或正下方看，`cross(f, up)` 的长度为 0，归一化得到 NaN，整个画面消失或闪烁。俯视角游戏（MOBA、RTS）的相机经常接近这个角度，工程上的做法是：检测到 |f·up| 接近 1 时换一个备用 up（例如 (0,0,1)），或者直接用欧拉角/四元数构造相机矩阵。

2.5 法线为什么要用逆转置矩阵

如果模型矩阵包含**非均匀缩放**（比如 x 方向拉长 2 倍），直接用 M 变换法线，法线就不再垂直于表面了：

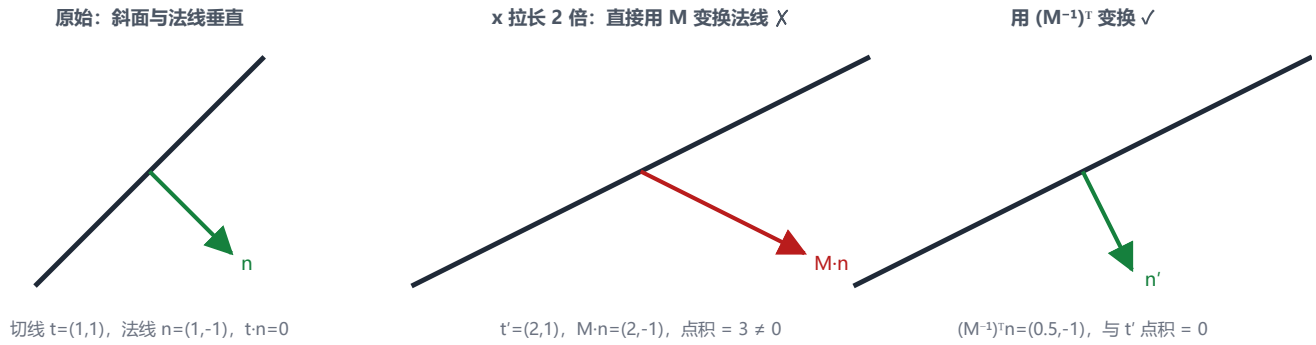


图 2-2　非均匀缩放下，法线必须用模型矩阵的逆转置变换才能保持与表面垂直。

证明只要一行。设表面切线 t 与法线 n 垂直：`nᵀt = 0`。切线跟着表面走，变换后 `t′ = M t`。要找矩阵 G 使 `n′ = G n` 仍然垂直于 t′：

$$n'^{T} t' = (G\,n)^{T} (M\,t) = n^{T} (G^{T}M)\,t = 0 \quad \Leftarrow \quad G^{T}M = I \quad \Rightarrow \quad G = (M^{-1})^{T}$$

- 如果 M 只有旋转和**均匀缩放**，`(M⁻¹)ᵀ` 与 M 只差一个标量，归一化后方向相同，可以直接用 M（实测程序 E13 里的 `stdVS` 就是这么做的）。
- 只需要取左上角 3×3（法线是方向，不受平移影响）。
- 引擎通常在 CPU 端预先算好"世界到物体"矩阵（Unity 的 `unity_WorldToObject`），Shader 里用它的转置去乘法线。

2.6 三对最容易混淆的约定

约定	两种写法	会影响什么
行向量 / 列向量	列向量：`p′ = M·p`，矩阵从右往左作用（GL、本书） 行向量：`p′ = p·M`，从左往右作用（老 D3D 教程、UE）	矩阵乘法顺序：列向量写 `P·V·M`，行向量写 `M·V·P`；两者的矩阵互为转置
行主序 / 列主序	行主序：内存里先存第一行（C/C++ 二维数组、DirectXMath 的 XMFLOAT4X4） 列主序：先存第一列（GLSL 默认；HLSL 的 cbuffer 默认也按 column_major 打包）	只影响**内存布局**，不影响数学。CPU 传矩阵给 Shader 时布局不一致，就得转置一次
左手系 / 右手系	右手系：x 右、y 上、z 指向观察者（GL、本书、Blender） 左手系：z 指向屏幕里（D3D 传统、Unity、UE 是 z 向上的左手系）	叉积方向、三角形正面绕序、投影矩阵 z 项的符号

> ⚠️ **面试易错**
>
> "行主序就是行向量"是错的，这是两件独立的事：一个是*数学约定*（向量放在矩阵左边还是右边），一个是*存储约定*（二维数组怎么摊平成一维）。遇到"矩阵传进 Shader 后物体飞了"的问题，先确认这两件事，再确认是否需要转置。

2.7 实测：跟踪一个顶点的旅程

实测程序 E1 取立方体的一个角 (0.5, 0.5, 0.5)，打印它在每个空间里的坐标，并对比三种深度约定（D3D、OpenGL、Reversed-Z）的投影结果：

```
soft_raster.exe · E1

局部坐标 (立方体一角)            : (0.500, 0.500, 0.500, 1)
世界坐标 = Model * p             : (0.683, 0.500, -4.817, 1)   [平移(0,0,-5)·绕Y转30°]
观察坐标 = View * world          : (0.683, 0.473, -7.819, 1)   [相机(0,1,3)看向(0,0,-5)，-Z 为前方]
[D3D [0,1]     ] 裁剪坐标 (0.665, 0.820, 7.7264, w=7.819) → NDC (0.0851, 0.1049, 0.988198) → 屏幕 (347.24, 161.12) 深度缓冲值 0.988198
[OpenGL [-1,1] ] 裁剪坐标 (0.665, 0.820, 7.6341, w=7.819) → NDC (0.0851, 0.1049, 0.976397) → 屏幕 (347.24, 161.12) 深度缓冲值 0.988198
[Reversed-Z    ] 裁剪坐标 (0.665, 0.820, 0.0923, w=7.819) → NDC (0.0851, 0.1049, 0.011802) →
```

```
屏幕 (347.24, 161.12) 深度缓冲值 0.011802
观察：w_clip 恰好等于 −z_view = 7.819（相机到该点沿视线方向的距离），透视除法就是"除以距离"。
```

🔬 **从输出里能读出什么**

- **w_clip = 7.819 = −z_view**：投影矩阵的第 4 行就是 (0,0,−1,0)，它把"距离"搬进了 w。
- **三种约定的 x、y、屏幕坐标完全一样**，只有 z 不同——深度约定只影响深度缓冲里存的值，不影响"画在哪里"。
- 这个点离相机 7.8 米，在 near=0.1、far=100 下，标准 D3D 深度已经是 **0.988**——只剩不到 1.2% 的深度范围留给 7.8～100 米。这就是第 8 章深度精度问题的伏笔；Reversed-Z 下它是 0.0118，靠近 0 的地方浮点数最密。
- OpenGL 的 NDC 深度是 0.976，经 `(z+1)/2` 映射后存入缓冲的值与 D3D 完全相同。

2.8 游戏实战：大世界的浮点精度

float 只有 24 位有效数字。在距离原点 10 公里的地方，float 能表示的最小间隔约为 1 毫米；到 100 公里就是 8 毫米左右。如果顶点着色器里先算 `world = M·p` 再算 `V·world`，两个很大的数相减（物体位置 − 相机位置）会丢掉大量精度，表现为远离原点时**模型抖动、顶点颤动**。

🎮 **游戏实战：相机相对渲染（Camera-Relative Rendering）**

做法是在 CPU 上用 double 计算 `物体位置 − 相机位置`，把结果（一个不大的数）放进模型矩阵，然后观察矩阵里只保留旋转、不带平移。这样 GPU 上所有坐标都是"相对相机"的小数值。UE5 的大世界坐标（LWC）、Unity HDRP 的 Camera Relative Rendering 都是这个思路。MOBA 地图只有一两百米见方，一般遇不到这个问题，但开放世界、飞行类游戏必须处理。

本章小结

- 链路：**局部 →(M) 世界 →(V) 观察 →(P) 裁剪 →(÷w) NDC →(视口) 屏幕**；VS 只负责到裁剪空间。
- 齐次坐标让平移和透视都能用矩阵表达；点 w=1、方向 w=0。
- TRS 顺序是先缩放、再旋转、后平移（列向量约定下写作 `T·R·S`）。
- 观察矩阵是相机世界矩阵的逆，LookAt 利用正交矩阵"逆 = 转置"直接拼出来。

- 法线变换用 $(M^{-1})^T$，因为要保持 $n^T t=0$；只有旋转+均匀缩放时可以直接用 M。
- 行/列向量是数学约定，行/列主序是存储约定，两者无关。

▶ **自测：骨骼蒙皮在哪个空间、哪个阶段做？**

▶ **自测：为什么 W 分量对方向向量是 0？如果你不小心传了 1 会怎样？**

CHAPTER 03

投影与裁剪

投影矩阵把"距离"藏进 w；裁剪必须赶在除以 w 之前完成。

本章目标

- 能从相似三角形出发推导透视投影矩阵，说清每一项的来历
- 理解 NDC 深度是距离的**倒数**的线性函数——这是透视校正和深度精度问题的共同根源
- 说清为什么要在齐次裁剪空间裁剪，以及 w<0、w=0 会发生什么
- 了解保护带（Guard Band）：为什么 GPU 实际上几乎只裁近平面
- 掌握 D3D / OpenGL / Vulkan / Metal 在 NDC 和视口上的差异

3.1 视锥体：正交投影与透视投影

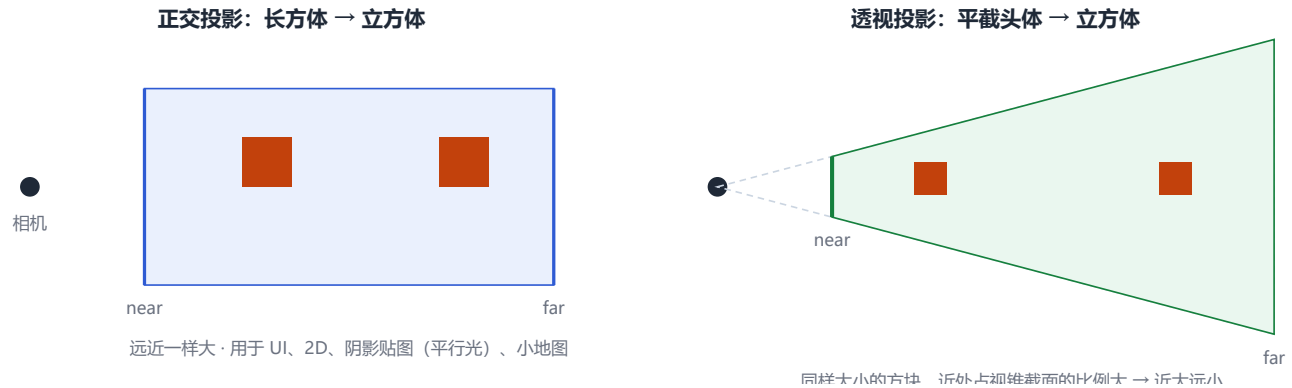


图 3-1　两种投影的视锥体。投影矩阵的任务就是把它"捏"成标准立方体（NDC）。

3.2 推导透视投影矩阵

在观察空间里，相机在原点、看向 −Z。一个点 (x, y, z) 投影到距离为 n 的近平面上，由相似三角形：

$$x_{proj} = n \cdot x / (-z) \qquad y_{proj} = n \cdot y / (-z)$$

除以 −z 不是线性运算，矩阵做不到。办法是：**让矩阵输出 w = −z，把除法推迟给硬件**。所以投影矩阵的第 4 行是 `(0, 0, -1, 0)`。

x 和 y 再乘一个缩放，使视野边缘落到 ±1：设垂直视场角为 fovy、宽高比为 aspect，令 `t = 1 / tan(fovy/2)`，则第 1、2 行是 `(t/aspect, 0, 0, 0)` 和 `(0, t, 0, 0)`。

最后是 z。我们也需要深度用于遮挡判断，而且希望它在除以 w 之后落在固定区间。z 只能写成 $z_{clip} =$ `A·z + B`（第 3 行是 `(0,0,A,B)`），于是：

$$z_{ndc} = (A \cdot z + B) / (-z) = -A - B / z \qquad \text{令 } z{=}-n \Rightarrow 0,\ z{=}-f \Rightarrow 1\ (\text{D3D 约定})$$

解这两个方程得 `A = f/(n-f)`，`B = n·f/(n-f)`。整理得到右手系、深度范围 [0,1] 的透视矩阵（实测程序 `perspective()` 的 `DM_D3D` 分支）：

$$P = \begin{bmatrix} t/\text{aspect} & 0 & 0 & 0 \\ 0 & t & 0 & 0 \\ 0 & 0 & f/(n-f) & nf/(n-f) \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

soft_raster.cpp · 四种深度约定的投影矩阵（只有第 3 行不同）

```
static M4 perspective(float fovyDeg, float aspect, float n, float f, int mode) {
    M4 r{};
    float t = 1.f / tanf(fovyDeg * (float)M_PI / 360.f);
    r.m[0][0] = t / aspect; r.m[1][1] = t; r.m[3][2] = -1;  // w_clip = -z_view
    switch (mode) {
    case DM_D3D:    r.m[2][2] = f / (n - f);       r.m[2][3] = n * f / (n - f); break;  // nea
    case DM_REV:    r.m[2][2] = n / (f - n);       r.m[2][3] = n * f / (f - n); break;  // nea
    case DM_GL:     r.m[2][2] = (f + n) / (n - f); r.m[2][3] = 2 * f * n / (n - f); break;  //
    case DM_REVINF: r.m[2][2] = 0;                 r.m[2][3] = n; break;  // far→∞, z_ndc = n
    }
    return r;
}
```

3.3 一个关键事实：NDC 深度是"距离倒数"的线性函数

把上面的 z_{ndc} 换成距离 d = −z：

$$z_{ndc}(d) = -A + B/d \quad = \quad a + b \cdot (1/d)$$

深度值和 `1/d` 成线性关系，而不是和 d 成线性关系。这个看似不起眼的事实，在后面会被引用三次：

① 深度可以在屏幕空间线性插值

在屏幕空间里，`1/w` 是线性变化的（第 6 章证明），所以 z_{ndc} 也是。硬件可以直接用重心坐标线性插值深度，不需要透视校正。

② 属性插值必须做透视校正

反过来，uv、颜色这些属性是在三维空间里线性分布的，在屏幕空间就不是线性的，必须借助 `1/w` 校正（第 6 章 E5）。

③ 深度精度极不均匀

1/d 在近处变化剧烈、远处几乎不变，深度值大量集中在近平面附近。E6 实测：near=0.1 时，**一半的深度值花在了前 0.2 米**（第 8 章）。

3.4 为什么必须在除以 w 之前裁剪

直觉上，先除以 w 得到 NDC，再把超出 [−1,1] 的部分切掉，似乎更简单。但这在有顶点位于相机**背后**时会出大问题：

- 相机背后的点 z>0，于是 **w = −z < 0**。除以负数会让 x、y 变号——点被"翻"到屏幕另一侧。
- 恰好在相机平面上的点 w = 0，除法直接得到无穷大。

实测程序 E2 构造了这样一个三角形：A、B 在相机前方 2 米，C 在相机背后 1 米（想象一块从你脚下延伸到前方的地面）：

```
soft_raster.exe · E2

顶点A 观察空间(-1.00,-0.50,-2.00)  clip=(-0.974,-0.866,  1.902, w= 2.000)  朴素除法→屏幕(   164.1,   257.9)
顶点B 观察空间( 1.00,-0.50,-2.00)  clip=( 0.974,-0.866,  1.902, w= 2.000)  朴素除法→屏幕(   475.9,   257.9)
顶点C 观察空间( 0.50,-0.50, 1.00)  clip=( 0.487,-0.866, -1.101, w=-1.000)  朴素除法→屏幕(   164.
```

```
1,    24.1)  ⟵ w<0：被翻到屏幕另一侧！
近平面裁剪后得到 4 个顶点（三角形变成四边形 → 扇形拆成 2 个三角形）：
  P0 观察空间(-1.000,-0.500,-2.000)  w=2.000  屏幕(  164.1,  257.9)  深度 0.9510
  P1 观察空间( 1.000,-0.500,-2.000)  w=2.000  屏幕(  475.9,  257.9)  深度 0.9510
  P2 观察空间( 0.683,-0.500,-0.100)  w=0.100  屏幕( 2450.4, 1738.8)  深度 0.0000
  P3 观察空间(-0.050,-0.500,-0.100)  w=0.100  屏幕(  164.1, 1738.8)  深度 0.0000
实际光栅化：输入 1 个三角形，被裁剪 1 个，送入光栅化 2 个，覆盖 38760 个像素（地面从脚下一直铺到远处）。
```

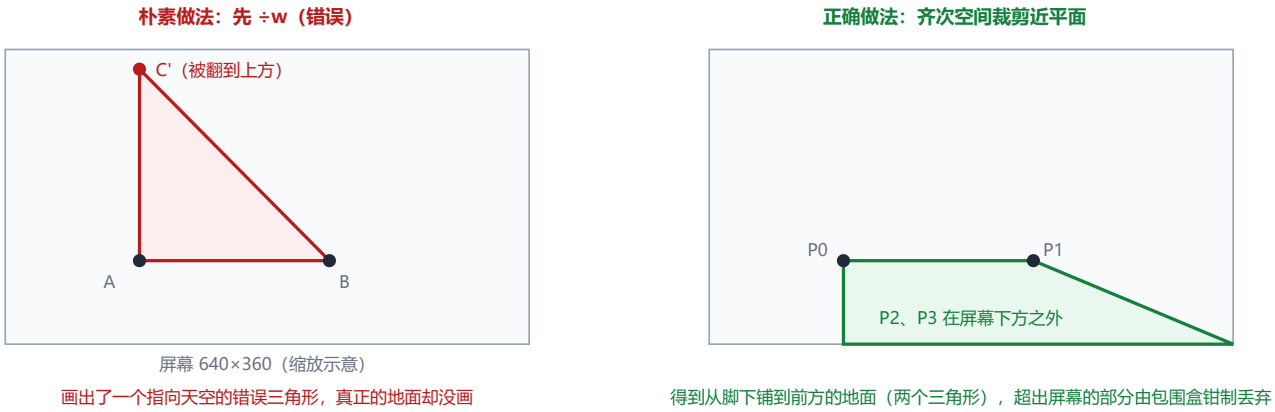


图 3-2　E2 的两种处理方式。裁剪必须发生在 w 还"有符号"的齐次空间里。

在齐次空间里，"点在视锥内"可以写成 6 个不需要除法的不等式：

$$-w \le x \le w \qquad -w \le y \le w \qquad 0 \le z \le w \text{（D3D/Vulkan/Metal）或 } -w \le z \le w \text{（OpenGL）}$$

每个不等式都是 4D 空间里的一个"平面"，裁剪就是用 Sutherland–Hodgman 算法依次用平面切多边形：保留在内侧的顶点，对跨越平面的边按有符号距离插值出新顶点。一个三角形被一个平面切，最多变成四边形；被 6 个平面切，最多变成 9 边形，再拆成扇形三角形送去光栅化。

soft_raster.cpp · 近平面裁剪（Sutherland–Hodgman，单平面）

```
static float nearDist(int mode, const V4& c) {        // 有符号距离：≥0 在内侧
    switch (mode) { case DM_D3D: return c.z;          // 0 ≤ z
                    case DM_GL:  return c.z + c.w;    // -w ≤ z
                    default:     return c.w - c.z; } // Reversed-Z：z ≤ w
}
static int clipNear(const State& s, const VOut* in, VOut* out, int na) {
    int n = 0;
    for (int i = 0; i < 3; i++) {
        const VOut& a = in[i]; const VOut& b = in[(i + 1) % 3];
        float da = nearDist(s.clip, a.c), db = nearDist(s.clip, b.c);
        if (da ≥ 0) out[n++] = a;                                          // a 在内侧，保留
        if ((da ≥ 0) ≠ (db ≥ 0)) out[n++] = lerpV(a, b, da / (da - db), na);  // 边跨越平面，
    }
    return n;   // 0、3 或 4 个顶点
}
```

深入：为什么在 4D 里线性插值是对的

裁剪空间到观察空间之间只差一个**线性**变换（投影矩阵可逆），所以观察空间里的直线在裁剪空间里仍是直线，在裁剪空间里对 (x,y,z,w) 和所有属性按同一个 t 线性插值，得到的就是三维空间中那条边上的真实点。一旦除以 w，这条性质就没有了——这又是"必须在除法之前裁剪"的一个理由。

3.5 保护带：GPU 其实很少真的裁 x/y

注意 E2 输出里的 P2：屏幕坐标 (2450, 1739)，远远超出 640×360 的屏幕。实测程序并没有对 x/y 平面做裁剪，而是让光栅化器**直接遍历被钳制到屏幕范围内的包围盒**——超出屏幕的像素根本不会被访问。

真实 GPU 也是这么做的，叫**保护带（Guard Band）**：光栅化器的定点坐标范围远大于屏幕（例如 ±32K 像素），只要三角形落在这个大范围内，就不需要真正生成新顶点，靠 Scissor 和包围盒丢掉屏幕外的部分即可。真正必须裁剪的只有：

- **近平面**：因为 w≤0 会导致除法翻转或除零，保护带帮不了；
- 超出保护带范围的极大三角形（定点数会溢出）；
- 远平面（可选，D3D 有 DepthClipEnable 开关；关掉时叫"深度钳制"，阴影渲染常用来避免投射体被远/近平面切掉）。

✅ 面试答法

问"裁剪是怎么做的"，可以分三层答：①**在哪**：在齐次裁剪空间、透视除法之前；②**为什么**：避免 w≤0 导致的翻转和除零，且 4D 空间插值保持线性；③**实际硬件**：x/y 依赖保护带 + 包围盒钳制，真正生成新顶点的只有近平面（以及超出保护带的情况），因为裁剪会改变图元数量、打断流水，硬件尽量避免。

3.6 视口变换与各 API 的差异

透视除法后得到 NDC，再由视口变换映射到像素坐标。以 W×H 的屏幕、左上角为原点为例：

$$x_{screen} = (x_{ndc} + 1) / 2 \cdot W \qquad y_{screen} = (1 - y_{ndc}) / 2 \cdot H \qquad depth = z_{ndc}\ (D3D)\ 或\ (z_{ndc}+1)/2\ (GL)$$

API	NDC 深度范围	NDC y 方向	屏幕原点	备注
Direct3D 11/12	[0, 1]	向上	左上	视口变换里翻转 y
OpenGL	[−1, 1]	向上	左下	可用 `glClipControl` 改成 [0,1]（GL 4.5）
Vulkan	[0, 1]	**向下**	左上	从 GL 移植要翻 y（负视口高度或改投影矩阵）
Metal	[0, 1]	向上	左上	纹理坐标原点在左上

⚠️ 面试易错：渲染到纹理后画面上下颠倒

这是跨平台引擎最常见的问题之一：GL 的纹理坐标原点在左下，D3D/Metal/Vulkan 在左上。Unity 用 `_ProjectionParams.x` 和 `UNITY_UV_STARTS_AT_TOP` 宏来处理；自己写引擎时要统一一个约定，并在"采样渲染目标"这条路径上集中做翻转。

3.7 游戏实战：近平面和 FOV 怎么设

近平面不能太小

近平面不能太大

深度精度主要由 near 决定（第 8 章）：E6 实测中 near 从 0.1 改成 1.0，1000 米处的 Z-Fighting 从 30.5% 降到 0%。第一人称游戏常设 0.01 ~ 0.1，第三人称/俯视角可以设到 0.3 ~ 1。

太大时相机贴近墙壁或角色会被"切开"（穿模）。第一人称的手臂/武器经常用单独的相机或单独的投影矩阵渲染。

宽屏适配

固定垂直 FOV、让水平 FOV 随宽高比变大（Hor+），宽屏玩家能看到更多。MOBA 这类竞技游戏会限制最大视野，避免超宽屏获得信息优势。

正交相机

UI、小地图、平行光阴影都用正交投影。正交投影 w 恒为 1，深度与距离线性相关，精度均匀，不需要透视校正。

本章小结

- 透视投影矩阵用第 4 行 `(0,0,-1,0)` 把**距离放进 w**，除法由硬件完成；x、y 由 fovy 和 aspect 缩放，z 的两个系数由"near→0、far→1"解出。
- NDC 深度是 **1/d 的线性函数**：所以深度能在屏幕空间线性插值、属性插值需要透视校正、深度精度集中在近处。
- 裁剪必须在**齐次空间、除以 w 之前**：避免 w<0 翻转和 w=0 除零；4D 中线性插值仍正确。
- 硬件依赖**保护带**，x/y 基本不真正裁剪，近平面才必须裁。
- D3D/Vulkan/Metal 深度 [0,1]，GL 默认 [−1,1]；Vulkan NDC y 向下。

▶ **自测：正交投影需要透视除法吗？深度精度有什么特点？**

▶ **自测：一个三角形三个顶点都在视锥外，它一定看不见吗？**

CHAPTER 04

顶点处理与图元装配

顶点从显存被读出、被着色、被组装成三角形，然后先过一道"背面剔除"的筛子。

本章目标

- 理解顶点缓冲、索引缓冲和 Post-Transform 顶点缓存，知道为什么索引能省顶点着色
- 知道曲面细分、几何着色器、Mesh Shader 分别解决什么问题，为什么移动端很少用前两者
- 掌握背面剔除的实际实现（屏幕空间有符号面积），并用实测数据说明它到底剔掉多少
- 建立"剔除漏斗"：从 CPU 视锥剔除到 GPU 零面积剔除，每一层在哪里、省什么

4.1 输入装配：从显存里读出顶点

一个网格在显存里通常由两块数据组成：

- **顶点缓冲（VB）**：每个顶点的属性——位置、法线、切线、uv、颜色、骨骼索引和权重等。布局可以*交错*（一个顶点的所有属性挨着放，AoS）也可以*分流*（每种属性一个流，SoA）。只画深度的 Pass 只需要位置，把位置单独放一个流可以减少带宽——这是很多引擎把 Position 单独成流的原因。
- **索引缓冲（IB）**：每 3 个索引组成一个三角形（Triangle List）。16 位索引最多引用 65536 个顶点，大网格要用 32 位或拆成多段。

输入装配（IA）阶段按照"输入布局"（Input Layout / Vertex Descriptor）解析这两块数据，把顶点送进顶点着色器。

4.2 顶点着色器与顶点缓存：为什么要用索引

顶点着色器对**每个顶点**运行一次，最少要输出裁剪空间位置 `SV_Position`，以及要传给像素着色器的插值属性（varyings）。

一个最小的顶点着色器（HLSL）

```
cbuffer PerObject { float4x4 _MVP; float4x4 _ObjectToWorld; float4x4 _WorldToObject; };
struct VSIn  { float3 pos : POSITION; float3 nrm : NORMAL; float2 uv : TEXCOORD0; };
struct VSOut { float4 posCS : SV_Position; float3 nrmWS : TEXCOORD0; float2 uv : TEXCOORD1; };

VSOut VS(VSIn v) {
    VSOut o;
    o.posCS = mul(_MVP, float4(v.pos, 1));                    // 裁剪空间，交给硬件裁剪与
    o.nrmWS = normalize(mul(v.nrm, (float3x3)_WorldToObject)); // 行向量乘 = 乘以逆矩阵的转
    o.uv    = v.uv;
    return o;
}
```

一个封闭网格里，每个顶点平均被 5～6 个三角形共享。如果直接用三角形列表把每个三角形的 3 个顶点都存一份，顶点着色器要做 3 倍于"顶点数"的工作。索引缓冲配合 GPU 的**Post-Transform 顶点缓存**解决这个问题：GPU 记住最近处理过的若干个顶点索引的输出，遇到重复索引直接复用结果。

网格（实测程序生成）	顶点数	三角形数	索引数	不用索引时 VS 次数	理想缓存下 VS 次数
立方体（每面 4 顶点，法线不共享）	24	12	36	36	24
UV 球 64×32	2145	3968	11904	11904	2145（省 82%）

衡量缓存效率的指标叫 **ACMR**（平均每个三角形的缓存未命中数）：理想的规则网格接近 0.5，完全随机的索引顺序接近 3。离线的网格优化工具（如 meshoptimizer、Forsyth 算法）会重排三角形顺序，让共享顶点的三角形挨在一起提交。

> ⚠️ **面试易错：立方体为什么是 24 个顶点而不是 8 个**
>
> "顶点"在 GPU 眼里是**所有属性的组合**，而不只是位置。立方体的一个角被三个面共享，但三个面的法线（和 uv）不同，所以要拆成 3 个顶点：8×3=24。同理，模型的 UV 接缝、硬边（法线不连续处）都会让顶点数增加。美术报给你的"面数"和引擎统计的"顶点数"差很多，原因就在这里。

4.3 可选的几何阶段：曲面细分、几何着色器、Mesh Shader

阶段	能做什么	代价/现状
曲面细分（Hull Shader → Tessellator → Domain Shader）	在 GPU 上把粗网格细分成细网格，配合位移贴图做地形、海面；按距离自适应细分	小三角形会让光栅化和 Quad 效率急剧下降（第 6 章 E8）；移动端支持差、功耗高
几何着色器（GS）	以图元为单位输入，可以输出 0～N 个新图元：广告牌扩展、阴影体、单 Pass 渲染立方体贴图	输出数量不定，硬件难以并行，**普遍很慢**；Metal 根本不支持。现代做法改用计算着色器或 Mesh Shader
任务/网格着色器（Task/Amplification + Mesh Shader）	用类似计算着色器的方式直接生成"小块网格"（Meshlet，约 64 顶点 / 126 三角形），并在 GPU 上逐 Meshlet 做视锥/背面/遮挡剔除	DX12 Ultimate / Vulkan 扩展 / Metal 3；UE5 Nanite 思路的硬件化。替代 IA + VS + GS

4.4 图元装配与背面剔除

顶点着色完毕，经过裁剪和透视除法，3 个顶点被组装成一个屏幕空间三角形。下一步是**背面剔除**：一个封闭物体背对相机的那一半表面一定被正面挡住，没必要光栅化。

"背面"怎么判定？约定从外面看三角形顶点是**逆时针**（CCW，GL/本书默认）或**顺时针**（CW，D3D 默认）排列的为正面。投影到屏幕后，背面三角形的绕序会反过来——所以只需要计算屏幕空间的**有符号面积**：

$$2A = (x_1 - x_0)(y_2 - y_0) - (y_1 - y_0)(x_2 - x_0) \qquad \text{符号决定正反面，} A = 0 \text{ 为退化三角形}$$

soft_raster.cpp · rasterTri 开头：用定点坐标算有符号面积，做背面剔除

```
i64 area = (X[1] - X[0]) * (Y[2] - Y[0]) - (Y[1] - Y[0]) * (X[2] - X[0]);
if (area == 0) { T.st.degenerate++; return; }        // 零面积（包括被吸附到同一点的微小三角形）
bool front = area < 0;  // NDC 中逆时针 = 屏幕(y 向下)中有符号面积为负
if (s.cull && !front) { T.st.culled++; return; }
```

为什么不用"法线点乘视线"判断？在透视投影下，同一个三角形上不同点看过去的视线方向不同，用某一个点的视线判断可能出错；而屏幕空间面积的符号是**精确**的（它就是"投影之后这个三角形朝哪边"），而

且光栅化本来就要算这个面积（第 5 章的边函数），几乎零额外开销。

实测：背面剔除到底剔掉多少

常听到"背面剔除能剔掉一半三角形"。E3 用一个 64×32 的 UV 球，从不同距离观察：

```
soft_raster.exe · E3

球体三角形数：3968
相机距离  输入      剔除      退化      剔除率   理论 1-(1-r/d)/2
1.5       3968      3448      0          86.9%          83.3%
3.0       3968      2869      0          72.3%          66.7%
10.0      3968      2300      0          58.0%          55.0%
100.0     3968      2002      0          50.5%          50.5%
```

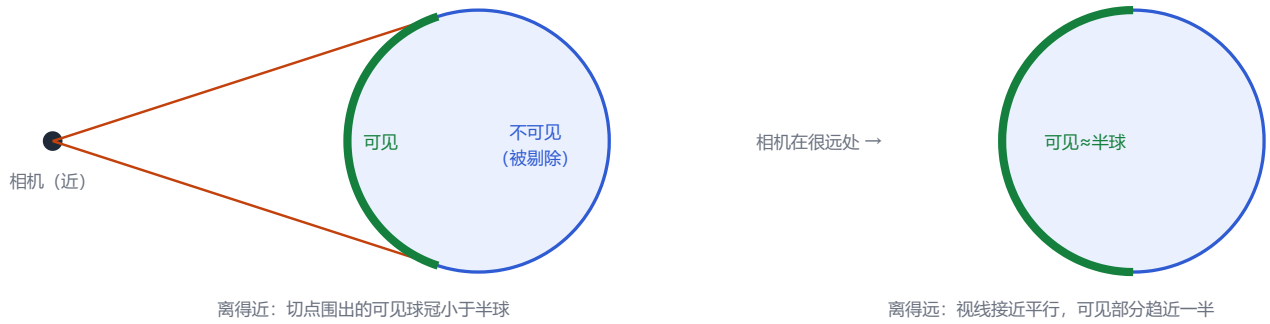


图 4-1　从距离 d 看半径 r 的球，可见球冠的面积占比为 (1 − r/d)/2。

🔬 从输出里能读出什么

- 背面剔除率**不是固定 50%**：相机离凸物体越近，剔除率越高（1.5 倍半径处达到 86.9%）；远处趋近 50%。
- 实测略高于理论值，因为网格是离散的：轮廓附近的三角形只要中心略偏到背面就整体被剔除；而且 UV 球两极的三角形更小更密。
- 背面剔除省的是**光栅化和像素着色**，不省顶点着色——顶点在剔除之前已经算完了。要省顶点着色，必须在更早的阶段剔除（CPU 视锥剔除、Mesh Shader 的 Meshlet 法线锥剔除）。

🎮 游戏实战：哪些东西要关背面剔除

树叶、草、旗帜、披风这类"单片"模型需要双面渲染（Cull Off），而它们背面的法线是反的——像素着色器里要用 `SV_IsFrontFace`（GLSL 的 `gl_FrontFacing`）把法线翻过来，否则背面光照是错的。另一个常见例外是阴影 Pass：有的引擎会改成剔除正面（Cull Front），只让背面写入阴影贴图，以减少自阴影痤疮（第 11 章）。

4.5 剔除漏斗：每一层在哪里、省什么

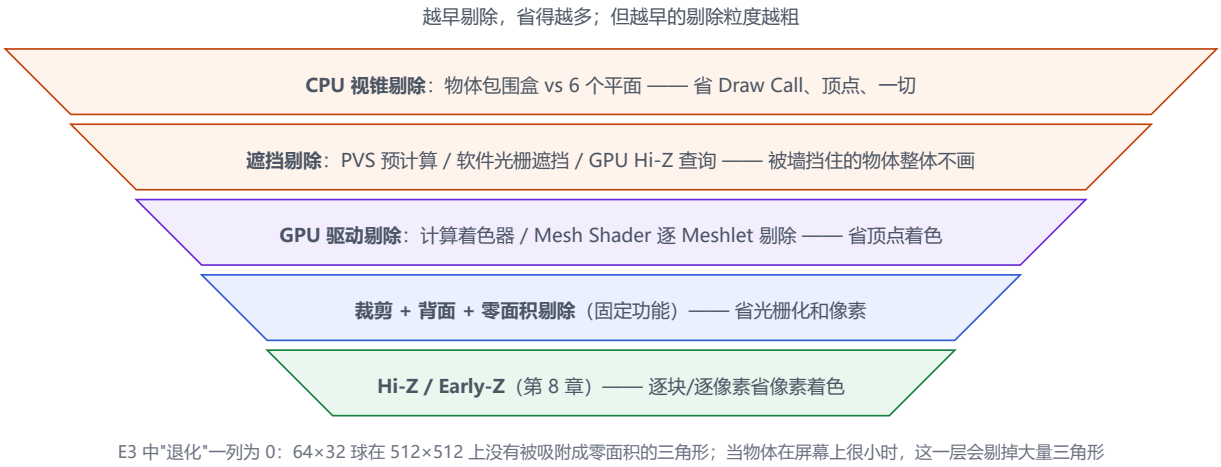


图 4-2　剔除漏斗。背面剔除只是最后几道筛子之一。

4.6 游戏实战：MOBA 里的顶点预算

- **小兵和野怪**：同一模型大量重复，适合 GPU Instancing；骨骼动画则用 *GPU 蒙皮*或把动画烘焙进纹理（Vertex Animation Texture / 骨骼矩阵纹理），让 Instancing 不受"每个实例动画不同"的限制。
- **顶点格式压缩**：uv 和颜色用 half 或 UNorm8；法线用八面体编码压到 2 个 16 位数；骨骼权重用 4×UNorm8。顶点带宽是移动端带宽的重要组成部分。
- **LOD**：MOBA 镜头距离固定，LOD 切换主要发生在"技能镜头/回城动画"之类的特写；更多的收益来自控制同屏最大顶点数（例如英雄 5k～1 万面、小兵 1～2 千面量级，具体看项目）。

本章小结

- 顶点缓冲存属性，索引缓冲存三角形；Post-Transform 缓存让共享顶点只着色一次（64×32 球省 82% 的 VS 调用）。
- "顶点"是所有属性的组合：硬边、UV 接缝都会复制顶点（立方体 24 个顶点）。
- 几何着色器普遍慢；曲面细分会产生小三角形；Mesh Shader 是现代替代方案。
- 背面剔除靠**屏幕空间有符号面积**判断，精确且几乎免费；剔除率随距离变化，近处可达 80% 以上，远处趋近 50%。
- 背面剔除不省顶点着色；越早的剔除（CPU 视锥、遮挡、GPU 驱动）省得越多。

▶ **自测：模型导入后有一半的面"消失"了，可能是什么原因?**

▶ **自测：为什么几何着色器慢，而 Mesh Shader 快?**

PART THREE

第三部分　光栅化：连续的三角形变成离散的像素

几何阶段输出的是三个浮点坐标，像素阶段需要的是"哪些像素被覆盖、每个像素的属性是多少"。这一步是整条管线最"离散"的地方，也是走样、裂缝、精度问题的发源地。

CHAPTER 05

光栅化：像素归谁

三条边函数、一个定点网格、一条 Top-Left 规则，保证相邻三角形不漏一个像素、也不多画一个像素。

本章目标

- 理解"覆盖"的定义：像素中心（采样点）是否落在三角形内
- 掌握边函数：几何意义、增量计算、与重心坐标的关系
- 说清为什么 GPU 要把顶点吸附到定点亚像素网格上，以及 Top-Left 填充规则解决什么问题
- 了解包围盒遍历、分块遍历和层次化光栅化
- 能在交互实验里亲手观察覆盖、Top-Left 和 2×2 Quad

5.1 光栅化在回答什么问题

光栅化的输入是一个屏幕空间三角形（三个顶点的浮点坐标），输出是一组**片元（Fragment）**——"这个像素被这个三角形覆盖了，它的深度和插值属性是多少"。

"覆盖"的标准定义是：**像素的采样点落在三角形内部。**不开 MSAA 时，每个像素只有一个采样点，就在像素中心 `(x+0.5, y+0.5)`。三角形哪怕盖住了一个像素 49% 的面积，只要没盖住中心，这个像素就不算覆盖——这就是锯齿（几何走样）的来源。

5.2 边函数：用三个半平面圈出三角形

对有向边 a→b 和任意点 p，定义边函数：

$$E_{ab}(p) = (b.x - a.x)(p.y - a.y) - (b.y - a.y)(p.x - a.x)$$

它是向量 ab 与 ap 的二维叉积，有三层含义：

① 半平面测试

E > 0 表示 p 在边的一侧，E < 0 在另一侧，E = 0 恰好在边上。三条边的 E 同号 ⇔ p 在三角形内。

② 面积

$E_{ab}(p)$ 等于三角形 abp 面积的 2 倍（有符号）。把 p 换成第三个顶点 c，就得到整个三角形的 2 倍面积——也就是第 4 章背面剔除用的那个数。

③ 重心坐标

$E_{bc}(p)/E_{bc}(a)$ 正是 p 相对顶点 a 的重心坐标 λ_a。光栅化算出来的边函数值，插值时直接复用（第 6 章）。

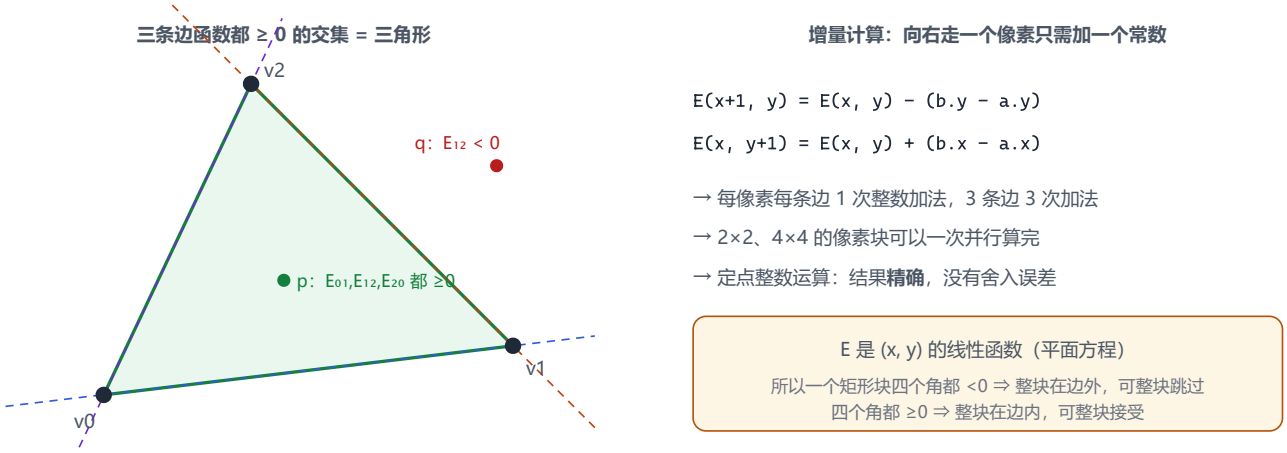


图 5-1 边函数是线性函数：既能逐像素增量计算，也能对整块像素做快速接受/拒绝。

实测程序的光栅化内循环就是这三个边函数（为了可读性没有写成增量形式）：

soft_raster.cpp · rasterTri 内循环（每像素、每采样点）

```
auto E = [&](int a, int b, i64 px, i64 py) { return (X[b] - X[a]) * (py - Y[a]) - (Y[b] - Y[a]
for (int py = y0; py ≤ y1; py++) for (int px = x0; px ≤ x1; px++) {
    for (int si = 0; si < T.S; si++) {                    // S = 采样数（1 或 MSAA 的 4）
        i64 SX = (i64)px * ONE + T.offX[si], SY = (i64)py * ONE + T.offY[si];   // 采样点（定点
        i64 w0 = E(1, 2, SX, SY), w1 = E(2, 0, SX, SY), w2 = E(0, 1, SX, SY);
        cov[si] = (w0 + b0 ≥ 0) && (w1 + b1 ≥ 0) && (w2 + b2 ≥ 0);       // b0..b2: Top-Lef
        if (cov[si]) zs[si] = ((float)w0 * t.z[0] + (float)w1 * t.z[1] + (float)w2 * t.z[2]) *
    }
    ...
}
```

5.3 定点亚像素：为什么不直接用浮点数

GPU 在三角形设置阶段会把顶点的屏幕坐标**吸附（snap）到一个定点网格**上：D3D11 要求 8 位亚像素精度，即每个像素被细分为 256×256 个格子，坐标用整数表示。这样做有三个理由：

1. **精确**：边函数变成整数乘法和加法，结果没有舍入误差。共享同一条边的两个三角形，它们对这条边计算的 E 值*恰好互为相反数*，不会出现一个说"在内"、另一个也说"在内"（或都说"在外"）的情况——这叫**水密（Watertight）**。
2. **便宜**：整数加法器比浮点加法器小得多、快得多。
3. **确定**：结果不依赖运算顺序，同一个三角形每次光栅化结果完全一致——这是深度 EQUAL 测试（Z-Prepass）能工作的前提。

🔬 实测：浮点边函数真的会出错吗

E4 的压力测试：在 1024×768 的屏幕上，用 720 个细长三角形围成一个扇形（共享中心点和 720 条约 600 像素长的边），统计扇形内 75 万个像素有多少被漏画或重复画：

```
soft_raster.exe · E4 压力测试

配置                                              漏画  重复画  检查像素
定点(8位亚像素) Top-Left，屏幕坐标整体平移 +0          0       0     757652
定点(8位亚像素) Top-Left，屏幕坐标整体平移 +3000       0       0     757652
定点(8位亚像素) Top-Left，屏幕坐标整体平移 +12000      0       0     757652
float 边函数 Top-Left，屏幕坐标整体平移 +0          0       0     757652
float 边函数 Top-Left，屏幕坐标整体平移 +3000       2       1     757652
float 边函数 Top-Left，屏幕坐标整体平移 +12000      1       1     757652
```

坐标较小时 float 也能碰巧算对；但把所有坐标平移到几千像素外（相当于一个很大的保护带坐标），float 边函数在共享边两侧的舍入不再对称，出现了漏画和重复画。数量不多，但**"偶尔一个像素"的错误恰恰最难查**：漏画表现为模型上闪烁的小黑点（"闪点"），重复画在半透明物体上表现为一条亮线。定点光栅化在任何位置都是 0。

🧠 深入：PS1 为什么"顶点抖动"

初代 PlayStation 的 GTE 把顶点坐标直接取整到整像素，**没有亚像素精度**，物体缓慢移动时顶点会一格一格地跳，于是整个模型看起来在"抖"。再加上它不做透视校正（第 6 章 E5），就成了今天被刻意模仿的"PS1 风格"。8 位亚像素精度意味着顶点每次跳 1/256 像素，肉眼完全看不出来。

5.4 Top-Left 规则：边上的像素归谁

定点数解决了"算得准"，但还有一个逻辑问题：**采样点恰好落在共享边上（E = 0）时，这个像素归哪个三角形?**

- 如果"E ≥ 0 就算覆盖"，两个三角形都会画它——**重复画**（半透明叠两次、模板计数加两次）。
- 如果"E > 0 才算覆盖"，两个三角形都不画它——**漏画**（一条缝）。

D3D 和 OpenGL 采用 **Top-Left 规则**：E = 0 的采样点只有在*上边*或*左边*上才算覆盖。

- **上边**：完全水平、且三角形其余部分在它下方的边；
- **左边**：三角形左侧的边（沿顺时针绕行时向上走的边）。

一条共享边对一个三角形是"左边"，对另一个三角形必然是"右边"，所以 E=0 的像素恰好只归其中一个。实现上非常简单：对不是上边/左边的边，在比较前给 E 减去 1（定点最小单位），把 `E ≥ 0` 变成 `E > 0`：

soft_raster.cpp · Top-Left 偏置（屏幕 y 向下、面积已调整为正）

```
auto isTL = [&](int a, int b) { i64 dx = X[b] - X[a], dy = Y[b] - Y[a];
                                return (dy == 0 && dx > 0) || dy < 0; };   // 上边：水平且向右
i64 b0 = (s.topLeft && !isTL(1, 2)) ? -1 : 0;
i64 b1 = (s.topLeft && !isTL(2, 0)) ? -1 : 0;
i64 b2 = (s.topLeft && !isTL(0, 1)) ? -1 : 0;
```

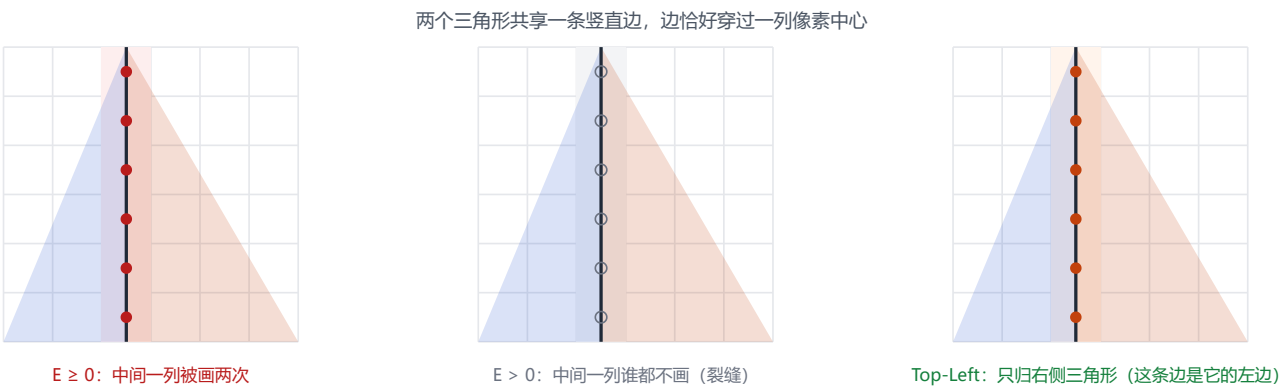


图 5-2　三种规则在共享边上的结果。只有 Top-Left 能做到"恰好一次"。

E4 的网格测试把这件事量化了：15×15 个四边形（450 个三角形）组成的网格，所有顶点都放在像素中心上，让共享边尽可能多地穿过采样点：

```
soft_raster.exe · E4 网格测试（统计内部 3481 个像素）

配置                                                    漏画  重复画
网格A(顶点在像素中心) 定点   规则 w≥0（全收）            0     2131
网格A(顶点在像素中心) 定点   规则 w>0 （全不收）        2131       0
网格A(顶点在像素中心) 定点   Top-Left                  0        0
网格B(随机亚像素抖动) 定点   Top-Left                  0        0
网格B(随机亚像素抖动) 定点   Top-Left 坐标+32768        0        0
```

3481 个像素里有 2131 个恰好落在边上（因为顶点和边都对齐了像素中心），"全收"会多画 2131 次，"全不收"会漏掉 2131 个，Top-Left 一个不差。

5.5 遍历策略：哪些像素需要测试

最朴素的做法是遍历三角形的**包围盒**（实测程序就是这样，钳制到屏幕范围内）。但细长的斜三角形包围盒里绝大部分像素都在外面，浪费严重。真实硬件用的是**分块（Tiled）+ 层次化**遍历：

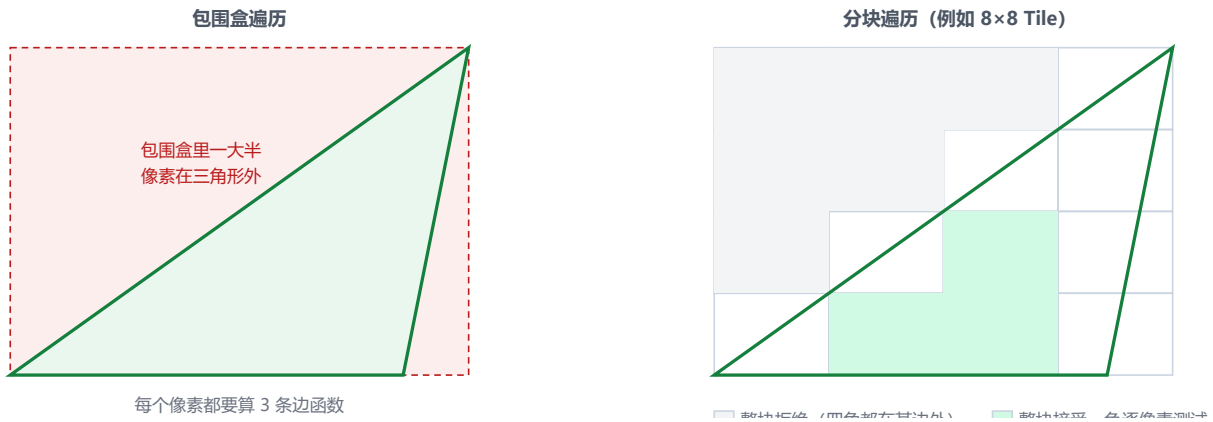


图 5-3　分块遍历：利用边函数的线性性，只用每块四个角的值就能整块拒绝或整块接受，剩下的"部分覆盖"块再逐像素测试。

分块遍历还天然配合两件事：**Hi-Z 粗粒度深度剔除**（一个 Tile 的最远深度如果都比三角形最近深度近，整块跳过，第 8 章）和**Quad 打包**（覆盖结果按 2×2 交给像素着色器，第 6 章）。

5.6 交互实验：亲手光栅化一个三角形

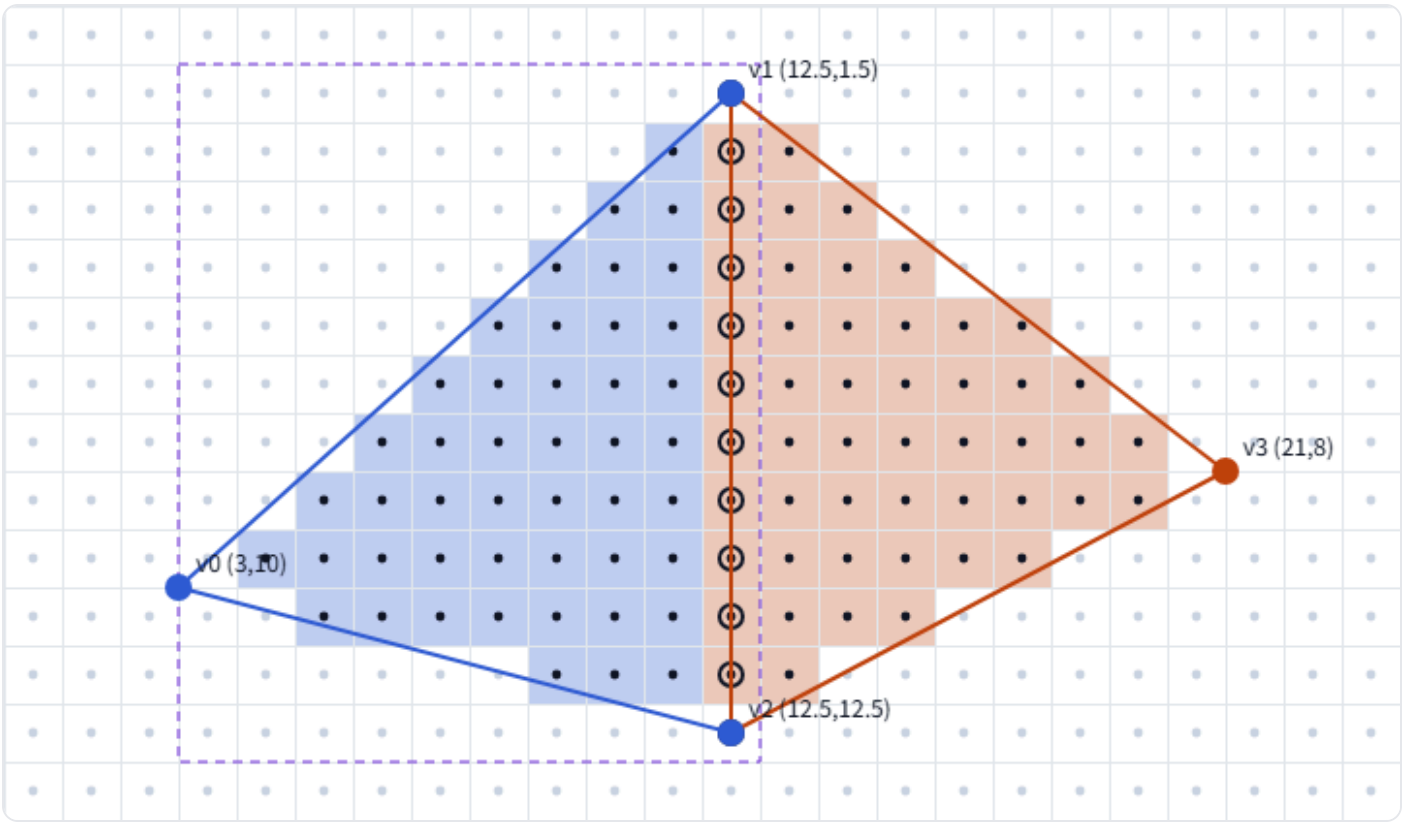
实验 1 • 光栅化网格

拖动三个顶点（会吸附到 1/4 像素网格，便于制造"边恰好穿过像素中心"的情况）。像素中心的圆点表示采样点；填色表示被覆盖。打开"显示 2×2 Quad"可以看到像素着色器实际要运行的通道数。

覆盖规则 Top-Left（D3D/GL） 显示 2×2 Quad 显示包围盒

同时画相邻三角形（共享边 v1–v2）

预设：共享竖边穿过像素中心 预设：细长三角形 预设：微小三角形 预设：大三角形



规则：Top-Left　三角形A（蓝）面积 52.25 px²，覆盖 46 像素
2x2 Quad：14 个 → PS 通道 56，Quad 效率 82.1%（打开"显示 2×2 Quad"可见 Helper 像素）
三角形B（橙）覆盖 50 像素；两者重复覆盖 0 像素
共享边 v1–v2 恰好穿过 12 个像素中心（黑圈）：被画 0 次 2 个，1 次 10 个，2 次 0 个　← 有裂缝

5.7 游戏实战：光栅化相关的几个工程问题

模型"闪点"

相邻三角形之间偶尔漏一个像素，通常不是光栅化器的问题（硬件是水密的），而是**两个三角形的共享顶点坐标不完全相同**：比如 LOD 接缝、地形块之

细长三角形

边函数测试浪费多、2×2 Quad 效率低（E8 实测只有约 48%），还容易产生插值走样。建模时避免长条三角形，程序生成网格时注意剖分方式。

间顶点高度不一致、顶点着色器里对同一顶点算出了不同结果（不同的计算顺序、`precise` 缺失）。

保守光栅化

只要三角形和像素有任何重叠就算覆盖（而不只看中心）。用于体素化（GI）、光源分块剔除、遮挡剔除的软件光栅等"宁可多、不可漏"的场景。

CPU 软件光栅

很多引擎在 CPU 上跑一个低分辨率、只写深度的软件光栅器做**遮挡剔除**（如 Intel 的 Masked Occlusion Culling），原理正是本章的边函数 + 分块。

本章小结

- 覆盖 = **采样点**（默认像素中心）落在三角形内；部分覆盖但没盖住中心的像素不算，这是几何走样的根源。
- 边函数 E 是二维叉积：同号即在内；它是线性函数，可增量计算、可整块判断；它还直接给出重心坐标。
- 顶点吸附到**定点亚像素网格**（8 位），使边函数精确、水密、确定；E4 实测浮点边函数在大坐标下会漏画/重复。
- **Top-Left 规则**决定边上的像素归谁，保证共享边的像素恰好被画一次（E4：2131 个边上像素零误差）。
- 硬件用分块 + 层次化遍历，配合 Hi-Z 和 Quad 打包。

▶ **自测：为什么 D3D 要求 8 位亚像素精度，而不是 16 位?**

▶ **自测：一个三角形面积不为 0，却一个像素都没覆盖，可能吗?**

CHAPTER 06

属性插值：重心坐标、透视校正与 2×2 Quad

顶点只有三个，像素有成百上千个。每个像素的 uv、法线、颜色都要从三个顶点"插"出来——而屏幕空间里的直线插值是错的。

本章目标

- 理解重心坐标及其与边函数的关系
- 证明"1/w 在屏幕空间线性"，推导透视校正插值公式，并用实测看清不校正的后果
- 知道 `nointerpolation`、`noperspective`、`centroid` 这些插值修饰符的用途
- 理解像素着色器为什么按 2×2 Quad 执行、`ddx/ddy` 怎么来、Helper Lane 是什么
- 用实测数据说明小三角形为什么"贵"

6.1 重心坐标

三角形内任一点 p 都可以唯一写成三个顶点的加权和：

$$p = \lambda_0 \cdot v_0 + \lambda_1 \cdot v_1 + \lambda_2 \cdot v_2, \quad \lambda_0 + \lambda_1 + \lambda_2 = 1, \quad \text{内部时 } \lambda_i \geq 0$$

λ_i 就是 p 与"对边"围成的小三角形面积占总面积的比例。而第 5 章的边函数恰好是 2 倍有符号面积，所以光栅化时**重心坐标几乎是免费的**：

$$\lambda_0 = E_{12}(p) \,/\, 2A, \quad \lambda_1 = E_{20}(p) \,/\, 2A, \quad \lambda_2 = E_{01}(p) \,/\, 2A$$

有了重心坐标，任何顶点属性都可以插值：`attr(p) = λ₀·a₀ + λ₁·a₁ + λ₂·a₂`。问题在于：**这里的 λ 是在屏幕空间算的**，而属性是在三维空间里沿三角形表面线性分布的。

6.2 为什么屏幕空间线性插值是错的

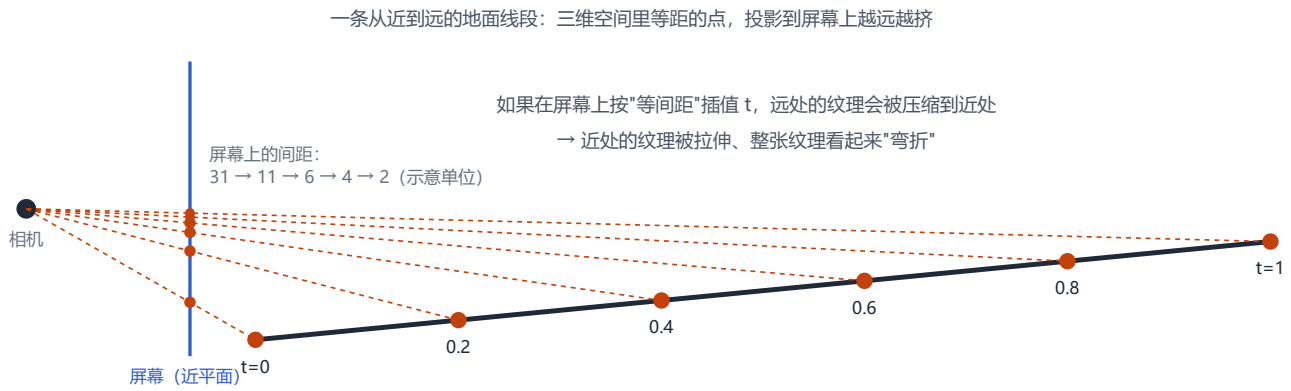


图 6-1　透视投影不保持距离比例：三维中的线性分布在屏幕上不再线性。

推导：什么在屏幕空间里是线性的

考虑观察空间中一条线段，端点的齐次裁剪坐标为 c_0、c_1（w 分别为 w_0、w_1）。线段上的点 `c(t) = (1−t)c₀ + t·c₁`，它的 w 也是 `w(t) = (1−t)w₀ + t·w₁`。屏幕坐标是 x/w，设屏幕上的插值参数为 s：

$$x_s(t) = [(1-t)x_0 + t\cdot x_1] \,/\, [(1-t)w_0 + t\cdot w_1] \quad = \quad (1-s)\cdot(x_0/w_0) + s\cdot(x_1/w_1)$$

解出 s 与 t 的关系，可以得到一个关键结论（把 s 看成屏幕上的线性参数）：

$$1/w(s) = (1-s)\cdot(1/w_0) + s\cdot(1/w_1) \qquad a/w(s) = (1-s)\cdot(a_0/w_0) + s\cdot(a_1/w_1)$$

即：**1/w 和 a/w 在屏幕空间是线性的，a 本身不是。**所以透视校正插值的做法是：插值 a/w 和 1/w，再相除：

$$a(p) = \left(\Sigma\, \lambda_i \cdot a_i / w_i\right) / \left(\Sigma\, \lambda_i / w_i\right)$$

soft_raster.cpp · Tri::interp（透视校正插值）

```
void interp(float px, float py, float* out) const {
    // 屏幕空间重心坐标：三个边函数 / 总面积
    float e0 = (sx[2] - sx[1]) * (py - sy[1]) - (sy[2] - sy[1]) * (px - sx[1]);
    float e1 = (sx[0] - sx[2]) * (py - sy[2]) - (sy[0] - sy[2]) * (px - sx[2]);
    float e2 = (sx[1] - sx[0]) * (py - sy[0]) - (sy[1] - sy[0]) * (px - sx[0]);
    float s = e0 + e1 + e2; float b0 = e0 / s, b1 = e1 / s, b2 = e2 / s;
    if (persp) {  // 透视校正：权重乘以各顶点的 1/w，再归一化
        float w0 = b0 * iw[0], w1 = b1 * iw[1], w2 = b2 * iw[2]; float r = 1.f / (w0 + w1 + w2
        b0 = w0 * r; b1 = w1 * r; b2 = w2 * r;
    }
    for (int i = 0; i < na; i++) out[i] = b0 * a[0][i] + b1 * a[1][i] + b2 * a[2][i];
}
```

深入：为什么深度不需要透视校正

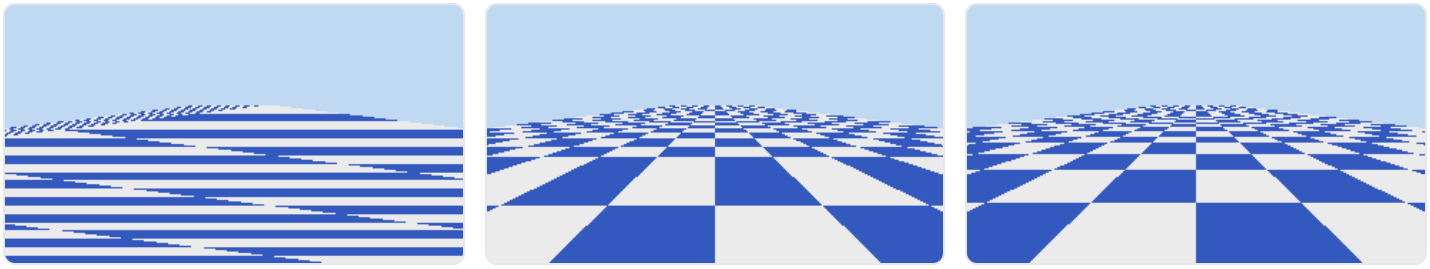
$z_{ndc} = z_{clip}/w$ 本身就是"某个量除以 w"的形式，而 z_{clip} 在三维中是线性的，所以 z_{ndc} 和 a/w 一样在屏幕空间线性。硬件直接用屏幕空间重心坐标插值深度（实测程序里 `zs[si]` 就是这样算的）。这也是"深度缓冲存的是 z/w 而不是 z"的根本原因：**为了能线性插值**，而代价是第 8 章的精度不均匀。

6.3 实测：不做透视校正会怎样

E5 渲染一块从相机脚下延伸到 60 米外的地面（棋盘格纹理），对比三种方式，并对每个像素用**射线与地面求交**得到 uv 的解析真值来计算误差：

soft_raster.exe · E5

```
配置                                  平均uv误差  最大uv误差  近处(<5m)平均误差
仿射插值，地面 2 个三角形          17.4443      20.1966          17.0373
仿射插值，地面细分 16x16=512 三角形      0.3785       0.6835           0.4684
透视校正，地面 2 个三角形            0.0001       0.0066           0.0000
像素(200,120) 仿射 uv=(3.3037, -19.6608)   射线求交真值 uv=(0.3648, -1.3479)
像素(200,120) 透视校正 uv=(0.3648, -1.3479)   射线求交真值 uv=(0.3648, -1.3479)
（uv 单位 = 纹理重复次数；1 个棋盘格 = 0.5。误差 0.5 就是错一整格）
```



仿射插值（2 个三角形）　仿射插值（细分为 512 个三角形）　透视校正（2 个三角形）

图 6-2　E5 实测渲染（320×180）。仿射插值下纹理完全错乱；细分后错误被限制在每个小三角形内部，但棋盘格的边仍是折线；透视校正与真值一致。

🔬 **从输出里能读出什么**

- 只有 2 个大三角形时，仿射插值的平均 uv 误差高达 17 个纹理重复——这块地面的 uv 范围是 24×24，几乎完全错了。
- 细分到 512 个三角形，误差降到 0.38（不到一格）：**细分能缓解但不能消除**，因为误差和三角形在深度方向上的跨度成正比。这正是 PS1 时代的补救办法——地面和墙壁都切得很碎。
- 透视校正的结果与射线求交的解析真值在第 4 位小数上一致，最大误差 0.0066 来自 float 精度和像素中心的离散化。

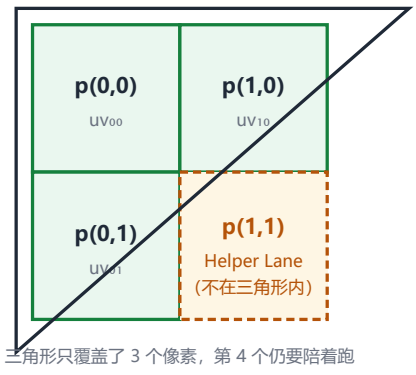
6.4 插值修饰符：不是所有属性都该透视校正

HLSL / GLSL	含义	典型用途
`linear`（默认）/ `smooth`	透视校正插值	uv、法线、世界坐标、颜色
`nointerpolation` / `flat`	不插值，整个三角形使用"激发顶点"（provoking vertex）的值	材质 ID、实例 ID、整数属性（整数不能插值）、Low-Poly 平直着色
`noperspective`	屏幕空间线性插值	屏幕空间效果：线框渲染里"到边的像素距离"、屏幕空间 UV
`centroid`	MSAA 下在"被覆盖采样点的重心"处求值，而不是像素中心	避免像素中心在三角形外时属性被外插：纹理图集边缘串色、uv 超出 [0,1]
`sample`	在每个采样点求值，迫使 PS 按采样频率运行（变成 SSAA）	对边缘有高频着色的特殊材质

6.5 2×2 Quad 与屏幕空间导数

像素着色器里经常需要知道"一个属性在屏幕上变化得有多快"——最典型的就是纹理采样选 Mip 层级（第 7 章）：uv 在相邻像素间变化越大，说明纹理被缩小得越厉害，越应该用更模糊的 Mip。

GPU 的做法简单粗暴：**像素着色器永远以 2×2 的像素块（Quad）为单位执行**，四个像素同时跑同一段代码，于是任何变量的导数都可以用 Quad 内相邻像素的差分得到：



三角形只覆盖了 3 个像素，第 4 个仍要陪着跑

```
ddx(uv) = uv₁₀ − uv₀₀  // 同一行相邻像素之差
ddy(uv) = uv₀₁ − uv₀₀  // 同一列相邻像素之差
```

- Quad 内 4 个像素共享同一组导数（coarse），或按行/列分别求（fine）
- 不在三角形内的像素作为 **Helper Lane** 照样执行，只是结果不写出
- 导数用的是"外插"的属性值，所以三角形边缘处导数仍然有效
- 一个 Warp(32) = 8 个 Quad；小三角形会让大量通道变成 Helper

在"非一致"的分支里调用 Sample()/ddx，导数是未定义的
分支里请用 SampleLevel / SampleGrad，或者把采样移到分支外

图 6-3 2×2 Quad 是屏幕空间导数的来源，也是 Helper Lane 开销的来源。

实测程序没有真的按 Quad 并行执行，而是在需要导数时直接用三角形的插值函数在 (x+1, y) 和 (x, y+1) 处再求一次值（效果等价于 Quad 差分的解析版本）：

soft_raster.cpp · E9 中计算 uv 的屏幕空间导数

```
float a1[MAXA], a2[MAXA];
f.t->interp(f.px + 1, f.py, a1);            // 右边一个像素处的属性（≈ Quad 里的邻居）
f.t->interp(f.px, f.py + 1, a2);            // 下边一个像素处的属性
float dudx = a1[6] - f.a[6], dvdx = a1[7] - f.a[7];   // ddx(uv)
float dudy = a2[6] - f.a[6], dvdy = a2[7] - f.a[7];   // ddy(uv)
```

6.6 实测：小三角形为什么贵

E8 在 512×512 的屏幕上随机光栅化 3000 个大小相同的三角形，统计"实际覆盖的像素数"和"像素着色器实际要跑的通道数（被触及的 Quad 数 × 4）"：

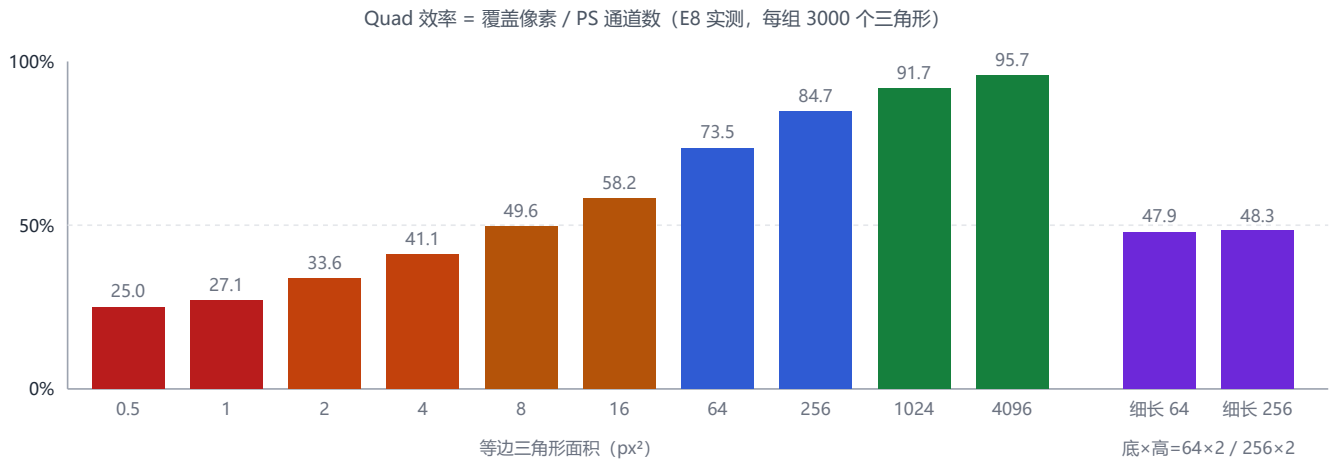


图 6-4 三角形越小、越细长，Quad 效率越低。面积 1 像素的三角形，像素着色器 3/4 的工作是白做的。

soft_raster.exe · E8（节选）

三角形（面积 px²）	覆盖像素	PS lanes	Quad效率	零覆盖三角形	每三角形lanes
等边 0.5	1489	5956	25.0%	1511	2.0
等边 1	2966	10936	27.1%	477	3.6
等边 16	48102	82632	58.2%	0	27.5
等边 1024	3072122	3349324	91.7%	0	1116.4
细长 256（底256x高2）	746799	1544600	48.3%	0	514.9

🔬 从输出里能读出什么

- 面积 0.5 像素的三角形，一半（1511/3000）一个像素都没覆盖——它们的顶点着色、三角形设置全部白做；覆盖到像素的那些，每覆盖 1 个像素要跑 4 个通道（效率 25%）。
- 效率随面积上升，但要到上千像素才超过 90%：**边缘上的 Quad 总是半空的**，三角形越小，边缘占比越大。
- 细长三角形面积 256 像素，效率却只有 48%，和面积 8 像素的等边三角形差不多——因为它**几乎全是边缘**。

🎮 游戏实战：Micro-Triangle 问题

远处的高模角色、过度细分的地形、密集的毛发卡片都会产生大量 1～2 像素的三角形。对策：① **LOD**：保证三角形在屏幕上至少有约 10 像素量级的面积；② 不要对远处开曲面细分；③ UE5 Nanite 的做法是对小三角形改用**计算着色器软光栅**，绕过硬件的 Quad 打包——这在本章的数据下就很好理解了。

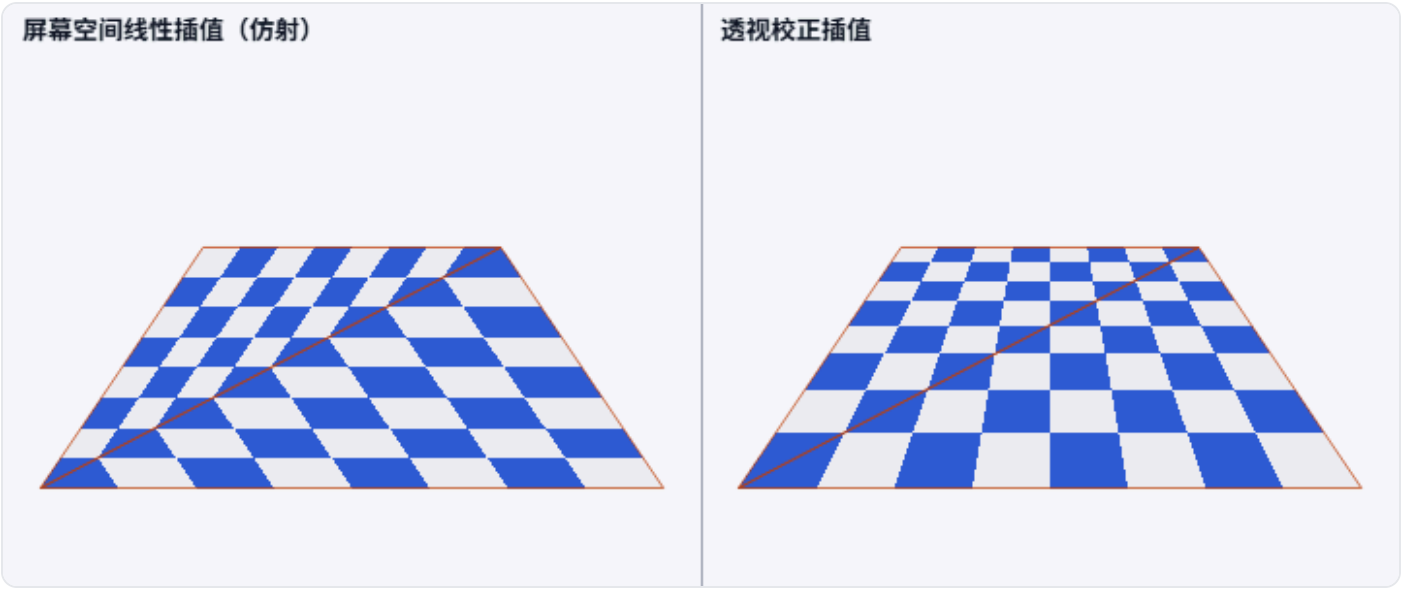
6.7 交互实验：透视校正插值

💉 实验 2 · 仿射插值 vs 透视校正

一块贴了棋盘格纹理的正方形平面，绕水平轴倾斜。左边在屏幕空间直接线性插值 uv，右边做透视校正。增大倾角或减少细分，观察左图的"折痕"。

倾斜角 62°

每边细分 1×1（2 个三角形） 显示三角形边



仿射插值相对透视校正的 uv 误差：平均 1.073 格，最大 2.061 格（1 格 = 棋盘的一个方块）
倾角越大，同一三角形三个顶点的 w 相差越大，误差越大；细分能把误差压到每个小三角形内部，但无法消除。

本章小结

- 重心坐标 = 边函数 / 2 倍面积，光栅化时顺手得到。
- 透视投影下 **1/w 与 a/w 在屏幕空间线性**，a 本身不是；透视校正：$a = \Sigma\lambda_i a_i/w_i \div \Sigma\lambda_i/w_i$。
- 深度 z/w 本身是"除以 w"的形式，可直接线性插值。
- E5：不校正时平均 uv 误差 17 个重复；细分 512 份降到 0.38 但仍有折线；校正后与解析真值一致。
- PS 按 **2×2 Quad** 执行，导数由 Quad 内差分得到；三角形外的像素作为 Helper Lane 陪跑。
- E8：面积 1 像素的三角形 Quad 效率 27%，细长三角形不到 50%——小三角形和细长三角形都很贵。

▶ **自测：为什么在 if 分支里用 tex.Sample() 可能出现"块状"错误?**

▶ **自测：顶点光照（Gouraud）和逐像素光照（Phong）的区别和插值有什么关系?**

PART FOUR

第四部分　像素阶段：颜色、可见性与合成

片元到了这里要回答三个问题：它是什么颜色（像素着色与纹理）、它能不能被看见（深度与模板）、它怎样和已有的颜色合在一起（混合与抗锯齿）。

CHAPTER 07

片元着色与纹理采样

纹理是被采样的信号：放大时要插值，缩小时要先滤波——Mipmap 就是一张预先滤好波的金字塔。

本章目标

- 知道像素着色器的输入输出、`discard` 与写深度的代价
- 理解纹理放大（magnification）与缩小（minification）是两个不同的问题
- 掌握点采样、双线性、三线性、各向异性过滤的原理与开销
- 能写出 Mip LOD 的计算公式，解释"远处模糊""远处闪烁""斜看糊成一片"分别是什么原因
- 理解纹理缓存、纹理压缩、sRGB 与 Mip 对带宽的意义

7.1 像素着色器做什么

像素着色器（Fragment Shader）对每个片元运行一次，输入是插值好的属性（uv、法线、世界坐标……）和系统值（`SV_Position` 的像素坐标、`SV_IsFrontFace`、MSAA 的 `SV_SampleIndex` 等），输出是一个或多个颜色（MRT），可选输出深度和覆盖掩码。

在 PS 里做的事	对管线的影响
纹理采样、光照计算	正常；开销取决于 ALU 指令数和纹理带宽
`discard` / `clip()`（Alpha Test）	深度写入必须等 PS 执行完才能决定 → **Early-Z 的写入被推迟**，移动端 TBDR 的隐藏面消除也会失效（第 8、10 章）
写 `SV_Depth`	深度要等 PS 算完才知道 → **Early-Z 完全关闭**；可以用 `SV_DepthGreaterEqual` 等保守深度挽回一部分
写 UAV / 原子操作	有副作用的 PS 不能被跳过，需要 `[earlydepthstencil]` 显式要求早测

7.2 纹理是被采样的信号

纹理是一张离散的 texel 网格，而像素着色器给出的 uv 是连续的。于是有两种情况：

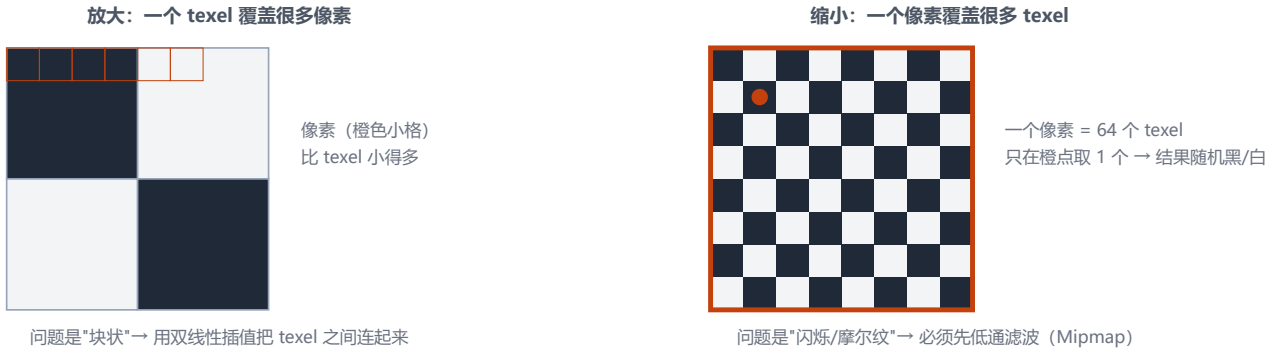


图 7-1　放大和缩小是两回事。缩小时，只取一个点等于用远低于奈奎斯特频率的采样率去采一个高频信号。

缩小时的走样有一个精确的信号处理解释：采样定理要求采样频率至少是信号最高频率的两倍。一个像素覆盖 8×8 个 texel，相当于每 8 个 texel 只采一次，纹理里任何周期小于 16 texel 的细节都会"折叠"成错误的低频图案——这就是远处地面的摩尔纹，镜头移动时它还会闪烁。**解决办法只有一个：采样之前先把高频去掉。**

7.3 Mipmap：预先滤好波的金字塔

Mipmap 预先把纹理逐级缩小一半（box 滤波或更好的 Kaiser 滤波），得到 Level 0（原图）、1（1/2）、2（1/4）……直到 1×1。采样时根据"一个像素覆盖多少个 texel"选择合适的层级，于是每次采样都只需要读少量 texel，却得到了大范围的平均值。

- **内存开销**：1 + 1/4 + 1/16 + … = 4/3，只多 33%。
- **带宽收益**：远处像素读的是小 Mip，texel 在内存里很集中，缓存命中率大幅提高（E9：80.8% → 99.3%）。

LOD 怎么算

用第 6 章的屏幕空间导数：一个像素在纹理空间里的"脚印"可以用两个向量近似——沿屏幕 x 方向走一个像素，uv 变化 `(∂u/∂x, ∂v/∂x)`；沿 y 方向变化 `(∂u/∂y, ∂v/∂y)`。乘以纹理尺寸换成 texel 单位后：

$$\rho = \max\left(\sqrt{((\partial u/\partial x \cdot W)^2 + (\partial v/\partial x \cdot H)^2)},\ \sqrt{((\partial u/\partial y \cdot W)^2 + (\partial v/\partial y \cdot H)^2)}\right) \qquad \text{LOD } \lambda = \log_2 \rho$$

soft_raster.cpp · Tex::sampleGrad（各向同性部分）

```
float lx = std::sqrt(dudx * W * dudx * W + dvdx * H * dvdx * H);   // 像素 x 方向在纹理中的长度
float ly = std::sqrt(dudy * W * dudy * W + dvdy * H * dvdy * H);   // 像素 y 方向
float lod = log2f(std::max(std::max(lx, ly), 1e-8f));              // 取长边 → 宁可模糊，不可闪烁
return trilinear(u, v, lod);                                        // 在 floor(lod) 和 floor(
```

7.4 四种过滤方式

过滤	做法	每次采样读 texel	问题
点采样 Point	取最近的一个 texel，LOD 0	1	放大成马赛克、缩小严重闪烁
双线性 Bilinear	取最近的 2×2 texel 按距离加权	4	放大平滑；缩小仍然走样（只是"软一点的走样"）
三线性 Trilinear	在相邻两层 Mip 上各做一次双线性，再按 LOD 小数部分插值	8	斜视角下过度模糊（用长边选 LOD，短边方向被过度平均）
各向异性 Anisotropic (N×)	沿像素脚印的长轴方向取多个三线性样本，LOD 按短边选	最多 8N	贵；但对地面、道路这种掠射角表面效果显著

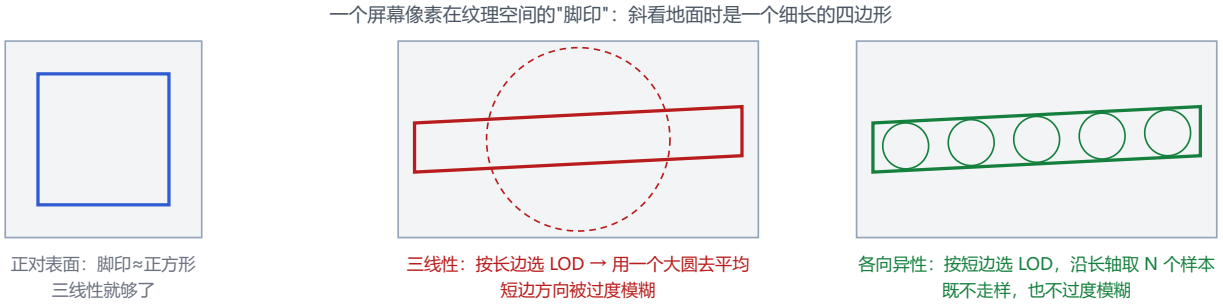
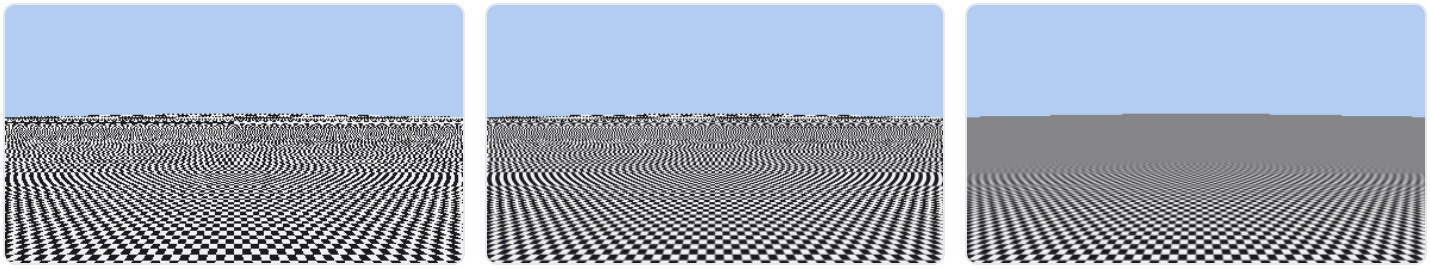


图 7-2　各向异性过滤的思想：脚印是长条形时，用多个小样本去铺满它，而不是用一个大圆。

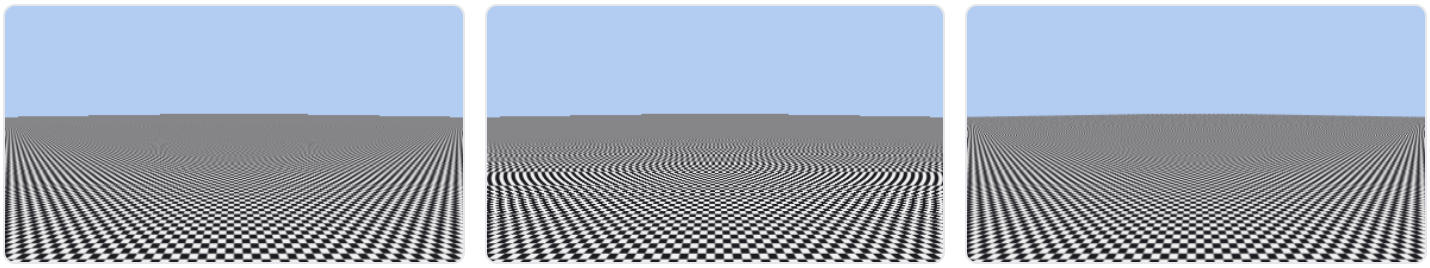
7.5 实测：远处的地面

E9 渲染一块高 1 米视角、一直延伸到 300 米的棋盘格地面。参考图用 8×8=64 次超采样（每个子样本做 LOD0 双线性）近似"像素内的真实平均颜色"，然后比较各种过滤方式的误差和开销：



点采样（无 Mip）　　双线性（无 Mip）　　三线性 Mipmap

图 7-3a　E9 实测（384×216）。点采样在中远处出现强烈摩尔纹；双线性只是让摩尔纹"软"一点；三线性去掉了摩尔纹，但远处糊成了一片灰。



三线性 + 16× 各向异性　　三线性 + LOD Bias −2　　参考（64× 超采样）

图 7-3b　各向异性过滤在远处保留了棋盘格的细节且没有走样，最接近参考图；负的 LOD Bias 让画面"看起来更锐"，实际是重新引入了走样。

soft_raster.exe · E9

采样方式	RMSE全图	近<3m	中3~12m	每像素texel	缓存命中	平均LOD
点采样 LOD0（无 mip）	0.3063	0.2152	0.3694	1.0	80.8%	0.00
双线性 LOD0（无 mip）	0.2210	0.0873	0.2913	4.0	93.7%	0.00
三线性 mipmap	0.0874	0.0550	0.1268	7.6	99.3%	2.50
三线性 + 16x 各向异性	0.0261	0.0173	0.0293	39.3	98.5%	0.70
三线性 + LOD Bias -2（过锐）	0.1221	0.0872	0.1719	5.8	97.0%	2.50

🔬 **从输出里能读出什么**

- **Mipmap 是降误差的第一功臣**：从双线性到三线性，全图误差降为 40%，中距离降为 44%。
- **Mipmap 同时也是省带宽的功臣**：每像素虽然读了 7.6 个 texel（比双线性多），但缓存命中率从 93.7% 升到 99.3%——真正打到显存的未命中次数反而更少。*"开 Mipmap 会更慢"是错误的直觉。*
- **各向异性把误差再降到 1/3**（0.087 → 0.026），平均 LOD 从 2.5 降到 0.7（用短边选 LOD，所以更清晰），代价是每像素读 39 个 texel、是三线性的 5 倍。真实硬件对此做了大量优化（按实际各向异性程度自适应取样数），桌面 GPU 上 16× 各向异性几乎免费，移动端则要慎重（常用 2×～4×，只对地面开）。
- **LOD Bias −2 "看起来更清楚"，误差却比三线性高 40%**——它用更高分辨率的 Mip 去采样，等于又把高频放了回来。TAA/DLSS 这类时域方法会刻意使用负 LOD Bias，因为它们靠多帧抖动累积来去走样。

7.6 纹理缓存、压缩与格式

纹理采样的瓶颈通常不是计算，而是**带宽和延迟**。几个关键事实：

纹理缓存按块组织

texel 在显存中按 Tile / Morton（Z 序）排列，2D 上相邻的 texel 在内存里也相邻；缓存行一次取一小块（实测程序模拟为 4×4）。所以"屏幕上相邻像素采样纹理上相邻位置"时命中率极高——这依赖 Mipmap 和规则的 uv。

块压缩（BCn / ASTC / ETC2）

以 4×4（ASTC 可变）为块固定码率压缩，GPU 解压是硬件完成的、可随机访问。RGBA8 是 32 bpp，BC1/ETC2 RGB 是 4 bpp，ASTC 6×6 约 3.56 bpp——**带宽直接降为 1/8～1/9**。移动端统一用 ASTC。

sRGB 纹理

颜色贴图以 sRGB 存储（感知均匀，8 位够用），采样时硬件自动转成线性空间再插值和计算；法线、粗糙度、遮罩等"数据纹理"必须标记为线性，否则数值被错误地做了伽马变换。

依赖纹理读取

用一次采样的结果去算另一次采样的 uv（视差、扭曲特效）会让第二次采样无法提前发出，延迟隐藏能力下降；在 Warp 数少（寄存器多）时尤其明显。

⚠️ **面试易错**

- "Mipmap 是为了让远处的纹理更清楚"——错，Mipmap 让远处**更模糊**，它是为了**消除走样和闪烁、提高缓存命中率。**
- "UI 也应该开 Mipmap"——不一定。UI 通常 1:1 显示，不会缩小，开 Mipmap 只会多占 33% 内存；但会被缩放显示的图标、世界空间 UI 应该开。
- "纹理 Wrap 模式不影响 Mip"——图集（Atlas）里相邻的子图在低 Mip 会互相渗色，需要留出足够的 padding 或使用 Texture Array。

7.7 游戏实战：贴图预算与 Mip Streaming

- **Mip Streaming / 纹理流送**：根据物体在屏幕上的大小只加载需要的 Mip 层级（例如远处的建筑只加载 256×256 及以下），内存可省一半以上。Unity 的 Mipmap Streaming、UE 的 Texture Streaming 都基于第 7.3 节的 LOD 估计。
- **MOBA 的地表**：俯视 45°～60° 左右，各向异性需求没有第一/三人称游戏强，但地面贴图分辨率大、重复多，Mipmap + ASTC 必不可少；地表常用多层混合（Splat Map），每多一层就多一组采样。
- **特效贴图**：粒子常用小尺寸的序列帧图集，注意图集 padding 和 Mip 渗色；全屏扭曲/热浪是依赖纹理读取，移动端要控制使用范围。

本章小结

- `discard` 推迟深度写入、写 `SV_Depth` 关闭 Early-Z——PS 的写法会反过来影响固定功能阶段。
- 纹理**放大**要插值（双线性），**缩小**要先滤波（Mipmap），否则高频折叠成摩尔纹和闪烁。
- LOD = $\log_2$(像素脚印在纹理中的长边长度)，长度来自 2×2 Quad 求出的 uv 导数。
- E9：三线性比无 Mip 误差降 60%、缓存命中 99%；16× 各向异性误差再降 70%，但取样数是 5 倍；负 LOD Bias 重新引入走样。
- 块压缩（ASTC/BCn）把纹理带宽降到 1/8 左右；颜色贴图用 sRGB，数据贴图用线性。

▶ **自测：为什么非 2 的幂（NPOT）纹理在老设备上有限制?**

▶ **自测：同样的 Shader，换成更小的贴图为什么能明显变快?**

CHAPTER 08

深度与可见性：Z-Buffer、Early-Z 与 Reversed-Z

深度缓冲回答"谁在前面"；Early-Z 决定"被挡住的像素要不要白算"；深度格式决定"远处还分不分得清前后"。

本章目标

- 理解 Z-Buffer 算法，以及它为什么让不透明物体可以"任意顺序绘制"
- 说清 Early-Z 的生效条件和失效条件，理解 Hi-Z 的作用
- 用实测数据比较随机顺序、从前往后、从后往前、Z-Prepass 四种画法
- 理解深度精度为什么集中在近处、Z-Fighting 的成因，以及 Reversed-Z 为什么能"免费"大幅提升精度

8.1 Z-Buffer：为每个像素记住"目前最近的距离"

深度缓冲为每个像素（MSAA 下为每个采样点）存一个深度值。每个片元到达时，拿它的深度与缓冲中的值比较：通过则写入颜色（和深度），不通过则丢弃。

优点：与绘制顺序无关

不透明物体以任意顺序绘制，最终结果都正确（深度相等的情况除外）。这让引擎可以按材质排序来减少状态切换，而不必严格按距离排序。

局限：只能记住一层

每个像素只存最近的一个深度，无法处理半透明（需要知道多层）。所以半透明物体必须单独排序绘制，且通常关闭深度写入（第 9 章）。

深度比较函数可配置：`LESS`（默认）、`LEQUAL`、`GREATER`（Reversed-Z 用）、`EQUAL`（Z-Prepass 后的着色 Pass 用）、`ALWAYS`（UI、全屏后处理）。深度写入也可以单独开关（ZWrite On/Off）。

8.2 Early-Z：在着色之前就丢掉被挡住的片元

按照逻辑管线，深度测试在像素着色器*之后*（输出合并阶段）。但如果一个片元注定被挡住，它的像素着色就白做了。现代 GPU 会尽量把深度测试**提前到像素着色之前**，这就是 Early-Z：

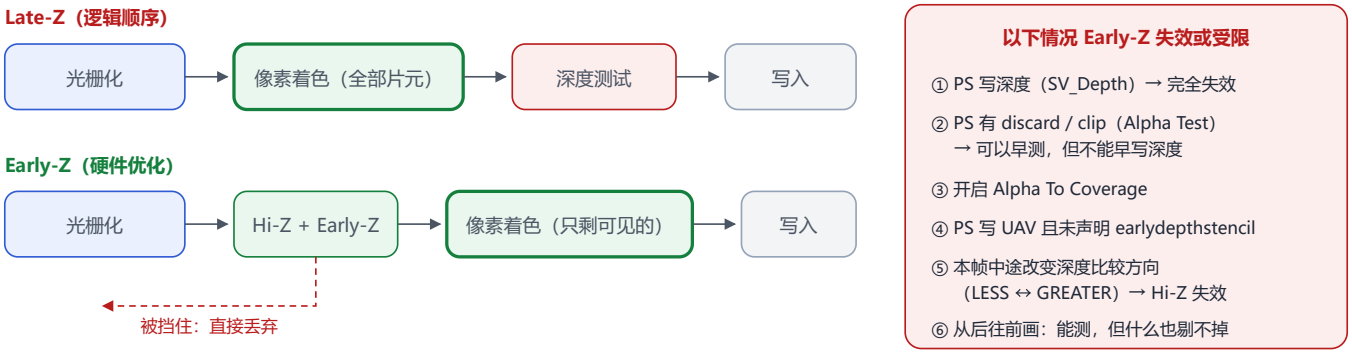


图 8-1　Early-Z 把深度测试挪到 PS 之前；但只有在"PS 不会改变深度/覆盖"时，硬件才敢这么做。

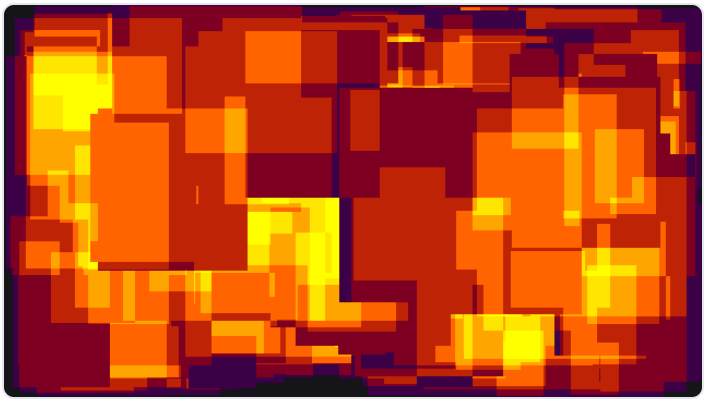
Hi-Z（Hierarchical Z）是 Early-Z 的粗粒度版本：硬件为每个 Tile（如 8×8 像素）维护一个保守的最远深度。三角形光栅化到某个 Tile 时，如果它在这个 Tile 内的最近深度都比 Tile 的最远深度还远，整个 Tile 直接跳过，连逐像素的深度读取都省了。

8.3 实测：同一批不透明方块，四种画法

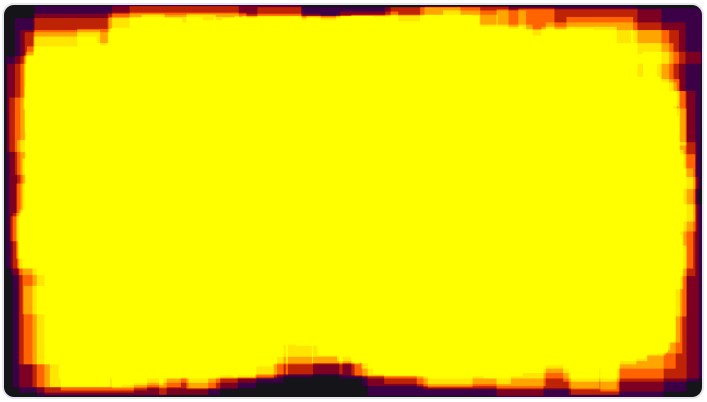
E7 在 640×360 的屏幕上画 400 个随机大小、随机深度的不透明矩形（平均 Overdraw 约 27 层），像素着色器是一段 48 次循环的"重"计算。分别用五种方式绘制：

```
soft_raster.exe · E7

画法                                  光栅片元    PS执行  平均overdraw  深度剔除    耗时ms
Late-Z（先着色后测试）  随机顺序     5992422    5992422       26.69    5254439     3763.7
  （屏幕覆盖像素 224506 / 230400）
Early-Z 随机顺序                     5992422     737983        3.29    5254439      510.8
Early-Z 从前往后排序                 5992422     224506        1.00    5767916      190.8
Early-Z 从后往前排序                 5992422    5992422       26.69          0     3771.3
Z-Prepass + EQUAL 着色              11984844     224506        1.00    5767916      248.2
```



Early-Z + 随机顺序：PS 执行次数热力图



Early-Z + 从后往前：PS 执行次数热力图

图 8-2 E7 每像素着色次数热力图（越亮越多）。随机顺序时 Early-Z 已经剔除了大部分片元；从后往前时每一层都比上一层近，Early-Z 什么也剔不掉。

🔬 从输出里能读出什么

- **Early-Z 对随机顺序就已经很有效**：PS 执行从 599 万次降到 74 万次（−88%），耗时从 3.8 秒降到 0.5 秒。因为随机顺序下，一个被 N 层覆盖的像素，"新片元比当前所有已画片元都近"的期望次数是调和级数 $1+\frac{1}{2}+\frac{1}{3}+\ldots+1/N \approx \ln N + 0.58$（N=27 时约 3.9）；各像素实际层数不一，平均下来就是实测的 3.29。
- **从前往后排序让 PS 执行次数恰好等于可见像素数**（224506，Overdraw 1.00），耗时再降到 191ms，是 Late-Z 的 1/20。
- **从后往前**是 Early-Z 的最坏情况：每一层都"更近"，全部通过——和关掉 Early-Z 一样慢。半透明必须从后往前画，所以半透明 Overdraw 无法靠深度测试省掉（第 9 章）。
- **Z-Prepass**：先用一个极简的 Pass 只写深度（光栅化片元翻倍到 1198 万），再用 `EQUAL` 测试做着色，PS 同样只跑 224506 次，耗时 248ms。它**不依赖排序**，代价是多一遍几何处理。

✅ Z-Prepass 值不值?

值得的情况：像素着色很重（复杂 PBR、多光源）、场景 Overdraw 高、很难精确排序（大物体互相穿插）；同时深度缓冲还能被后续的 SSAO、屏幕空间反射、软粒子等复用。不值得的情况：顶点很多而像素着色便宜（多一遍几何得不偿失）；**移动端 TBDR**：硬件自带隐藏面消除（HSR/FPK，第 10 章），Prepass 往往是纯开销。

8.4 深度精度：为什么远处会"打架"

第 3 章得出：标准 D3D 深度 `z = a + b/d`，即深度值与距离的**倒数**线性相关。E6 用 near=0.1、far=10000 计算各距离处"深度缓冲中相邻两个可表示值对应的世界距离差"（越小越好）：

soft_raster.exe · E6 精度表（near=0.1, far=10000）

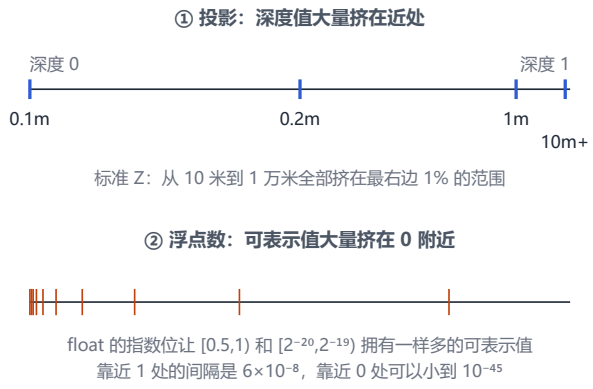
距离 d	标准Z float	标准Z 24bit	GL float Reversed float	Rev 无限远
0.5	1.4901e-07m	1.4901e-07m	7.45051e-08m	3.72525e-08m	3.72529e-08m
10.0	5.96044e-05m	5.96044e-05m	2.98021e-05m	9.31313e-07m	9.31323e-07m
100.0	0.00596048m	0.00596048m	0.00298033m	1.16414e-05m	1.16415e-05m
1000.0	0.596371m	0.596371m	0.298096m	7.27588e-05m	7.27596e-05m
5000.0	14.9254m	14.9254m	7.45157m	0.000227371m	0.000454747m
9000.0	48.353m	48.353m	24.2414m	9.20854e-05m	0.000736691m

标准Z 的非线性：z=0.5 对应 d=0.2000m，z=0.9 对应 d=0.9999m，z=0.99 对应 d=9.990m —— 一半的深度值花在了前 **0.20** 米

1000 米处标准 Z 只能分辨 0.6 米的前后差——两个相距 0.3 米的面在那里就会随机地"你压我、我压你"，这就是 **Z-Fighting**。

Reversed-Z：把 near 映射到 1、far 映射到 0

为什么仅仅反过来就能好这么多？关键在于两个"不均匀"恰好可以互相抵消：



Reversed-Z = ① 和 ② 互相抵消

标准 Z：远处 → 深度≈1 → 浮点最稀疏
两种"不均匀"叠加，远处精度雪上加霜

Reversed-Z：远处 → 深度≈0 → 浮点最密集
投影把远处压扁，浮点又在那里最精细

结果：相对精度在整个范围内近似恒定
E6：1000 米处 0.6m → 0.00007m（约 8000 倍）

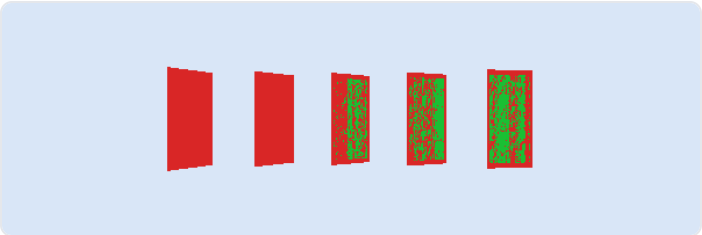
前提：深度缓冲必须是 **32 位浮点**（D32F）
24 位定点没有指数位，反转毫无用处

图 8-3 Reversed-Z 的原理：让深度值最密集的区域（远处，经投影后 ≈ 0）正好落在浮点数最精细的区域。

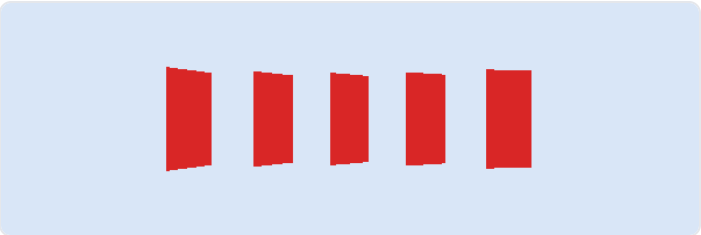
E6 的渲染实测：5 组倾斜的墙面，分别位于 100、300、1000、3000、8000 米处，每组的绿墙在红墙后方 0.25 米且*先画*，理论上应该完全看不到绿色：

soft_raster.exe · E6 渲染实测（各距离"绿色穿出"像素比例）

```
配置                                  100m      300m      1000m     3000m     8000m
标准Z float      near=0.1             0.0%      0.0%      30.5%     44.3%     50.4%
标准Z 24bit定点  near=0.1           0.0%      0.0%      30.5%     44.3%     50.4%
标准Z float      near=1.0             0.0%      0.0%      0.0%      46.8%     43.8%
OpenGL float     near=0.1               0.0%      0.0%      35.1%     43.0%     49.1%
Reversed float   near=0.1               0.0%      0.0%      0.0%      0.0%      0.0%
Reversed 无限远  near=0.1           0.0%      0.0%      0.0%      0.0%      0.0%
```



标准 Z（float，near=0.1）



Reversed-Z（float，near=0.1）

图 8-4　E6 实测渲染（600×200）。从左到右为 100～8000 米的墙。标准 Z 从 1000 米开始红绿交错（Z-Fighting），Reversed-Z 全部正确。

🔬 从输出里能读出什么

- 标准 Z 下 float 和 24 位定点的结果**完全一样**：在 [0.5, 1) 区间里 float 的间隔 2^{-24} 恰好与 24 位定点相同，用 float 存标准 Z 是浪费。
- **把 near 从 0.1 调到 1.0**，1000 米处的错误从 30.5% 降到 0——near 对精度的影响远大于 far（深度误差近似与 $d^2/near$ 成正比）。
- OpenGL 的 [−1,1] 映射比 D3D 还差一点：中间的 0 落在近处，远处挤在 1 附近，且 `(z+1)/2` 的转换又损失了精度。`glClipControl(GL_ZERO_TO_ONE)` 才能在 GL 上用好 Reversed-Z。
- **Reversed-Z 在 8 公里处仍然零错误**；"无限远平面"版本（far→∞）也同样好，所以用 Reversed-Z 时甚至可以不设远平面。

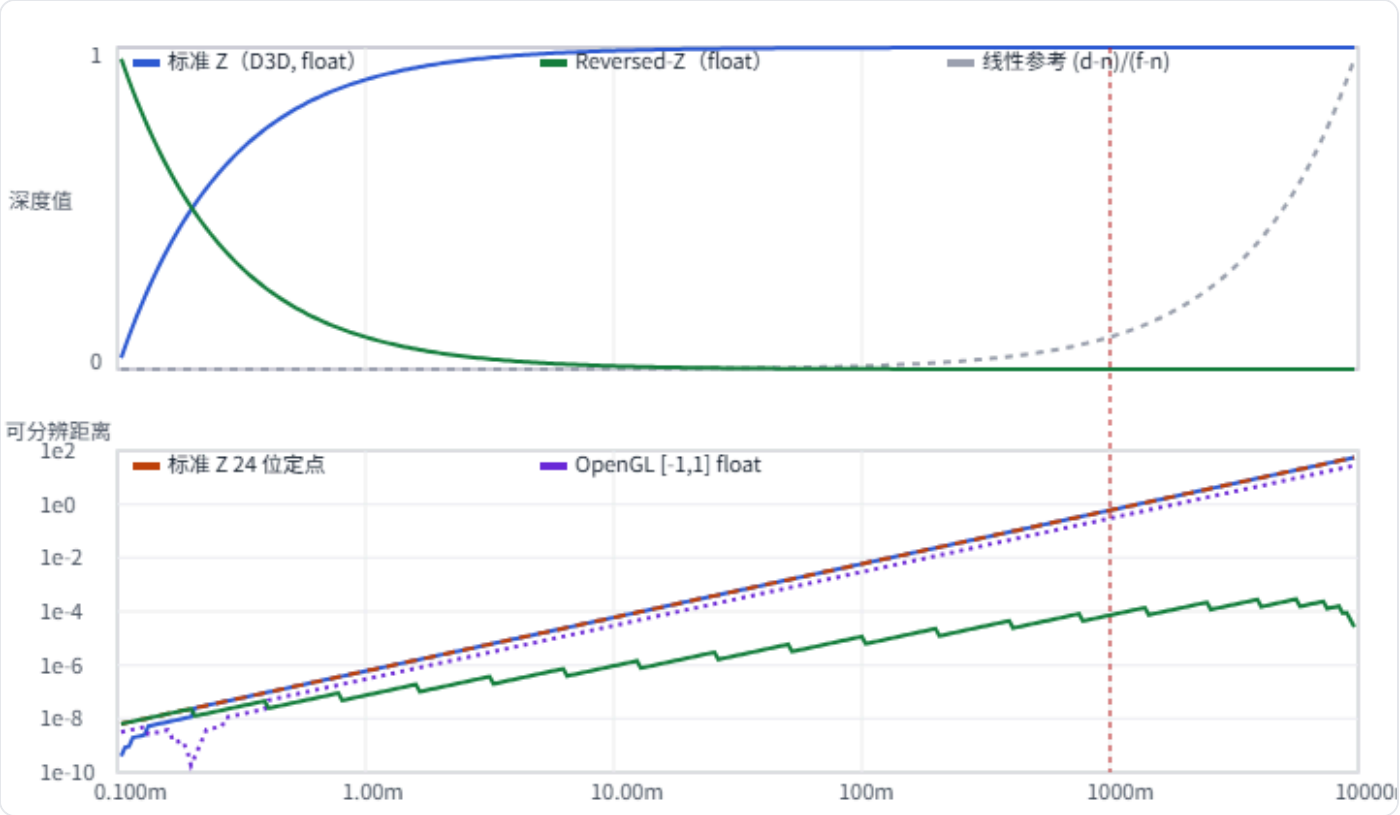
8.5 交互实验：深度精度曲线

🧪 实验 3 • 深度值与精度

上图：不同映射下深度值随距离的变化（横轴对数刻度）；下图：各距离处能分辨的最小前后距离（纵轴对数刻度，越低越好）。调 near、far 和深度格式，观察曲线怎么变。

near 0.100 m far 10000 m

查询距离 1000 m



near=0.100m, far=10000m；标准 Z 下 z=0.5 对应 0.200m（一半深度值用在这之前）

在 1000m 处：标准Z深度值 0.9999100，Reversed 深度值 9.000e-5

可分辨的最小前后距离：标准Z float 5.96e-1m | 24位 5.96e-1m | GL float 2.98e-1m | Reversed float 7.28e-5m（约好 8192 倍）

8.6 游戏实战：Z-Fighting 的排查与修复

手段	适用	说明
增大 near	所有项目	收益最大、成本为零；用"相机离墙最近能有多近"来定 near 的上限
Reversed-Z + D32F	PC/主机、支持 D32F 的移动设备	UE4/UE5、Unity（D3D/Metal/Vulkan 下默认）都已经采用
美术上避免共面	贴花、道路、地毯	拉开距离或直接合并到同一个面
深度偏移（Depth Bias / Polygon Offset）	贴花、阴影贴图	按斜率和常数推远/拉近深度，第 11 章 E13 详细实测
Decal 用模板或延迟贴花	大量弹孔、血迹	不再依赖深度比较决定贴花是否可见
对数深度 / 分层渲染	天文尺度、飞行模拟	在 PS 里写对数深度（但会关闭 Early-Z），或把近景和远景分两个相机渲染

🎮 游戏实战：不透明物体的绘制顺序

引擎通常按"渲染队列 → 材质/Shader → 距离"排序：不透明物体**大致从前往后**（利用 Early-Z）但优先合并相同材质（减少 SetPass）；天空盒放在不透明物体之后、深度设为最远（只填充没被挡住的像素）；Alpha Test 物体（草、树叶）放在纯不透明之后，避免它们推迟深度写入影响其他物体的 Early-Z。MOBA 的俯视角下，地面会被英雄、建筑、草丛挡住一部分，所以"先画英雄和建筑、最后画大地面"能明显减少地面的着色量——E13 的主 Pass 就这么做了，地面有 8215 个片元被 Early-Z 剔除。

本章小结

- Z-Buffer 让不透明物体与绘制顺序无关，但只能记住一层，处理不了半透明。
- Early-Z 把深度测试提前到 PS 之前；写深度、discard、Alpha To Coverage、UAV 写入会让它失效或受限；Hi-Z 以 Tile 为单位粗剔除。
- E7：Early-Z 随机顺序 PS −88%；**从前往后** PS 次数 = 可见像素数（20 倍提速）；从后往前等于没有 Early-Z；Z-Prepass 不依赖排序，代价是两遍几何。
- 标准 Z 精度集中在近处：near=0.1 时一半深度值用于前 0.2 米；1000 米处只能分辨 0.6 米。
- **Reversed-Z + 浮点深度**让投影的不均匀与浮点的不均匀互相抵消：E6 中 8 公里内零 Z-Fighting。24 位定点深度反转无效。

- 修 Z-Fighting 的第一选择是增大 near。

▶ **自测：Alpha Test 的物体为什么会影响 Early-Z？应该怎么画？**

▶ **自测：深度缓冲里存的是线性深度吗？后处理要用线性深度怎么办？**

CHAPTER 09

输出合并：模板、混合与抗锯齿

片元通过了所有测试，最后一步是和帧缓冲里已有的东西"合"在一起——这一步决定了半透明能不能画对、锯齿能不能消掉。

本章目标

- 记住逐片元操作的顺序：裁剪矩形 → (Alpha Test) → 模板测试 → 深度测试 → 混合 → 写掩码
- 会用模板缓冲实现描边、遮挡显示（X-Ray）、遮罩、传送门等效果
- 理解混合方程，以及为什么半透明必须排序、为什么加法混合可以不排序
- 理解预乘 Alpha 解决的问题，以及 OIT 的基本思路
- 比较 SSAA、MSAA、FXAA、TAA：分别消除哪一种走样、代价是什么

9.1 逐片元操作的顺序



模板和深度测试在硬件中通常与 Early-Z 一起提前执行；"模板测试结果"还能决定深度测试失败/通过时对模板值做什么操作

图 9-1　输出合并阶段（D3D 称 Output Merger，GL 称 Per-Fragment Operations）。

9.2 模板缓冲：给像素打标签

模板缓冲为每个像素存一个 8 位整数（通常与 24 位深度打包成 D24S8，或 D32F + S8）。模板测试把参考值与缓冲值按位掩码比较，并根据三种结果——**模板失败 / 模板通过但深度失败 / 都通过**——分别执行 Keep、Replace、Increment、Decrement、Invert、Zero 等操作。

效果	做法
角色描边	Pass 1 正常画角色并把模板写成 1；Pass 2 把角色沿法线外扩一点，只在模板 ≠ 1 的地方画描边色
遮挡显示（X-Ray / 透视轮廓）	角色被墙挡住的部分显示一个半透明剪影：画角色时 `ZTest Greater`（只在被挡住时通过），并用模板防止同一像素被叠多次
UI 遮罩（Mask）	先把遮罩形状写进模板，再只在模板匹配处画子元素；嵌套遮罩用递增的模板值
传送门 / 镜子	门框区域写模板，再在模板区域内用另一个相机矩阵渲染门后的场景
延迟光照的光源体积	用模板标记"被光源体积覆盖且在几何体前面"的像素，只对这些像素做光照（第 11 章）
阴影体（Shadow Volume）	Depth-Fail（Carmack's Reverse）：正面深度失败 +1、背面深度失败 −1，模板非零即在阴影中

> 🎮 **游戏实战：MOBA 里的模板**
>
> 英雄走进草丛或被建筑挡住时显示轮廓/半透明剪影、选中目标的高亮描边、技能指示器只画在地面上（不画到英雄身上）——这些都是模板的典型用法。注意：模板值是全局共享的 8 位资源，引擎要统一分配位（例如 bit0～2 给描边、bit3 给 UI 遮罩、bit4 标记角色），否则不同系统会互相覆盖。

9.3 混合方程

混合把像素着色器输出的颜色（源，src）和帧缓冲中已有的颜色（目标，dst）合成：

$$C_{out} = C_{src} \cdot F_{src} \quad \oplus \quad C_{dst} \cdot F_{dst} \qquad （\oplus \text{ 可以是 Add / Sub / RevSub / Min / Max}）$$

名称	F_{src} / F_{dst}	效果	满足交换律?
Alpha 混合（Over）	SrcAlpha / OneMinusSrcAlpha	普通半透明：玻璃、烟雾、UI	否
预乘 Alpha	One / OneMinusSrcAlpha	同上，但颜色已预先乘过 alpha（9.5 节）	否
加法（Additive）	One / One（或 SrcAlpha / One）	火焰、光效、能量、Bloom 叠加：只会变亮	是
乘法（Multiply）	DstColor / Zero	阴影贴花、暗化、染色	是
Min / Max	—	特殊效果、深度类数据的合成	是

9.4 半透明为什么必须排序

E11 在白色背景上依次叠加 3 层 alpha=0.5 的红、绿、蓝，穷举全部 6 种顺序：

```
soft_raster.exe · E11
```

```
白色背景上依次叠加 3 层 alpha=0.5 的 红/绿/蓝（Over 混合 src*a + dst*(1-a)）：
  顺序 R→G→B : Over 结果 (0.250, 0.375, 0.625)    Additive 结果 (1.30, 1.30, 1.30)
  顺序 R→B→G : Over 结果 (0.250, 0.625, 0.375)    Additive 结果 (1.30, 1.30, 1.30)
  顺序 G→R→B : Over 结果 (0.375, 0.250, 0.625)    Additive 结果 (1.30, 1.30, 1.30)
  顺序 G→B→R : Over 结果 (0.625, 0.250, 0.375)    Additive 结果 (1.30, 1.30, 1.30)
  顺序 B→R→G : Over 结果 (0.375, 0.625, 0.250)    Additive 结果 (1.30, 1.30, 1.30)
  顺序 B→G→R : Over 结果 (0.625, 0.375, 0.250)    Additive 结果 (1.30, 1.30, 1.30)
结论：Over 混合不满足交换律 → 半透明必须排序；Additive 满足交换律 → 粒子特效常用加法混合免排序。
```



图 9-2 E11 的六种结果（按 sRGB 近似显示）。

Over 混合 `dst' = a·src + (1−a)·dst` 的含义是"src 盖在 dst 上面"，所以必须先画远的（dst）、后画近的（src）。半透明物体的标准做法：

1. 先画完所有不透明物体（写深度）；
2. 半透明物体**从后往前**排序，开启深度测试（被不透明物体挡住的不画）、**关闭深度写入**（否则近处的半透明会把后画的、更远的半透明挡掉——但我们已经排过序了，这种情况本不该发生；关闭写入是为了让排序不完美时也不至于"消失"）。

> ⚠️ **面试易错：按物体排序也不完全对**
>
> 引擎按**物体中心到相机的距离**排序，但物体之间可能互相穿插（两片交叉的玻璃），或者一个物体自身有多层（一个半透明的球，前表面和后表面）——这些情况下物体级排序必然有错。常见缓解手段：自身多层的模型先画背面再画正面（Cull Front 再 Cull Back 两个 Pass）；或先用一个 Pass 只写深度、再用 LEQUAL 只画最前面一层（"半透明但只显示最外层"）；彻底解决需要 OIT。

顺序无关透明（OIT）

深度剥离 Depth Peeling

每个 Pass 剥出"下一层"最近的表面，N 层需要 N 个 Pass。结果精确但开销与层数成正比。

逐像素链表 / A-Buffer

每个像素用原子操作把所有片元追加到链表，最后排序合成。精确，显存不可控。

加权混合 WBOIT

用与深度相关的权重做可交换的加权平均，一个 Pass 完成。近似，但对烟雾、粒子足够好，移动端也可用。

Moment / MLAB / 等

用矩或固定数量的片上层数近似透射率，在质量和成本之间折中；移动端可借助 Pixel Local Storage 实现。

9.5 预乘 Alpha：消灭"黑边"

贴图里完全透明的像素颜色通常是任意值（常见的是黑色）。双线性过滤时，不透明像素和透明像素的颜色会被**先插值再按 alpha 混合**，透明像素的黑色就渗进了边缘：

```
soft_raster.exe · E11 双线性过滤 + 混合
```

```
双线性过滤的"黑边"：texel A=不透明白(1,1,1,a=1)，texel B=全透明但颜色为黑(0,0,0,a=0)，在两者正中采样，背景为绿(0,1,0)
  直通 Alpha：过滤后 (0.50,0.50,0.50,a=0.50) → 混合结果 (0.250, 0.750, 0.250)   ⟵ 偏暗，出现黑边
  预乘 Alpha：过滤后 (0.50,0.50,0.50,a=0.50) → 混合结果 (0.500, 1.000, 0.500)   ⟵ 正确：白与绿各半
```

直通（Straight）Alpha 下，过滤后的颜色 0.5 代表的是"一半白一半黑"，再乘以 alpha 0.5 混合，等于把黑色也算了进去。预乘（Premultiplied）Alpha 把颜色事先乘好 alpha（透明像素的颜色自然是 0），过滤时颜色和 alpha 按同样的比例变化，结果在数学上就是正确的。

- 预乘后混合因子变成 `One, OneMinusSrcAlpha`，还能**用同一个混合状态同时表达 Over 和 Additive**（alpha=0 时就是纯加法）——粒子系统可以把两种粒子合进一个批次。
- Mipmap 生成、缩放图片、多层合成也应该在预乘空间里做。

9.6 抗锯齿：三种走样，四种方案

"锯齿"其实是几种不同的走样：

几何走样

三角形边缘：每个像素只看中心一个点是否被覆盖（第 5 章）。

着色走样

像素内部：高频的高光、法线贴图、细条纹在每个像素只算一次时闪烁。

纹理走样

纹理缩小时（第 7 章），由 Mipmap/各向异性解决。

方案	做法	解决	代价
SSAA（超采样）	以 N 倍分辨率渲染再降采样；每个样本都跑 PS	几何 + 着色	PS、带宽、显存全部 ×N
MSAA（多重采样）	每像素 N 个覆盖/深度样本，**PS 只跑一次**，结果写入所有被覆盖的样本；最后 Resolve 平均	几何（边缘）	深度/颜色存储 ×N；PS 几乎不变；对着色走样无效
FXAA / SMAA（后处理）	在最终图像上检测高对比度边缘并沿边方向模糊	几何（部分）	很便宜；会糊掉细节文字和纹理；时间上不稳定
TAA（时域）	每帧对投影矩阵做亚像素抖动，用运动矢量把历史帧重投影后与当前帧混合	几何 + 着色 + 部分纹理	需要运动矢量；会产生拖影/鬼影，需要邻域裁剪；画面偏软

E10 用一个高频条纹着色的场景（含一个细长条），与 256 样本的参考图比较边缘像素和内部像素的误差：

soft_raster.exe · E10

方式	RMSE边缘	RMSE内部	PS执行	深度/颜色样本	边缘(纯色)RMSE
无 AA（1 样本）	0.1835	0.1683	4705	4705	0.2569
MSAA 4x（每像素着色 1 次）	0.1175	0.1683	4946	18810	0.1063
SSAA 4x（每样本着色）	0.0620	0.0067	18810	18810	0.1063

（边缘像素 539 个，内部像素 4371 个；内部是高频条纹着色，用来暴露"着色走样"）

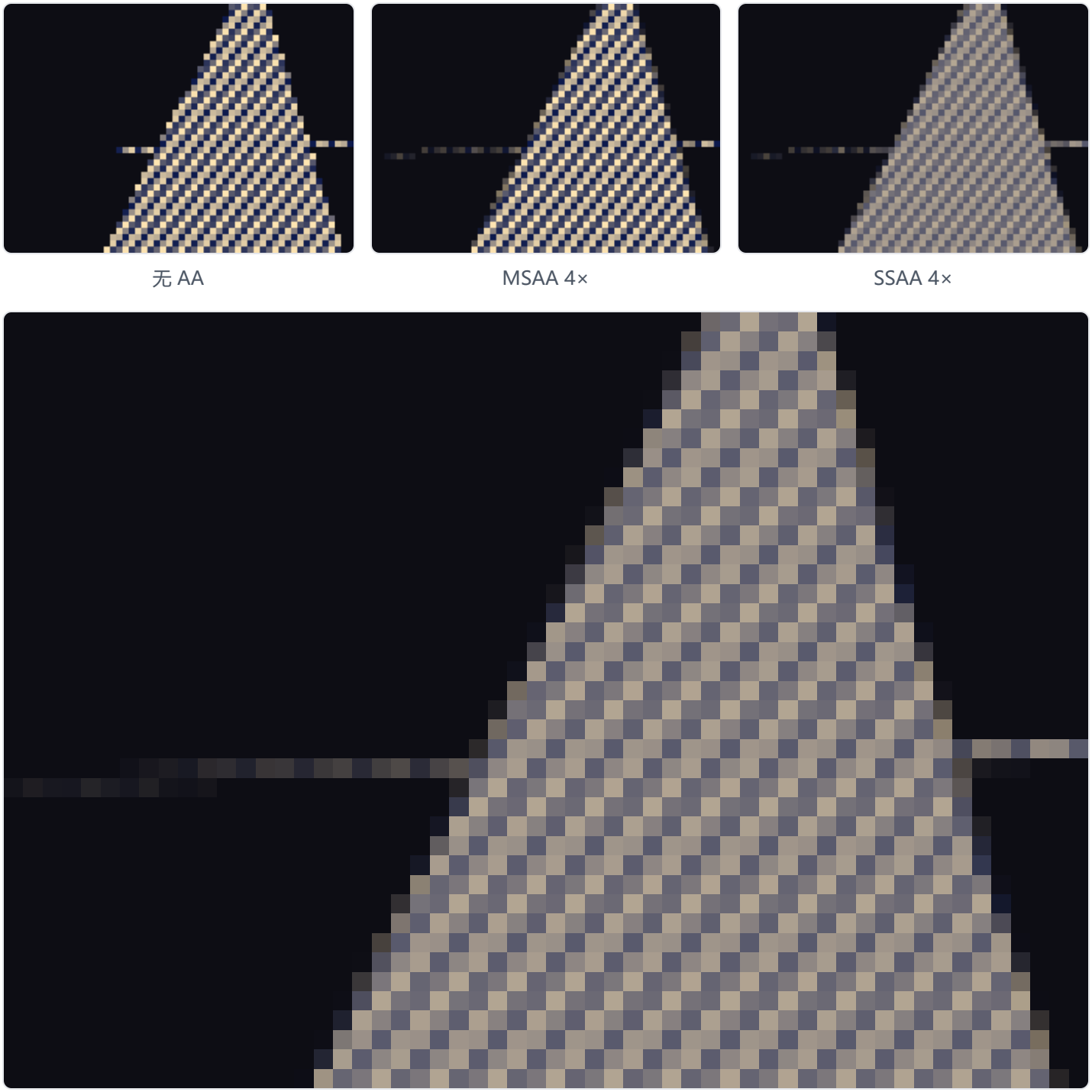


图 9-3　E10 局部放大 4 倍。MSAA 让三角形边缘变平滑，但内部的条纹和无 AA 一模一样（着色仍是每像素一次）；SSAA 同时平滑了边缘和内部条纹；最右为参考。

🔬 从输出里能读出什么

- **纯色几何（最后一列）上，MSAA 4× 与 SSAA 4× 的边缘误差完全相同**（0.1063）——对几何走样，两者等效；而 MSAA 的 PS 执行次数只比无 AA 多 5%（4946 vs 4705，多出来的是只被部分样本覆盖、但中心不在三角形内的边缘像素），SSAA 则是 4 倍。
- **内部着色走样 MSAA 毫无改善**（0.1683 与无 AA 相同），SSAA 降到 0.0067。这就是"MSAA 治不了高光闪烁和法线贴图闪烁"的直接证据。
- 两者的深度/颜色样本数都是 4 倍（18810）：**MSAA 省的是着色，不省存储和带宽**——在桌面 IMR 上这意味着 4 倍的帧缓冲带宽（第 10 章 E12：IMR 下 MSAA 4× 的流量是 TBR 的 134 倍）。

🧠 深入：MSAA 的几个细节

- **采样点位置**：4× 用旋转栅格（实测程序里的 `(-2,-6),(6,-2),(-6,2),(2,6)`/16），让接近水平和竖直的边也能得到 4 级灰度。
- **PS 在哪里求值**：默认在像素中心，即使中心不在三角形内（外插）；用 `centroid` 改为在被覆盖样本的重心处求值。
- **Alpha To Coverage**：把 PS 输出的 alpha 转成覆盖掩码（alpha=0.5 → 覆盖 2 个样本），让 Alpha Test 的树叶边缘也能被 MSAA 平滑，且不需要排序。
- **延迟渲染与 MSAA 不兼容**：G-Buffer 要按样本存储，带宽和显存爆炸；这是延迟渲染普遍用 TAA 的原因之一（第 11 章）。

🎮 游戏实战：移动端 MOBA 选哪种 AA

移动端 TBR 上 MSAA 很便宜：多重采样的颜色和深度只存在片上 Tile 内存里，Resolve 也在片上完成，只写回最终的 1× 结果（第 10 章）。所以前向渲染的手游通常首选 **MSAA 2×/4×**，并把 MSAA 渲染目标设为 memoryless（不分配系统内存）。注意如果中途需要把颜色或深度"拷出来"给后处理用（比如读深度做软粒子），就会触发 Resolve + 写回，MSAA 的优势就没了。TAA 在移动端则要考虑运动矢量 Pass 的开销和快速镜头移动下的拖影。

本章小结

- 输出合并顺序：Scissor → 模板 → 深度 → 混合 → 写掩码（硬件会把模板/深度尽量提前）。

- 模板缓冲是 8 位"像素标签"：描边、遮挡剪影、UI 遮罩、传送门、光源体积；引擎要统一分配模板位。
- Over 混合不满足交换律（E11：6 种顺序 6 种颜色），半透明必须**从后往前**、开深度测试、关深度写入；加法和乘法混合可交换，可免排序。
- 预乘 Alpha 让过滤和混合在数学上一致，消除黑边，还能把 Over 和 Additive 合并成一种混合状态。
- E10：MSAA 4× 对几何走样等效于 SSAA 4×，PS 只多 5%；但对着色走样无效，且存储/带宽是 4 倍。

▶ **自测：UI 为什么是移动端 Overdraw 的大户？怎么优化？**

▶ **自测：为什么开了 MSAA，角色身上的高光还是在闪？**

PART FIVE

第五部分　架构与工程：从逻辑管线到真实的一帧

前面讲的是"逻辑管线"。这一部分换到硬件和引擎的视角：同一条管线在桌面和手机上是怎么跑的、一帧里的多个 Pass 怎样配合、性能问题怎样定位。

CHAPTER 10

GPU 硬件视角：IMR 与 TBR / TBDR

桌面 GPU 拿到一个三角形就立刻画；手机 GPU 先把整帧的三角形分到小块里，再一块一块在片上画完。

本章目标

- 说清立即模式渲染（IMR）和分块渲染（TBR/TBDR）的流程差异
- 理解 TBR 为什么能大幅节省带宽，以及"带宽 = 功耗 = 发热"对移动端的意义
- 掌握 Load/Store Action、Memoryless、Framebuffer Fetch / Subpass 这些移动端专属概念
- 理解 TBDR 的隐藏面消除（HSR），以及为什么 Alpha Test 在移动端格外昂贵
- 知道 TBR 的代价：Binning 开销、中途读回帧缓冲导致的 Flush

10.1 两种架构

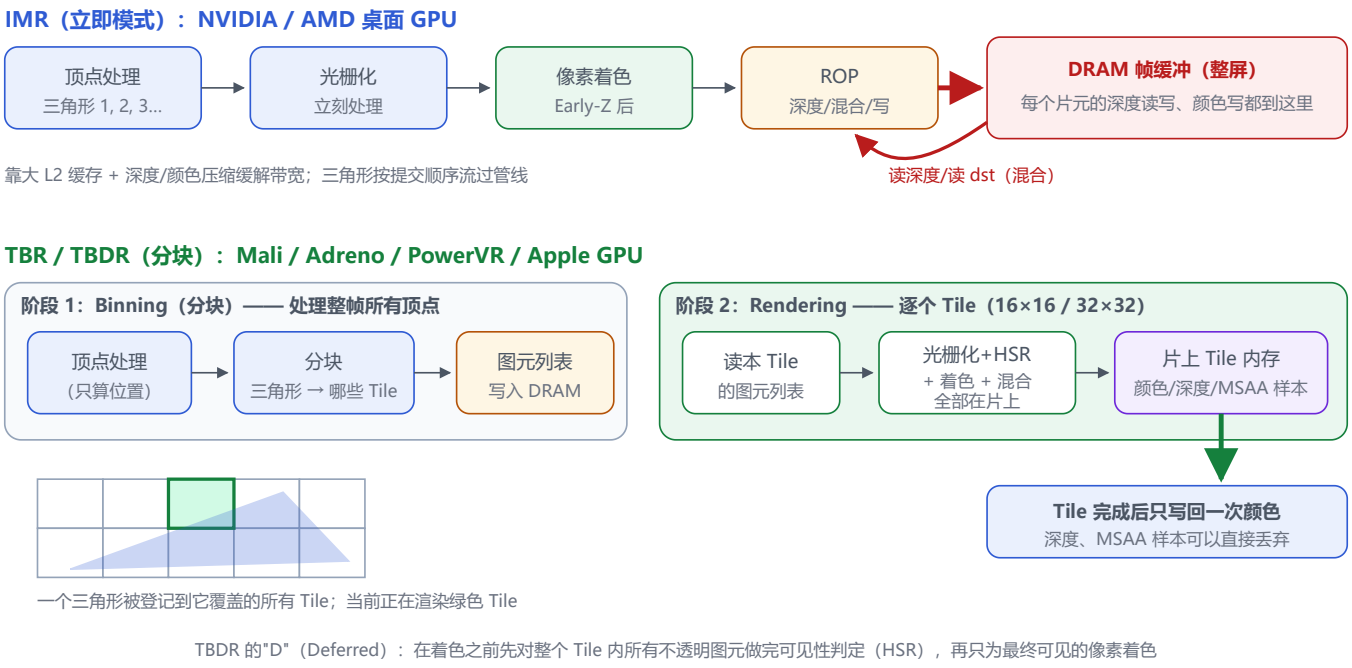


图 10-1　IMR 与 TBR/TBDR。关键区别：TBR 的深度测试、混合、MSAA 全部发生在片上，DRAM 只看到最终结果。

为什么手机要搞得这么复杂？因为**访问 DRAM 是移动 SoC 上最耗电的操作之一**：读写一次 DRAM 的能耗比片上 SRAM 高一到两个数量级，而手机的散热和电池都极其有限。桌面 GPU 有几百 GB/s 的独立显存和几百瓦的功耗预算，可以用大缓存和压缩硬扛；手机 GPU 只有几十 GB/s，还和 CPU 共享，于是选择从架构上减少带宽。

10.2 实测：一帧的显存流量

E12 用 E7 的统计数据（400 个随机方块、640×360）估算一帧的 DRAM 流量。模型很简化——IMR 每次深度读、深度写、颜色写都按 4 字节打到 DRAM，忽略缓存和压缩；TBR 只在每个 Tile 结束时写回 4 字节/像素的颜色：

`soft_raster.exe · E12（简化模型）`

画法（来自 E7）	IMR 流量 KB	TBR 流量 KB	IMR/TBR
Late-Z 随机顺序	29173	900	32.4x
Early-Z 随机顺序	29173	900	32.4x
Early-Z 前→后	25162	900	28.0x
Early-Z 后→前	70224	900	78.0x
Z-Prepass	50576	900	56.2x
MSAA4x Early-Z 随机（+Resolve）	121194	900	134.7x

按 60fps 换算：IMR(Early-Z 随机) ≈ 1709 MB/s，TBR ≈ 53 MB/s；移动端 DRAM 每 GB/s 约耗数百 mW，带宽就是功耗和发热。

🔬 **怎么读这组数据**

- 这是**上限估算**：真实 IMR 有 L2 缓存、深度压缩（Hi-Z、平面方程压缩）和颜色压缩（DCC），实际流量会小得多；真实 TBR 还有 Binning 阶段读写图元列表的流量。但量级关系是可靠的。
- Late-Z 和 Early-Z 随机顺序的 IMR 流量相同：深度测试的读取次数不变，Early-Z 省的是着色而不是深度带宽。
- **TBR 的流量与 Overdraw、绘制顺序、MSAA 都无关**——这些都在片上消化了。这就是为什么在 TBR 上 MSAA 4× 几乎免费，而 IMR 上 MSAA 的流量是 4 倍。
- 反过来，TBR 上 Overdraw 也不是零成本：它仍然消耗片上的 ALU 和纹理带宽（半透明片元每一层都要着色、都要采样纹理）。

10.3 Load / Store Action：告诉驱动哪些数据不用搬

TBR 在开始渲染一个 Tile 时，要决定 Tile 内存的初始内容；结束时要决定写不写回 DRAM。现代 API 让你显式声明：

动作	选项	带宽	什么时候用
Load（开始）	`Clear`	0（片上直接清）	每帧开始的颜色和深度（最常见）
	`DontCare`	0（内容未定义）	确定会被完全覆盖，如全屏后处理的目标
	`Load`	读整个 RT	需要在上一个 Pass 的结果上继续画
Store（结束）	`Store`	写整个 RT	后面还要用：最终颜色、要采样的阴影贴图
	`DontCare` / Memoryless	0	深度/模板（用完就扔）、MSAA 多采样缓冲（Resolve 后就不要了）

> ⚠️ **面试易错：每帧 Clear 是浪费吗?**
> 在 IMR 上，Clear 要写整个缓冲（虽然有快速清除优化）；在 TBR 上 **Clear 是最便宜的选择**——它只是把 Tile 内存初始化，反而是"不 Clear"（Load）要把上一帧内容从 DRAM 读进来。所以移动端的规范是：每个 Pass 开始时颜色和深度都 Clear（或 DontCare），结束时深度 DontCare。Unity 里"相机不清除"或"Clear Flags = Don't Clear"在移动端是性能陷阱。

Vulkan 的 **Subpass** 和 Metal 的 **Programmable Blending / Tile Shader**（GLES 的 `EXT_shader_framebuffer_fetch`）更进一步：让后一个 Pass 直接读取当前像素在 Tile 内存里的值（例如 G-Buffer），整个延迟渲染可以在一个 Tile 里完成、G-Buffer 永远不写回 DRAM（Memoryless G-Buffer）。

10.4 HSR：硬件替你做 Z-Prepass

TBDR（PowerVR 的 HSR、Apple GPU、Mali 的 FPK—Forward Pixel Kill、Adreno 的 LRZ 等机制类似）在一个 Tile 的所有图元光栅化完之后、着色之前，先做一次完整的可见性判定，**只为每个像素最终可见的那个不透明片元着色。**效果相当于 E7 里"Z-Prepass"的结果，却不需要你多画一遍——所以：

- 在 TBDR 上，不透明物体的前后排序收益变小（但仍建议大致排序，因为并非所有移动 GPU 的 HSR 都同样彻底，且 Binning 有容量上限）；
- 手动 Z-Prepass 在 TBDR 上通常是纯开销；
- **Alpha Test（discard）和写深度会打断 HSR**：硬件无法在着色前确定这个片元是否真的不透明，只能让它和它后面的东西都去着色。这就是"移动端少用 Alpha Test、多用 Alpha Blend 或者干脆用实体几何"的原因。

10.5 TBR 的代价与陷阱

Binning 开销

所有顶点要先处理一遍、结果（图元列表 + 变换后的顶点）写入 DRAM 再读回来。**顶点越多，这部分越贵**——TBR 对高面数场景比 IMR 更敏感。部分 GPU 只在 Binning 时计算位置（Position-only shading），把其他属性推迟到渲染阶段。

中途读回帧缓冲

在一个 Pass 中间去采样当前的颜色或深度（"抓屏"做扭曲、读深度做软粒子），会强制把所有 Tile 写回 DRAM 再重新开始（Flush / Resolve）。一帧里 RenderTarget 切来切去是移动端最常见的性能杀手。

巨大的三角形

一个全屏三角形会被登记到所有 Tile 里；大量覆盖全屏的半透明面片会让每个 Tile 的图元列表都很长。

Tile 内存容量

MRT 数量、像素格式（如 RGBA16F）、MSAA 倍数都会占用 Tile 内存；超过容量时驱动会自动缩小 Tile 尺寸，增加 Tile 数和开销。

10.6 游戏实战：移动端渲染清单

建议	原因（对应章节）
每个 RenderPass 开始时 Clear / DontCare，深度结束时 DontCare	避免 Load/Store 的整屏带宽（10.3）
减少 RenderTarget 切换，合并能合并的 Pass；后处理尽量合成一个 Uber Pass	每次切换都是一次整屏写回 + 读入（10.5）
MSAA 用 2×/4× 且多采样缓冲 Memoryless；避免在 MSAA Pass 中途读深度	片上 Resolve 几乎免费（9.6、10.2）
少用 Alpha Test；必须用时放在不透明之后绘制	打断 HSR / Early-Z（8.2、10.4）
控制半透明面积和层数（特效、UI）	Overdraw 消耗片上 ALU 和纹理带宽（10.2）
纹理用 ASTC + Mipmap；顶点属性压缩	纹理与顶点带宽（7.6、4.6）
精度用 half（mediump）	移动 GPU 的 FP16 吞吐通常是 FP32 的 2 倍，寄存器占用减半，Occupancy 提高（1.3）
控制顶点数与小三角形	Binning 开销与 Quad 效率（10.5、6.6）

🎮 游戏实战：一个 MOBA 后处理的例子

一个常见的反模式：Bloom 需要降采样链（1/2、1/4、1/8……），每级一个 RT；再加上屏幕扭曲（要抓屏）、颜色校正、FXAA，每帧切换十几次 RT。在 TBR 手机上，这可能比整个场景渲染还贵。优化方向：Bloom 用更少的级数、在低分辨率 RT 上完成；扭曲特效用"预先渲染到单独 RT"而不是抓取主屏；颜色校正（LUT）、暗角、FXAA 合并成一个最终 Pass；能用 Framebuffer Fetch 的效果就别抓屏。

本章小结

- **IMR**：三角形按提交顺序流过管线，深度/颜色直接读写 DRAM，靠大缓存和压缩扛带宽。
- **TBR/TBDR**：先 Binning 把整帧图元分到 Tile，再逐 Tile 在片上完成光栅化、深度、混合、MSAA，只写回最终颜色；TBDR 还在着色前做 HSR。
- E12 简化模型：IMR 流量是 TBR 的 28～78 倍，MSAA 4× 时 134 倍；TBR 的 DRAM 流量与 Overdraw、顺序、MSAA 无关。
- Load/Store Action 决定 Tile 内存的进出；Clear 和 DontCare 在 TBR 上最便宜；Memoryless 让深度和 MSAA 缓冲根本不占系统内存。
- TBR 的敌人：顶点过多（Binning）、中途读回帧缓冲（Flush）、Alpha Test（打断 HSR）、RT 频繁切换。

▶ **自测：同一个游戏，为什么 PC 上很流畅的"屏幕扭曲"特效在手机上掉帧严重?**

▶ **自测：TBR 是不是在所有场景下都比 IMR 好?**

CHAPTER 11

综合案例：一帧里的多个 Pass

把前十章的每个阶段串起来：先从光源"看"一遍场景写出阴影贴图，再从相机"看"一遍做光照——然后再看看大型项目怎样组织几十个 Pass。

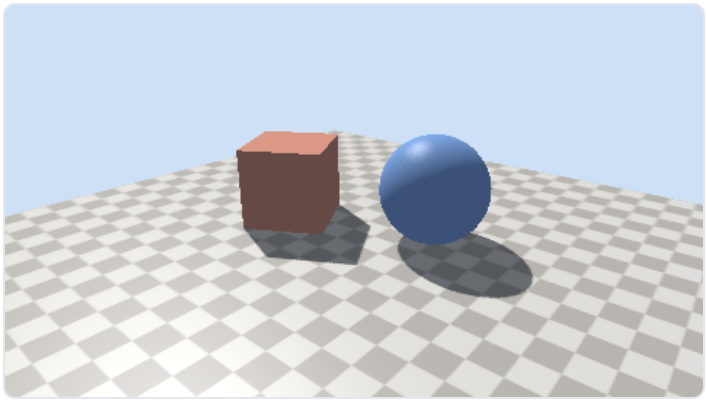
本章目标

- 完整走一遍 Shadow Map 两遍渲染，并能指出每一步用到了管线的哪个阶段
- 理解阴影痤疮（Shadow Acne）和彼得潘（Peter Panning）的成因，以及深度偏移、斜率缩放偏移、PCF 的作用
- 比较前向渲染、延迟渲染、Forward+ / Clustered 的结构和适用场景
- 能对着一个 RenderDoc 截帧说出每个 Pass 在做什么

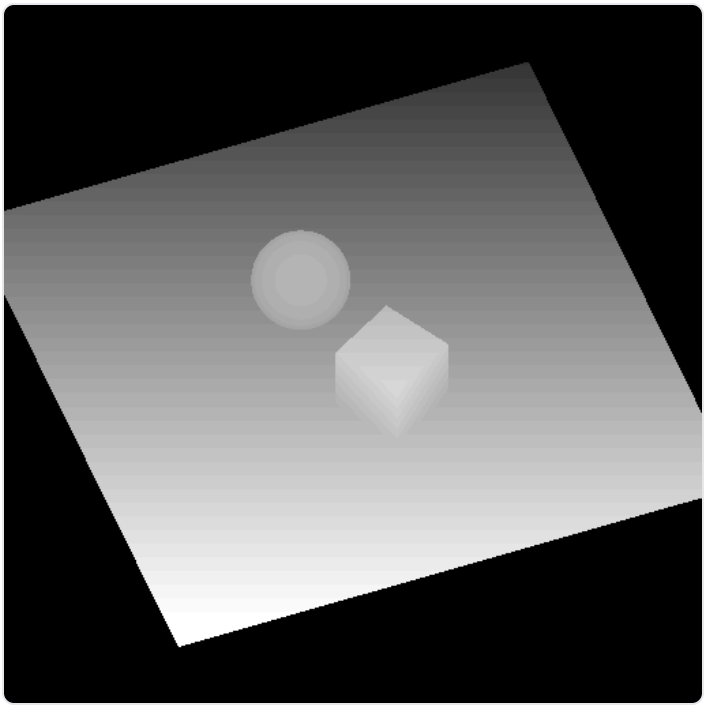
11.1 案例场景

E13 渲染一个最小但"五脏俱全"的场景：一块 12×12 米、贴了棋盘格纹理（带 Mipmap）的地面，一个旋转 30° 的立方体，一个球体，一盏平行光。两遍渲染：

Pass	相机	渲染目标	用到的管线阶段
1 · Shadow Pass	光源方向的**正交投影**（14m×14m，深度范围 30m）	1024×1024 深度贴图，**不写颜色**	VS → 裁剪 → 光栅化 → 深度测试/写入（无 PS）
2 · Main Pass	主相机透视投影（FOV 45°）	480×270 颜色 + 深度	VS → 裁剪 → 背面剔除 → 光栅化 → 透视校正插值 → Early-Z → PS（三线性采样 + Blinn-Phong + 阴影查询 + 3×3 PCF）→ 写入



最终结果（斜率缩放偏移 + 3×3 PCF）



Pass 1：光源视角的深度贴图（越亮越近）

图 11-1　E13 两遍渲染的输出。右图是从光源方向看到的深度，左图在主相机下查询它得到阴影。

11.2 Pass 1：从光源"看"场景

平行光没有位置，只有方向，所以用正交投影：把相机放在光源方向上足够远处，看向场景中心，视锥是一个能框住所有投射阴影物体的长方体。这一遍只需要深度：

soft_raster.cpp · E13 Pass 1（只写深度）

```
V3 toL = nrm({-0.8f, 1.0f, -0.3f});                          // 指向光源的方向
M4 lightVP = ortho(-7, 7, -7, 7, 0.1f, 30) * lookAt(toL * 15.f, {0, 0, 0}, {0, 1, 0});
Target SMT(SM, SM); SMT.clear({0, 0, 0, 1}, 1.f);
State sp; sp.colorWrite = false; sp.cull = false;            // 只写深度；不剔除背面（让薄物体也
drawMesh(SMT, sp, ground, mGround, lightVP, nullptr);        // fs = nullptr：没有像素着色器
drawMesh(SMT, sp, cube,   mCube,   lightVP, nullptr);
drawMesh(SMT, sp, sphere, mSph,    lightVP, nullptr);
```

✅ 对应到真实引擎

Shadow Pass 的 Shader 通常只有顶点着色器（或只有一个空 PS）——没有颜色输出、没有纹理采样，是最"纯粹"的几何负载。所以阴影 Pass 的开销主要取决于**投射阴影物体的顶点数和 Draw Call 数**，优化手段是：给远处物体关闭投影、用更低的 LOD 渲染阴影、对静态物体缓存阴影贴图。Alpha Test 的树叶在 Shadow Pass 里需要 PS 做 discard，会让这个 Pass 显著变贵。

11.3 Pass 2：查询阴影

主 Pass 的像素着色器拿到片元的世界坐标 P，用**同一个 lightVP** 把它变换到光源的裁剪空间，得到它在阴影贴图上的位置 (sx, sy) 和它到光源的深度 z，然后比较：

$$\text{lit} = (\, z - \text{bias} \le \text{ShadowMap}[sx, sy] \,) \;?\; 1 : 0$$

soft_raster.cpp · E13 阴影查询（含偏移与 PCF）

```
auto smLookup = [&](V3 P, V3 N, int mode, int pcf) -> float {
    V4 lp = lightVP * V4{P.x, P.y, P.z, 1};                          // 世界 → 光源裁剪空间（正
    float sx = (lp.x * 0.5f + 0.5f) * SM, sy = (0.5f - 0.5f * lp.y) * SM, z = lp.z;
    float ndl = std::max(dot(N, toL), 0.f);
    float tanT = std::sqrt(std::max(1 - ndl * ndl, 0.f)) / std::max(ndl, 0.05f);    // 表面与光
    float bias = /* mode 3: 斜率缩放 */ std::min(0.0002f + 0.0006f * tanT, 0.01f);
    int r = pcf ? 1 : 0; float lit = 0; int n = 0;
    for (int dy = -r; dy <= r; dy++) for (int dx = -r; dx <= r; dx++) {          // PCF: 3x3
        int ix = (int)floorf(sx) + dx, iy = (int)floorf(sy) + dy; n++;
        lit += (z - bias <= SMT.depth[(size_t)iy * SM + ix]) ? 1.f : 0.f;
    }
    return lit / n;
};
```

阴影痤疮与彼得潘

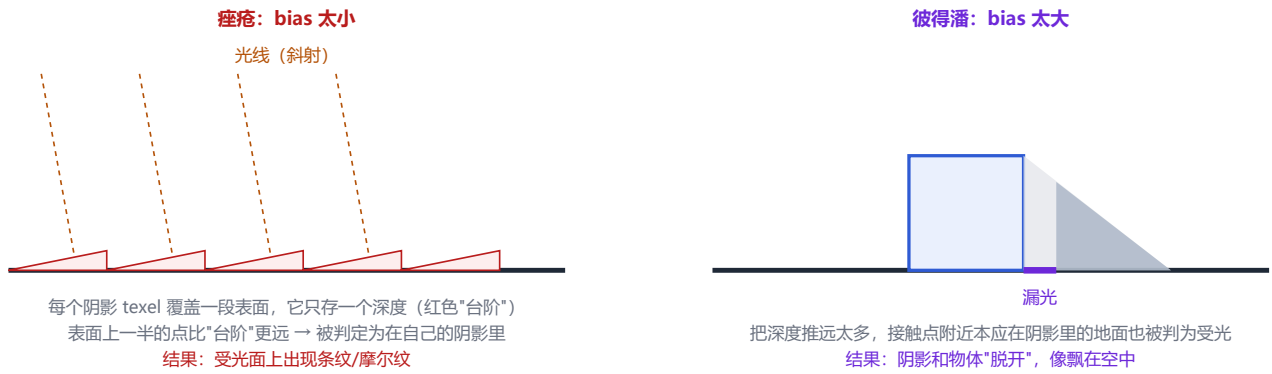


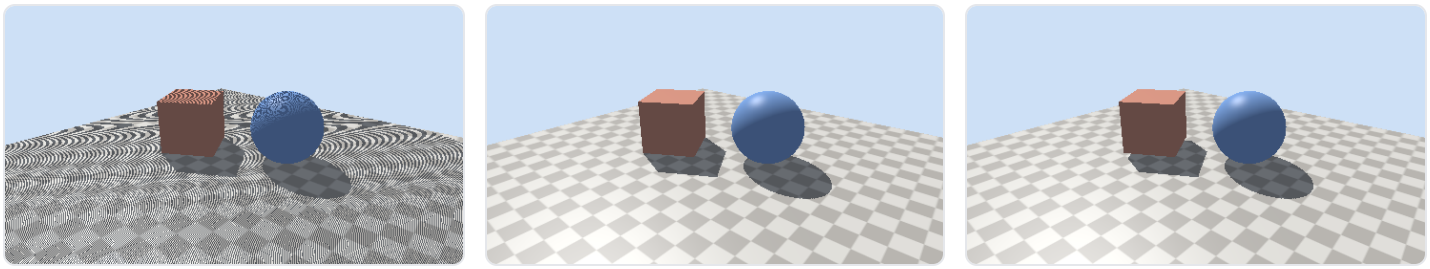
图 11-2 深度偏移的两难：太小出痤疮，太大出彼得潘。光线越斜，每个 texel 覆盖的表面深度差越大，需要的偏移越大——所以要"斜率缩放"。

soft_raster.exe · E13

光源方向 toLight=(-0.61,0.76,-0.23)，Shadow Map 1024x1024，正交范围 14m x 14m（1 texel ≈ 1.4cm），深度范围 29.9m

深度偏移	误判为阴影(痤疮+边缘)	漏光(误判受光)	受光面像素
bias = 0	46.65%	0.02%	68963
常量 bias = 0.0005 (≈1.5cm)	0.28%	0.03%	68963
常量 bias = 0.003 (≈9cm)	0.02%	0.03%	68963
斜率缩放 bias（0.0002+0.0006·tanθ）	0.02%	0.03%	68963
常量 bias = 0.02 (≈60cm)	0.02%	0.08%	68963

最终图（斜率缩放 bias + 3x3 PCF）：Shadow Pass 8.8 ms，Main Pass 9.4 ms；主 Pass 输入三角形 2348，剔除 1246，光栅化 1102，片元 84016，PS 执行 75801，深度剔除 8215



bias = 0（痤疮）　斜率缩放 bias　bias = 0.02（彼得潘）

图 11-3 E13 三种偏移下的渲染（无 PCF）。左：受光面布满摩尔纹；中：干净；右：立方体底部阴影与物体之间出现亮缝。

🔬 从输出里能读出什么

- "误判"是与**逐像素射线求交的解析真值**对比得到的。bias=0 时 46.65% 的受光像素被错判为阴影——几乎一半，正是图 11-2 的"一半在台阶下面"。
- 常量 bias 1.5cm 已经消除了大部分痤疮，9cm 基本干净；但 60cm 时漏光比例是最小 bias 时的 2.7 倍。这个场景里光线比较陡，所以常量 bias 和斜率缩放的差异不大；光线越斜（日出日落），常量 bias 的"两难"越明显。
- 主 Pass：2348 个三角形中背面剔除掉 1246 个（53%）；先画立方体和球、最后画地面，让地面被遮挡的 8215 个片元在 Early-Z 阶段就被丢弃——第 4、8 章的结论在这里同时生效。

🧠 深入：真实引擎里的阴影

- **硬件深度偏移**：D3D 的 `DepthBias / SlopeScaledDepthBias`（GL 的 `glPolygonOffset`）在光栅化阶段直接按三角形的深度斜率加偏移，Shadow Pass 里设置一次即可。另一种思路是 Normal Offset：查询时把 P 沿法线推出去一点。
- **PCF**：硬件支持"比较采样器"（`SampleCmp` / `sampler2DShadow`），一次采样就返回 2×2 比较结果的双线性加权，比手写 3×3 循环便宜得多。
- **级联阴影 CSM**：把主相机视锥按距离切成 2～4 段，每段一张阴影贴图，近处精度高、远处覆盖广。MOBA 视角固定、范围小，通常一级或两级就够。
- **MOBA 手游的折中**：英雄用实时 Shadow Map（甚至只给英雄投影），场景静态阴影烘焙进光照贴图；低端机退化为脚底的圆形"假阴影"贴片。

11.4 渲染路径：前向、延迟与 Forward+

场景里只有一盏光时，怎么画都行；当有几十上百盏动态光（技能光效、路灯、爆炸）时，"每个像素要算哪些光"就成了架构问题。

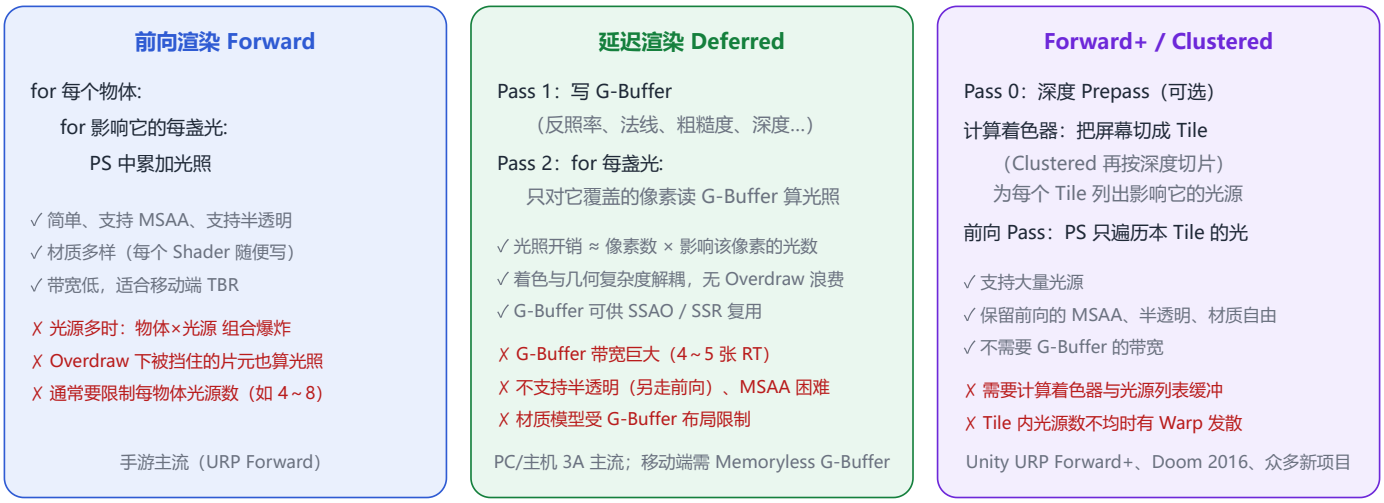


图 11-4　三种渲染路径。核心区别在于"光照计算和几何绘制是否解耦"以及"解耦的代价由谁来付"。

> ⚠️ **面试易错：延迟渲染"光源再多也不影响性能"**
>
> 不对。延迟渲染把光照开销从"物体数 × 光源数"降到了"像素数 × 每像素光源数"，但每盏光仍要读一遍它覆盖范围内的 G-Buffer（带宽），大量重叠的光源依然很贵——Tiled/Clustered Deferred 就是为此而生，让每个像素一次读 G-Buffer、在寄存器里累加所有光。另外，G-Buffer 本身的写入和读取是固定的高额带宽成本，这在移动端往往比光照计算更致命。

11.5 一帧的真实结构：读懂截帧

用 RenderDoc（PC/Android）、Xcode GPU Capture（iOS）、Snapdragon Profiler / Mali Graphics Debugger 截一帧，事件列表大致是这样的（以一款延迟渲染的 PC 游戏为例）：

顺序	Pass	本书对应	看什么
1	Depth Prepass	8.3	是否只写深度、物体是否从前往后
2	Shadow Depths（CSM 若干级 + 点光阴影）	11.2	投影物体数量、是否有不必要的投影者
3	G-Buffer / Base Pass	11.4	G-Buffer 格式与数量；Overdraw 视图
4	SSAO、接触阴影等屏幕空间效果	—	分辨率（常为半分辨率）
5	Lighting（Tiled / Clustered）	11.4	每 Tile 光源数热力图
6	天空、雾、体积光	8.6	天空是否在不透明之后、深度测试是否开启
7	Translucency（半透明、粒子）	9.4	排序、Overdraw、是否关闭深度写入
8	后处理：TAA → Bloom → 色调映射 → UI	9.6、10.6	RT 切换次数、各 Pass 耗时

对于一个移动端前向渲染的 MOBA，结构更简单：Shadow → Opaque（英雄/建筑/地面）→ Skybox → Transparent（技能特效）→ 少量后处理 → UI。第 12 章讲怎样在这样的截帧里找到瓶颈。

本章小结

- Shadow Map = 两遍渲染：Pass 1 从光源只写深度（无 PS），Pass 2 把片元变换到光源空间比较深度。
- 阴影 texel 覆盖一段表面却只存一个深度 → bias=0 时近一半受光像素误判（E13：46.65%）；bias 太大 → 阴影与物体脱开（漏光增加 2.7 倍）；光线越斜需要的 bias 越大 → 斜率缩放偏移。PCF 柔化边缘。
- 前向：简单、MSAA/半透明友好、移动端主流；延迟：光照与几何解耦，但 G-Buffer 带宽大、不支持半透明和 MSAA；Forward+/Clustered：用计算着色器做光源分块，兼顾两者。
- 读截帧时按 Pass 对应到管线阶段，逐个检查剔除、排序、Overdraw、RT 切换。

▶ **自测：Shadow Pass 为什么经常剔除正面而不是背面?**

▶ **自测：延迟渲染里半透明物体怎么画?**

CHAPTER 12

性能分析与优化：沿着管线找瓶颈

优化的第一步永远是"定位"：先知道慢在哪一站，再决定改什么。

本章目标

- 掌握"CPU 瓶颈 vs GPU 瓶颈"以及 GPU 内部"顶点 / 像素 / 带宽 / 纹理"瓶颈的判别方法
- 会用"一次只改一个变量"的实验法定位瓶颈
- 建立一张"管线阶段 → 常见问题 → 优化手段"的对照表
- 能完整讲述一个"团战掉帧"的排查过程

12.1 先分清：CPU 还是 GPU

回到第 1 章的帧流水：帧时间由最慢的泳道决定。判断方法：

观察	说明
主线程/渲染线程的 CPU 耗时 ≈ 帧时间，GPU 耗时明显更短	**CPU 瓶颈**：逻辑（脚本、物理、动画）或渲染线程（剔除、排序、Draw Call 提交）
GPU 耗时 ≈ 帧时间，CPU 线程有空闲（在等 Present / Fence）	**GPU 瓶颈**
降低渲染分辨率，帧率明显上升	GPU 像素相关瓶颈（像素着色、填充率、带宽）
降低分辨率帧率不变，减少物体数量帧率上升	CPU（Draw Call）或 GPU 顶点瓶颈

> ⚠️ **面试易错：手机上的"GPU 瓶颈"可能是发热**
>
> 移动设备在持续高负载下会降频（Thermal Throttling），前 5 分钟 60fps、之后 40fps 并不罕见。所以手游性能测试要看**长时间稳定帧率和功耗**，而不只是峰值帧率。降低带宽（第 10 章）往往是对功耗最有效的优化。

12.2 GPU 内部：一次只改一个变量

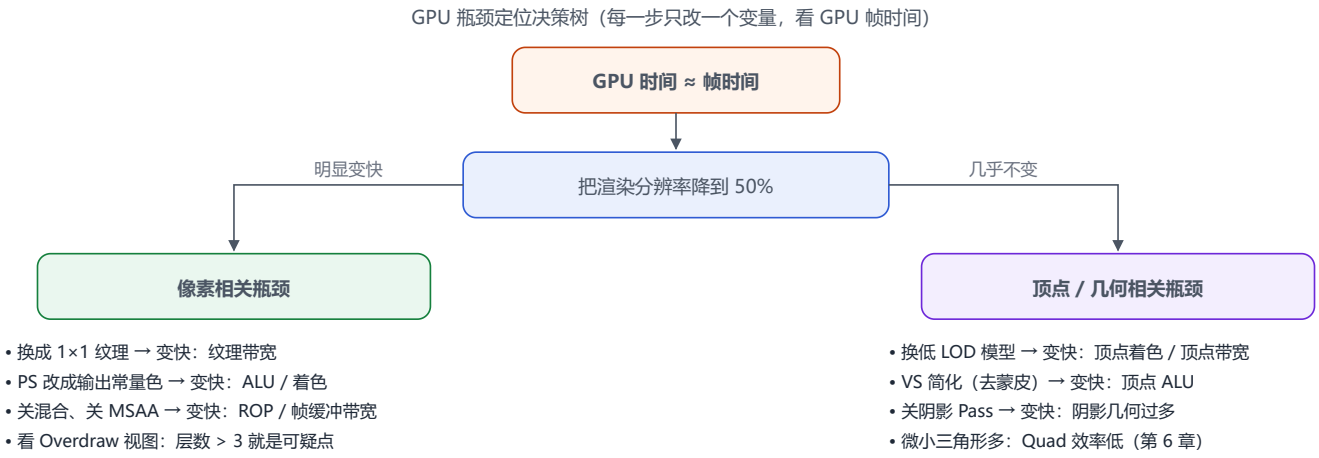


图 12-1 "改变一个变量"的定位法。真机上配合 GPU 厂商的硬件计数器（ALU 利用率、纹理缓存命中、带宽）会更精确。

RenderDoc

PC / Android 截帧：逐 Draw Call 看输入输出、状态、纹理、网格、Shader 调试，Overdraw / 四边形开销可视化。

Xcode GPU Frame Capture

iOS：每个 Pass 的耗时、带宽、Load/Store Action 检查（会直接提示不必要的 Load/Store）。

Snapdragon Profiler / Arm Mobile Studio

Adreno / Mali 的硬件计数器：Binning 时间、片上带宽、Shader 周期、纹理缓存命中率。

引擎内置

Unity Frame Debugger / Profiler、UE 的 `stat gpu`、`ProfileGPU`、Insights：先在引擎里定位到 Pass，再用厂商工具深挖。

12.3 管线阶段 × 常见问题 × 优化手段

阶段	症状	优化手段	本书
应用阶段（CPU）	渲染线程耗时高、Draw Call / SetPass 多	视锥/遮挡剔除；合批、GPU Instancing、SRP Batcher；减少材质与 Shader 变体；多线程录制命令	1.2
顶点处理	降分辨率无效、减面有效	LOD、减少顶点属性、压缩格式、顶点缓存优化（索引重排）、阴影用低模	4.2、4.6
图元装配/光栅化	大量微小/细长三角形	LOD 保证三角形面积；避免远处曲面细分；Nanite 式软光栅	5.5、6.6
像素着色	降分辨率有效、PS 简化有效	减少 Overdraw（排序、Prepass/HSR）；half 精度；把计算挪到 VS 或预计算到贴图；减少分支发散；动态分辨率	1.3、8.3
纹理	换小纹理有效	Mipmap、ASTC/BCn 压缩、减少采样次数、合并通道（Mask 图打包到 RGBA）、各向异性只开给地面	7.5、7.6
深度/模板	Early-Z 失效	避免 discard / 写深度；Alpha Test 物体后画；开 Reversed-Z 修 Z-Fighting	8.2、8.4
混合/输出	半透明 Overdraw 高、UI 面积大	控制特效面积与层数；粒子用低分辨率 RT 渲染再合成；UI 裁透明区、全屏界面关掉下层	9.4
帧缓冲带宽	手机发热、关 MSAA/后处理有效	Load/Store 用 Clear/DontCare；Memoryless；减少 RT 切换；合并后处理；MSAA 片上 Resolve	10.3、10.6

12.4 案例：MOBA 团战掉帧排查（典型流程示例）

下面是一个典型问题的排查思路（数字为示意，不同项目差异很大）：中端安卓机平时 60fps，十人团战放大招时掉到 35fps。

① 先分 CPU / GPU

引擎 Profiler 显示团战时渲染线程从 6ms 涨到 9ms（技能特效多了 Draw Call），但 GPU 帧时间从 14ms 涨到 28ms → **主要是 GPU 瓶颈**，CPU 的增量是次要问题。

② GPU 里分像素 / 顶点

把渲染分辨率从 100% 降到 70%，团战帧率从 35 升到 52 → **像素相关**。开 Overdraw 视图：大招范围内特效层数达到 15 ~ 20 层。

③ 定位具体资源

RenderDoc 截帧，按耗时排序 Draw Call：前 10 名中有 7 个是某两个英雄的大招特效——全屏大小的半透明面片、Additive + 扭曲（Grab Pass）、每层采样 3 张 512^2 纹理。扭曲特效还触发了一次额外的 RenderPass 切换（第 10 章）。

④ 对症优化

- 特效面片按实际形状裁剪（四边形 → 多边形网格），透明区域面积减少约一半；
- 同屏同类特效设上限，远离镜头中心的特效降级（减少层数和粒子数）；
- 扭曲改为只对英雄技能开启、低端机关闭，避免 Grab Pass；
- 特效纹理检查 Mipmap 和 ASTC 压缩，合并 Mask 通道减少采样次数；
- （可选）特效统一渲染到 1/2 分辨率 RT 再合成。

⑤ 验证与防回退

同一回放录像重跑，统计团战段的平均帧率、1% Low 帧率和功耗；把"特效 Overdraw 层数""单特效最大面积"做成资源规范和自动检查，防止新英雄再次引入。

> ✅ **面试答法**
>
> 面试官问"遇到掉帧怎么查"，按"**复现** → **CPU/GPU 归因** → **GPU 内部归因（改变量）** → **截帧定位到 Pass/Draw Call** → **对症优化** → **回归验证与规范化**"的顺序讲，并在每一步说出一个具体工具和一个具体判据。比直接罗列"合批、LOD、压缩纹理"更能体现工程能力。

12.5 几条容易被忽视的经验

- **Shader 变体爆炸**：每个 `multi_compile` 关键字翻倍变体数，导致包体、内存和首次加载编译卡顿（"Shader 编译卡顿"）。用 `shader_feature`、变体剥离、预热（Warmup）和 PSO 缓存。

- **精度**：移动端 PS 默认用 half，但世界坐标、UV 大范围平铺、时间累加量必须用 float，否则远处或长时间运行后出现抖动和条纹。
- **动态分辨率**：GPU 时间超标时自动降低 3D 渲染分辨率（UI 保持原分辨率），是对像素瓶颈最"兜底"的手段。
- **帧率稳定比峰值重要**：一次 50ms 的卡顿比平均帧率低 5fps 更伤体验；资源加载、Shader 编译、GC 都会造成尖峰。

本章小结

- 先分 CPU / GPU：看各线程耗时与 GPU 耗时，谁 ≈ 帧时间谁就是瓶颈。
- GPU 内部用"改一个变量"定位：降分辨率 → 像素相关；减面 → 顶点相关；换小纹理 → 纹理带宽；PS 输出常量 → ALU；关混合/MSAA → ROP 与帧缓冲带宽。
- 每个管线阶段都有对应的典型问题和优化手段（表 12.3），优化前要先对号入座。
- 排查流程：复现 → 归因 → 截帧定位 → 对症优化 → 回归验证 → 规范化防回退。

PART SIX

第六部分　面试：把知识变成口述答案

面试不是背书。这一部分把全书的内容整理成"30 秒口述版"、高频题参考答案、易错说法对照表和速记卡，并留出一组预留问题，供后续模拟面试逐题深挖。

CHAPTER 13

面试题库与速记

每题先给"一句话答案"，再给要点；能带上本书实测数字的地方，都标了出处。

13.1 "请介绍一下渲染管线"——30 秒 / 2 分钟版

30 秒版

渲染管线分三大段：**应用阶段**在 CPU 上做剔除、排序、合批并提交 Draw Call；**几何阶段**在 GPU 上由顶点着色器把顶点从局部空间经 MVP 变换到裁剪空间，然后硬件做裁剪、透视除法、视口变换和背面剔除；**光栅化与像素阶段**把三角形离散成片元、做透视校正插值，经过 Early-Z 后运行像素着色器（纹理采样和光照），最后在输出合并阶段做模板/深度测试和混合，写入帧缓冲。可编程的是顶点和像素着色器，其余是可配置的固定功能。

2 分钟版要额外点出的 5 个"深度信号"

1. **w 的意义**：投影矩阵把距离放进 w；裁剪必须在除以 w 之前做，否则相机背后的点会翻转（E2）。
2. **光栅化的精确性**：定点亚像素 + Top-Left 规则保证共享边不漏不重（E4）。
3. **透视校正**：1/w 在屏幕空间线性，属性要插值 a/w 和 1/w 再相除（E5）。
4. **Early-Z 与排序**：不透明从前往后画，PS 次数可降到可见像素数；discard 和写深度会破坏它（E7）。
5. **移动端 TBR**：深度、混合、MSAA 在片上完成，带宽是第一约束（E12）。

13.2 高频题与参考答案

A • 坐标变换与投影

Q1 MVP 分别做什么？为什么不一步到位？

M 把局部坐标变到世界，V 把世界变到以相机为原点的观察空间，P 把视锥映射到裁剪空间（齐次）。分开是因为每个空间各有用途：局部空间便于实例化和蒙皮，世界空间做光照物理，观察空间让投影公式简单。实际上 CPU 通常预乘成一个 MVP 传给 Shader，但世界坐标仍需要单独的 M。

Q2 为什么要用齐次坐标？w=0 和 w=1 分别表示什么？

为了用矩阵表示平移和透视。w=1 是点（受平移影响），w=0 是方向（不受平移影响）。透视时 w 存放距离，硬件除以 w 实现"近大远小"。

Q3 推一下透视投影矩阵。

相似三角形得 $x'=n\cdot x/(-z)$；令第 4 行 $(0,0,-1,0)$ 使 $w=-z$；x、y 按 $1/\tan(fov/2)$ 和宽高比缩放；z 行写成 $(0,0,A,B)$，由"$-n\to 0$、$-f\to 1$"解出 $A=f/(n-f)$、$B=nf/(n-f)$（D3D [0,1] 约定）。关键结论：z_{ndc} 与 $1/d$ 线性相关。（第 3.2 节）

Q4 法线为什么用逆转置矩阵？

要保持法线与切线垂直：$n^{T}t=0$，$t'=Mt$，设 $n'=Gn$，则需 $G^{T}M=I$，即 $G=(M^{-1})^{T}$。只有旋转和均匀缩放时可直接用 M。（第 2.5 节）

Q5 为什么在裁剪空间裁剪，而不是 NDC？

相机背后的点 $w<0$，除以 w 会翻转到屏幕另一侧；$w=0$ 会除零；而且裁剪空间与观察空间是线性关系，插值出的新顶点正确。E2：一个顶点在相机后 1 米，朴素除法把它投到屏幕上方，近平面裁剪后得到 4 个顶点、拆成 2 个三角形。实际硬件靠保护带几乎只裁近平面。（第 3.4、3.5 节）

Q6 D3D 和 OpenGL 在投影上有什么不同？

NDC 深度范围 D3D/Vulkan/Metal 为 [0,1]，GL 为 [−1,1]；Vulkan 的 NDC y 向下；纹理坐标原点 GL 在左下、其他在左上；传统上 D3D 左手系、GL 右手系（这只是约定，由矩阵决定）。（第 3.6 节）

B • 光栅化与插值

Q7 光栅化是怎么判断像素在三角形内的?

用三条边函数 E(p)=(b−a)×(p−a)，三个值同号则采样点在内。边函数是线性函数，可增量计算、可对整块像素快速接受/拒绝，三者除以面积就是重心坐标。（第 5.2 节）

Q8 两个三角形共享一条边，边上的像素会被画两次吗?

不会。硬件把顶点吸附到 8 位亚像素定点网格，用整数算边函数（精确、水密），并用 Top-Left 规则决定 E=0 的样本归谁。E4：顶点对齐像素中心的网格里 2131 个边上像素，"全收"重复 2131 次、"全不收"漏 2131 个，Top-Left 零误差；float 边函数在大坐标下会出现个别漏画/重复。（第 5.3、5.4 节）

Q9 什么是透视校正插值？为什么深度不需要?

屏幕空间里 1/w 和 a/w 是线性的，a 不是；所以插值 a/w 和 1/w 再相除。深度存的是 z/w，本身就是"除以 w"的形式，可直接线性插值。E5：不校正时地面 uv 平均误差 17 个纹理重复，校正后与射线求交真值一致。（第 6.2、6.3 节）

Q10 ddx/ddy 是怎么来的？为什么 PS 按 2×2 执行?

PS 以 2×2 Quad 为单位执行，导数就是 Quad 内相邻像素的差分；三角形外的像素作为 Helper Lane 陪跑。这让 Mipmap LOD 选择成为可能，代价是小三角形浪费：E8 中 1 像素三角形 Quad 效率 27%，细长三角形不到 50%。（第 6.5、6.6 节）

Q11 为什么小三角形对 GPU 不友好?

① Quad 效率低（边缘 Quad 半空）；② 很多三角形一个像素都不覆盖却走完了顶点和设置（E8：0.5 像素三角形一半零覆盖）；③ 光栅化器每周期处理的三角形数有上限；④ TBR 上 Binning 开销随三角形数增长。对策是 LOD，以及 Nanite 式的软光栅。

C • 深度、模板与可见性

Q12 什么是 Early-Z？什么时候失效?

把深度测试提前到 PS 之前，丢弃被挡住的片元。PS 写深度时完全失效；discard/Alpha Test、Alpha To Coverage 会推迟深度写入；写 UAV 需要显式声明；中途改变深度比较方向会让 Hi-Z 失效；从后往前画时剔不掉任何东西。（第 8.2 节）

Q13 不透明物体为什么要从前往后画？效果有多大？

让 Early-Z 尽早建立近处深度，后面被挡的片元不再着色。E7：400 个矩形、平均 Overdraw 26.7 层，Late-Z PS 执行 599 万次，Early-Z 随机顺序 74 万次，从前往后 22.4 万次（等于可见像素数），耗时 3764ms → 191ms；从后往前和 Late-Z 一样慢。（第 8.3 节）

Q14 Z-Prepass 有什么用？什么时候不该用？

先只写深度再用 EQUAL 着色，不依赖排序就能让 PS 次数等于可见像素数（E7：224506 次，248ms），深度还能被 SSAO 等复用。代价是多一遍几何。PS 便宜而顶点多的场景、以及移动端 TBDR（硬件 HSR 已经做了同样的事）不该用。

Q15 什么是 Z-Fighting？怎么解决？

两个面的深度差小于深度缓冲在该距离处的分辨率，前后关系随机。标准 Z 的精度集中在近处（near=0.1 时一半深度值用于前 0.2 米，1000 米处只能分辨 0.6 米）。解决：增大 near（最有效）、Reversed-Z + D32F、避免共面、深度偏移、贴花改用模板或延迟贴花。（第 8.4、8.6 节）

Q16 Reversed-Z 为什么能提高精度？24 位深度能用吗？

投影让远处的深度挤在一起，浮点数让可表示值挤在 0 附近；反转后远处映射到 0，两种不均匀互相抵消，相对精度近似恒定。E6：1000 米处可分辨距离从 0.6m 降到 0.00007m，8 公里内零 Z-Fighting。24 位定点是均匀的，反转无效。

Q17 模板缓冲能做什么？举三个游戏里的例子。

8 位像素标签。描边（先写模板再外扩画轮廓）、角色被墙遮挡时显示剪影、UI 遮罩、传送门/镜子、延迟渲染光源体积、阴影体。（第 9.2 节）

D • 纹理与像素

Q18 Mipmap 解决什么问题？代价是什么？

纹理缩小时一个像素覆盖很多 texel，只采一个点会走样（摩尔纹、闪烁）。Mipmap 预先滤波，按 LOD 选层。代价是多 1/3 内存；收益是走样消失且缓存命中率大幅提高（E9：80.8% → 99.3%）。它让远处**更模糊**而不是更清楚。

Q19 LOD 怎么计算？

由 Quad 差分得到 uv 的屏幕导数，乘纹理尺寸得到像素在纹理中的两个边长，取较长者 ρ，$LOD=\log_2\rho$。三线性在 floor 和 floor+1 两层之间插值。（第 7.3 节）

Q20 双线性、三线性、各向异性的区别?

双线性：一层内 2×2 加权（4 texel）；三线性：两层各一次双线性再插值（8 texel），解决 Mip 层切换的接缝；各向异性：斜视时像素脚印是细长的，按短边选 LOD、沿长边取多个样本。E9：全图误差 0.221 → 0.087 → 0.026，取样数 4 → 7.6 → 39.3。

Q21 discard 为什么在移动端特别贵?

它让硬件无法在着色前确定片元是否不透明：Early-Z 无法提前写深度，TBDR 的 HSR 被打断，被挡住的片元也得着色。移动端尽量用 Alpha Blend 或实体几何代替，必须用时放在不透明物体之后绘制。

Q22 Shader 里的 if 一定慢吗?

不一定。同一 Warp 走同一分支（uniform 分支，例如由材质参数决定）几乎无开销；按像素发散时两侧都要执行。另外分支里不能用依赖导数的 Sample()。（第 1.3、6.5 节）

E • 混合与抗锯齿

Q23 半透明物体为什么要从后往前画、关闭深度写入?

Over 混合不满足交换律（E11：三层 RGB 六种顺序得到六种颜色），正确的物理含义是近的盖在远的上面，所以从后往前；深度写入关闭，避免排序不完美时近处半透明把远处半透明整个挡掉。深度测试仍开启，被不透明物体挡住的不画。

Q24 物体互相穿插的半透明怎么办?

物体级排序必然有错。缓解：拆分网格、先背面后正面两个 Pass、深度预写只保留最外层；根本解决用 OIT：深度剥离、逐像素链表、加权混合 WBOIT（近似、单 Pass、移动端可用）。

Q25 什么是预乘 Alpha?

颜色预先乘以 alpha 存储，混合因子用 One / OneMinusSrcAlpha。它让过滤和混合在数学上一致，消除透明边缘的黑边（E11：直通 Alpha 结果 (0.25,0.75,0.25)，预乘 (0.5,1,0.5)），并能用同一混合状态表达 Over 与 Additive。

Q26 MSAA 和 SSAA 的区别?

SSAA 每个样本都着色；MSAA 每像素多个覆盖/深度样本，PS 只跑一次。对几何边缘两者等效（E10：纯色边缘误差都是 0.1063，MSAA 的 PS 只多 5%），但 MSAA 对着色走样无效（内部误差与无 AA 相同 0.1683）。两者的存储都是 N 倍。

Q27 为什么延迟渲染不好做 MSAA？TAA 的原理和缺点?

G-Buffer 要按样本存储和光照，带宽与显存乘以 N。TAA 每帧亚像素抖动投影矩阵，用运动矢量重投影历史帧并混合，等效于时间上的超采样；缺点是拖影、鬼影（需要邻域裁剪）、画面偏软、需要运动矢量 Pass。

F • 架构与性能

Q28 IMR 和 TBR 的区别?

IMR 来一个三角形画一个，深度和颜色直接读写显存；TBR 先 Binning 把整帧图元分到 Tile，再逐 Tile 在片上完成光栅化、深度、混合、MSAA，只写回最终颜色。TBDR 还在着色前做隐藏面消除。E12 简化模型：IMR 流量是 TBR 的 28～78 倍，MSAA 4× 时 134 倍。（第 10 章）

Q29 移动端渲染要注意什么?

带宽第一：Load/Store 用 Clear/DontCare、深度 Memoryless；减少 RT 切换和抓屏；MSAA 片上 Resolve；少用 Alpha Test；控制半透明面积；ASTC + Mipmap；half 精度；控制顶点数。（第 10.6 节）

Q30 Draw Call 为什么贵？怎么优化?

贵在 CPU 端的状态切换、资源绑定、驱动校验和提交。优化：静态合批（省 DC、费内存）、动态合批（CPU 变换顶点）、GPU Instancing（同网格大量重复）、SRP Batcher（不减 DC 但让每次 DC 更便宜）、减少材质和变体、现代 API 多线程录制。（第 1.2 节）

Q31 前向渲染和延迟渲染怎么选?

光源少、要 MSAA、半透明多、移动端带宽紧 → 前向（或 Forward+）；大量动态光源、复杂屏幕空间效果、PC/主机 → 延迟（Tiled/Clustered）。（第 11.4 节）

Q32 怎么定位 GPU 瓶颈?

先看 GPU 时间是否 ≈ 帧时间，再"改一个变量"：降分辨率有效 → 像素相关；换小纹理有效 → 纹理带宽；PS 输出常量有效 → ALU；关混合/MSAA 有效 → ROP/带宽；减面有效 → 顶点。然后截帧定位到具体 Pass 和 Draw Call。（第 12 章）

Q33 阴影痤疮和彼得潘是什么?

阴影贴图的一个 texel 覆盖一段表面却只存一个深度，bias 太小时表面一半判为自阴影（E13：46.65% 误判）；bias 太大时接触处漏光、阴影脱离物体（漏光增加 2.7 倍）。用斜率缩放偏移、Normal Offset、剔除正面、PCF 缓解。（第 11.3 节）

13.3 易错说法对照表

❌ 常见错误说法	✅ 更准确的说法
顶点着色器输出 NDC 坐标	输出**裁剪空间**齐次坐标，除以 w 由硬件在裁剪之后完成
深度缓冲存的是到相机的距离	存 z/w（与 1/d 线性相关），非线性；后处理要反算线性深度
背面剔除能剔掉一半三角形	与距离有关：近处可达 87%，远处趋近 50%（E3）；且不省顶点着色
Mipmap 让远处更清楚	让远处**更模糊**，目的是消除走样并提高缓存命中
MSAA 能解决所有锯齿	只解决几何边缘；高光/法线贴图闪烁（着色走样）无效
MSAA 很贵	IMR 上存储带宽 ×N；TBR 上片上 Resolve 几乎免费
Early-Z 一直生效	写深度、discard、A2C、UAV、比较方向改变时失效或受限
Z-Prepass 总是有益	多一遍几何；PS 便宜或 TBDR 上通常亏
Reversed-Z 对 24 位深度也有效	只对浮点深度有效
合批就是为了减少 Draw Call	本质是减少 CPU 端每次绘制的准备开销（状态切换、常量上传）
延迟渲染光源再多也不影响性能	每盏光仍要读覆盖区域的 G-Buffer；需要 Tiled/Clustered
行主序 = 行向量	一个是存储布局，一个是数学约定，彼此独立
每帧 Clear 是浪费	TBR 上 Clear 最便宜，Load 才要读 DRAM

13.4 预留问题：模拟面试题单

下面这些题在书中只点到为止，适合作为后续模拟面试的追问方向。每一题回答后会单独整理成一份 md 文档（放在 `Interview_QA/` 下），并按"点评 → 满分口述版 → 深度讲解 → 追问 → 速记卡"的格式补充。

#	题目	考察点
G01	从调用 DrawIndexed 到像素出现在屏幕上，完整讲一遍发生了什么（含驱动和交换链）	全局串联、CPU/GPU 同步
G02	手推透视投影矩阵，并说明为什么 z 不能直接用 −z_view	投影、深度插值
G03	为什么 1/w 在屏幕空间是线性的？给出证明	透视校正数学
G04	Hi-Z 和 Early-Z 的区别？GPU 驱动的遮挡剔除（Hi-Z Culling）怎么做？	可见性层次
G05	Alpha Test 和 Alpha Blend 各自的问题？草地/树叶怎么渲染最好？	Early-Z、HSR、A2C、排序
G06	MSAA 的 Resolve 发生在哪？延迟渲染下怎么做抗锯齿？	AA 与渲染路径
G07	TAA 的实现细节：抖动序列、运动矢量、历史裁剪、鬼影处理	时域方法
G08	级联阴影（CSM）的划分策略与抖动（Shimmering）问题	阴影工程
G09	PCF、PCSS、VSM、ESM 的区别	软阴影
G10	Forward+ / Clustered Shading 的光源分配是怎么做的？	计算着色器、光源剔除
G11	G-Buffer 怎么设计？法线如何压缩？移动端怎么做延迟渲染？	延迟渲染、带宽
G12	OIT 有哪些方案，各自的优缺点？	半透明
G13	Shader 变体是什么？变体爆炸怎么处理？	工程化
G14	GPU Instancing 的原理？和 SRP Batcher 能同时用吗？	CPU 优化
G15	Mesh Shader / Nanite 的思路是什么？为什么能处理海量三角形？	现代几何管线
G16	Occupancy 是什么？寄存器数怎么影响性能？	GPU 执行模型
G17	线性空间与 Gamma 空间：为什么光照要在线性空间算？sRGB 纹理和 sRGB RT 分别做了什么？	颜色管理
G18	HDR 与色调映射：为什么需要浮点 RT？移动端怎么省？	后处理、带宽

G19	描边有哪几种做法？各自优缺点？	模板、法线外扩、后处理边缘检测
G20	一个手游团战掉帧，你会怎么排查？（结合 TBR 讲）	综合性能分析

13.5 附录：速记卡

① 管线顺序

应用(CPU) → IA → VS → (HS/DS/GS) → 裁剪 → ÷w → 视口 → 背面剔除 → 光栅化 → 插值 → Early-Z → PS → 模板/深度 → 混合 → 帧缓冲

② 投影矩阵

第 4 行 (0,0,−1,0) → w=−z；z_ndc = a + b/d；D3D: A=f/(n−f), B=nf/(n−f)；Reversed: A=n/(f−n), B=nf/(f−n)

③ 裁剪

齐次空间：−w≤x,y≤w，0≤z≤w；先裁后除；硬件靠保护带，主要裁近平面（E2：3→4 顶点）

④ 光栅化

边函数同号即内；8 位亚像素定点；Top-Left：上边（水平向右）与左边（向上）包含 E=0（E4：零漏零重）

⑤ 透视校正

$a = \Sigma\lambda_i a_i/w_i \div \Sigma\lambda_i/w_i$；深度 z/w 直接线性插值（E5：误差 17 → 0.0001）

⑥ 2×2 Quad

导数 = Quad 差分；Helper Lane 陪跑；1px 三角形效率 27%，细长 48%（E8）

⑦ Mipmap

$LOD = \log_2(\max(|\partial uv/\partial x|, |\partial uv/\partial y|)\cdot size)$；+33% 内存；三线性 8 texel；各向异性按短边选层（E9）

⑧ Early-Z

失效：写深度、discard、A2C、UAV；从前往后 PS 次数=可见像素（E7：599 万 → 22 万）

⑨ 深度精度

精度 ∝ near/d^2；增大 near 最有效；Reversed-Z + D32F：1000m 处 0.6m → 0.07mm（E6）

⑩ 混合

Over 不可交换 → 从后往前、关 ZWrite；Additive 可交换；预乘 Alpha 消黑边（E11）

⑪ 抗锯齿

⑫ TBR

SSAA 全治全贵；MSAA 治边不治着色、PS ≈1×、存储 N×；FXAA 便宜偏糊；TAA 全面但拖影（E10）

Binning → 逐 Tile 片上渲染 → 只写颜色；Clear/DontCare/Memoryless；怕抓屏、RT 切换、Alpha Test、顶点多（E12）

⑬ 阴影

两遍：光源深度 → 比较；bias 小痤疮、大漏光；斜率缩放 + PCF（E13：46.65% → 0.02%）

⑭ 渲染路径

前向：简单/MSAA/半透明/移动端；延迟：多光源/G-Buffer 带宽；Forward+：计算着色器分块光源

⑮ 定位瓶颈

CPU/GPU → 降分辨率 → 换小纹理 → PS 输出常量 → 关混合/MSAA → 减面 → 截帧定位

⑯ Draw Call

贵在状态切换/绑定/驱动；静态合批、动态合批、Instancing、SRP Batcher 各省不同的东西
